

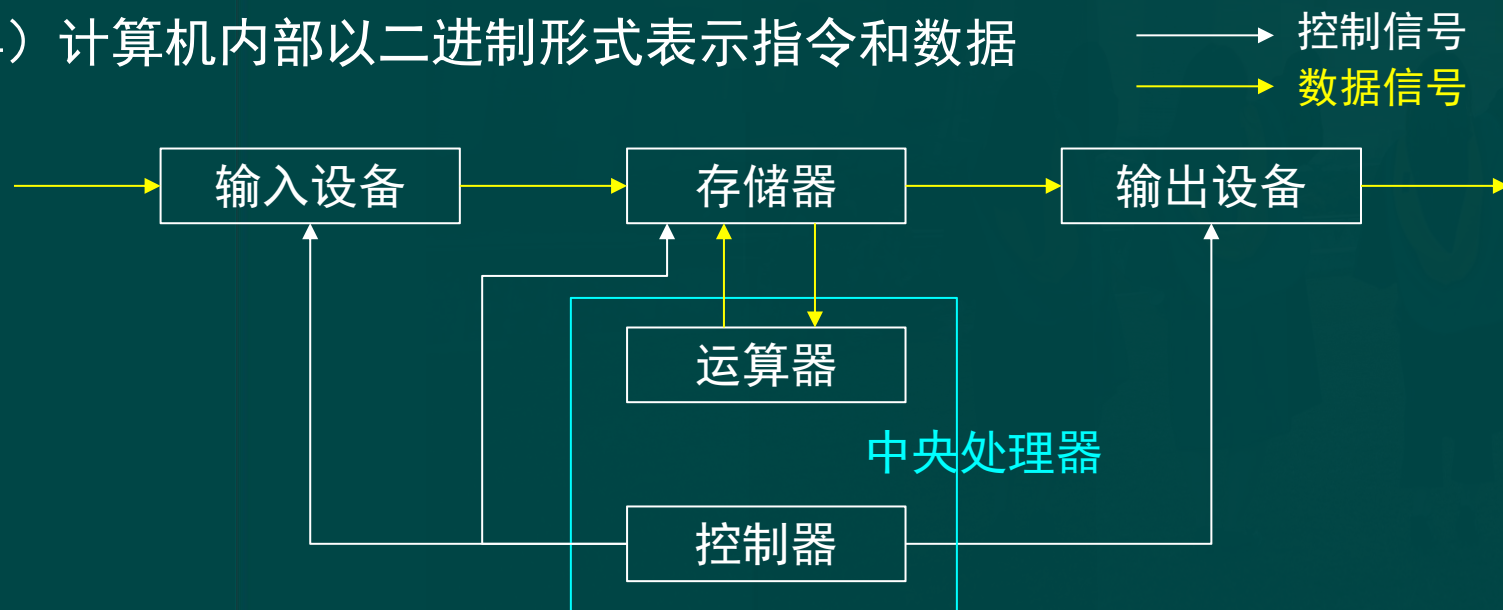
计算机系统原理

第一章 计算机系统概述

◎第一章 计算机系统概述

考点1 冯·诺依曼结构基本思想

- 1) 采用“存储程序”工作方式。
- 2) 计算机由运算器、控制器、存储器、输入设备和输出设备五大基本部件组成。
- 3) 存储器能存放数据，也能存放指令
- 4) 计算机内部以二进制形式表示指令和数据



◎第一章 计算机系统概述

考点2 程序和指令的执行过程

指令（instruction）是用0和1表示的一串0/1序列，用来指示CPU完成一个特定的原子操作，例如，

- 取数指令（load）从主存单元中取出数据存放到通用寄存器中；
- 存数指令（store）将通用寄存器的内容写入主存单元；
- 加法指令（add）将两个通用寄存器内容相加后送入结果寄存器，
- 传送指令（mov）将一个通用寄存器的内容送到另一个通用寄存器，如此等等。

◎第一章 计算机系统概述

考点2 程序和指令的执行过程

指令通常被划分为若干个字段，有操作码、地址码等字段。

- 操作码字段指出指令的操作类型，如取数、存数、加、减、传送、跳转等；
- 地址码字段指出指令所处理的操作数的地址，如寄存器编号、主存单元编号等。

◎第一章 计算机系统概述

考点3 程序设计语言和翻译程序

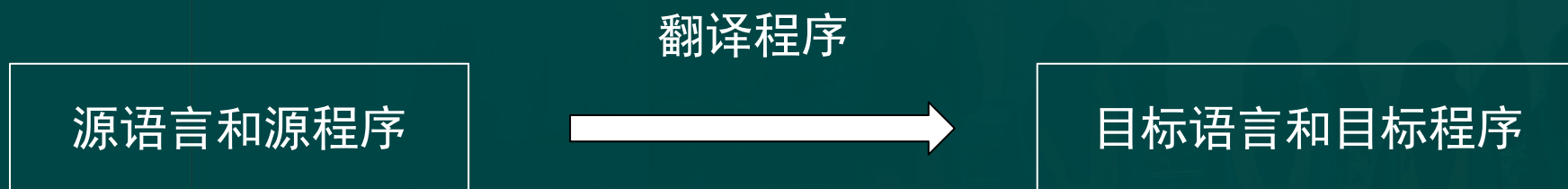
从抽象层次上来分，可以分成：

- 低级语言：机器语言，汇编语言
- 高级语言：其它的编程语言，如C，C++，JAVA，PYTHON等。

◎第一章 计算机系统概述

考点3 程序设计语言和翻译程序

因为计算机无法直接理解和执行高级编程语言程序，因而需要将高级语言程序转换成机器语言程序。这个转换过程通常由计算机自动完成，进行这种转换的软件统称为翻译程序（Translator）。



◎第一章 计算机系统概述

考点3 程序设计语言和翻译程序

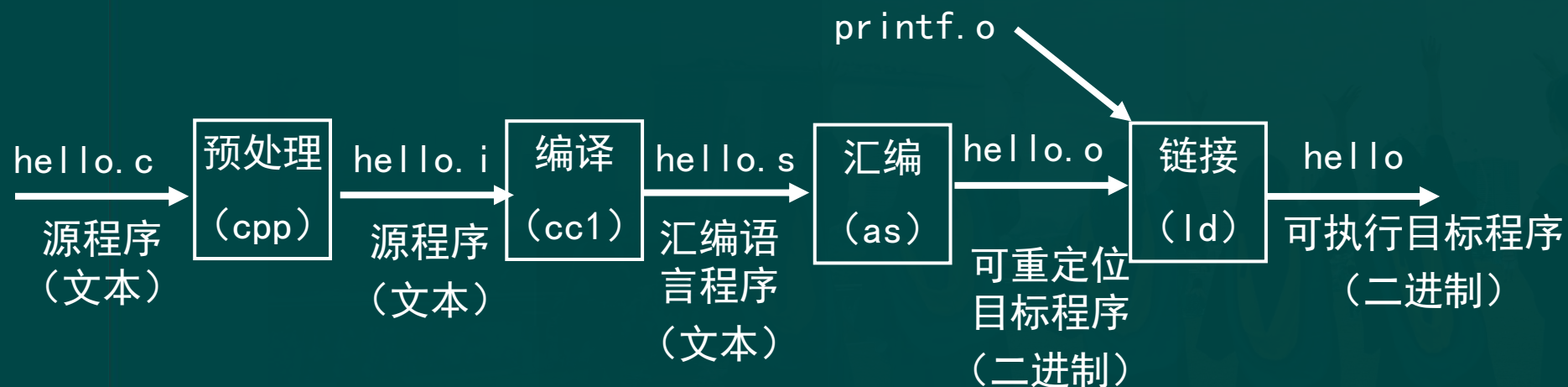
翻译程序有以下3类：

- 1) 汇编程序 (Assembler)：也称汇编器。用于将汇编语言源程序翻译成机器语言目标程序。
- 2) 解释程序 (Interpreter)：也称解释器。用于将源程序中的语句按其执行顺序逐条翻译成机器指令并立即执行，如PYTHON, BASIC等；
- 3) 编译程序 (Compiler)：也称编译器。用于将高级语言源程序翻译成汇编语言或机器语言目标程序，如C, C++语言。

◎第一章 计算机系统概述

考点4 从源程序到可执行文件

从hello.c到可执行目标文件hello的转换过程如图所示。



◎第一章 计算机系统概述

考点5 计算机系统抽象层的转换

计算机系统采用分层方式构建，即计算机系统是一个层次结构系统，通过向上层用户提供一个抽象的简清接口而将较低层次的实现细节隐藏起来。计算机解决应用问题的过程就是不同抽象层进行转换的过程。

最终用户希望计算机完成的应用（问题）到电子工程师使用器件完成基本电路设计的整个转换过程。



◎第一章 计算机系统概述

考点5 计算机系统抽象层的转换

- 指令集体系结构（Instruction Set Architecture, ISA）：计算机硬件和软件之间需要有一个“桥梁”；
- 实现ISA的电路逻辑结构称为计算机组织（Computer Organization）或微体系结构（Microarchitecture），简称微架构。
- 微架构中的功能部件由逻辑电路（Logic Circuit）实现，一个功能部件用不同的逻辑实现方式得到的性能和成本有差异，每个基本逻辑电路通过相应的器件技术（Device Technology）实现。

◎第一章 计算机系统概述

考点6 计算机系统的不同用户

计算机系统由硬件和软件两部分组成，

- 硬件（Hardware）是物理装置的总称，人们看到的各种芯片、板卡、外设、电缆等都是计算机硬件。
- 软件（Software）包括运行在硬件上的程序和数据以及相关的文档。程序（Program）是指挥计算机如何操作的一个指令序列，数据（Data）是指令操作的对象。根据软件的用途，一般将软件分成系统软件和应用软件两大类。

◎第一章 计算机系统概述

考点6 计算机系统的不同用户

系统软件（System Software）包括为有效、安全地使用和管理计算机以及为开发和运行应用软件而提供的各种软件，介于计算机硬件与应用程序之间，它与具体应用关系不大。

系统软件包括操作系统（如Windows、UNIX、Linux）、语言处理系统（如Visual Studio、GCC）、数据库管理系统（如Oracle）和各类实用程序（如磁盘碎片整理程序、备份程序）等软件。

◎第一章 计算机系统概述

考点6 计算机系统的不同用户

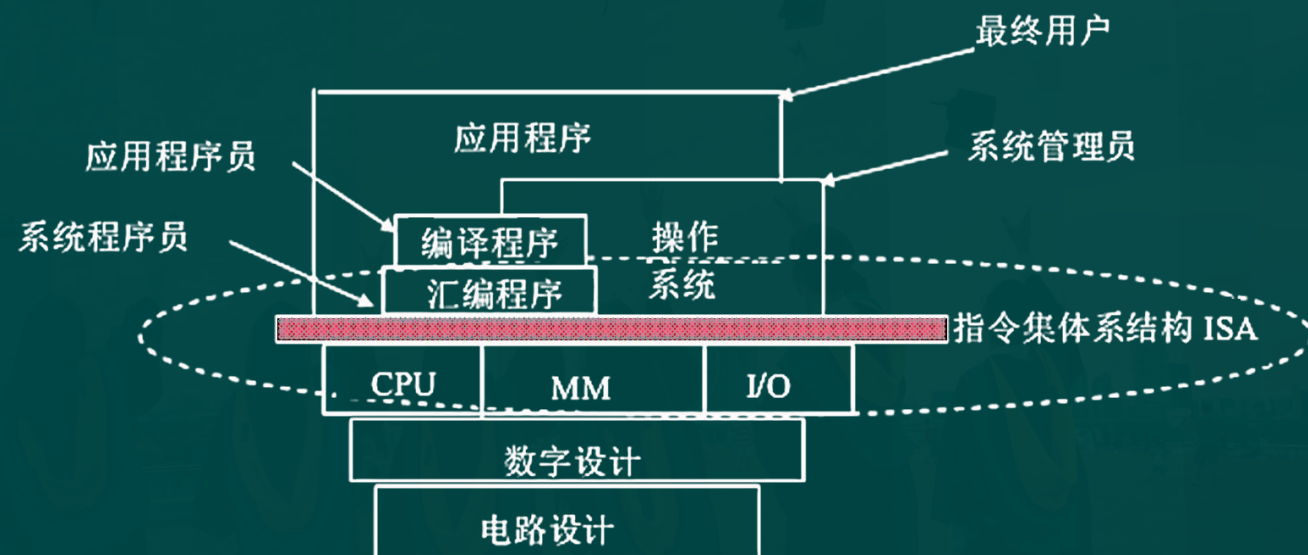
应用软件（Application Software）指专门为数据处理、科学计算、事务管理、多媒体处理、工程设计以及过程控制等应用所编写的各类程序。

◎第一章 计算机系统概述

考点6 计算机系统的不同用户

按照在计算机上完成任务的不同，可以把使用计算机的用户分成以下4类：最终用户、系统管理员、应用程序员和系统程序员。

在计算机技术中，一个存在的事物或概念从某个角度看似乎不存在，也即，对实际存在的事务或概念感觉不到，则称为透明。

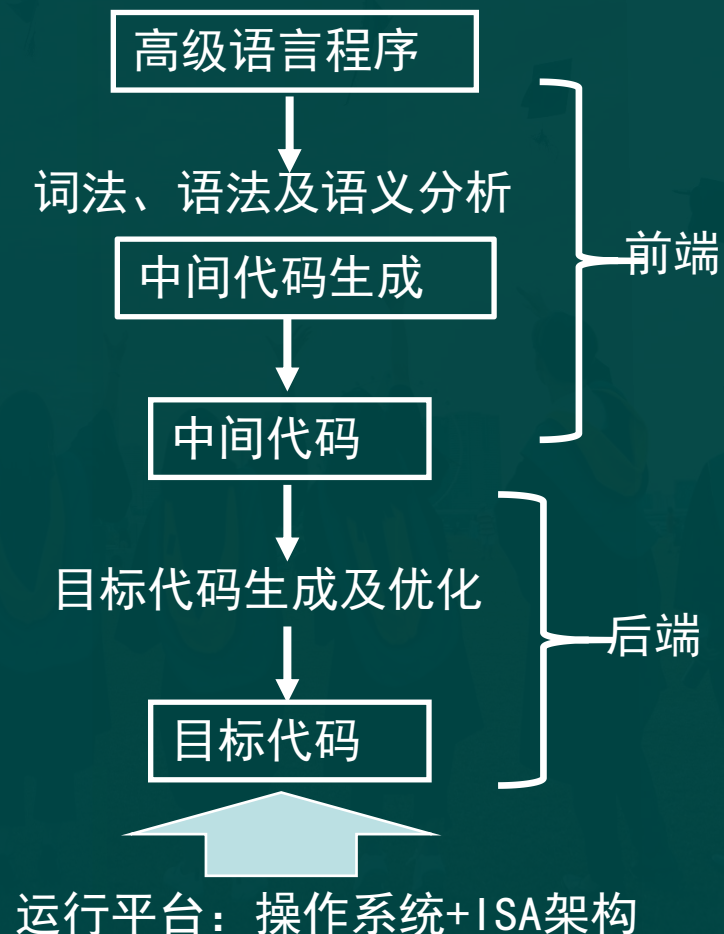


◎第一章 计算机系统概述

考点7 计算机系统核心层之间的关联

编译过程需要完成多个步骤，包括词法分析、语法分析、语义分析、中间代码生成、代码优化、目标代码生成和目标代码优化等。

程序转换的前端处理与高级编程语言标准规范密切相关，而后端处理则必须遵循ISA的规定和ABI规范。



◎第一章 计算机系统概述

考点7 计算机系统核心层之间的关联

1. 程序转换和编程语言标准之间的关系

每一种程序设计语言都有相应的标准规范，进行语言转换的编译程序前端必须按照编程语言标准规范进行设计。

程序执行结果不符合程序开发者预期的原因通常有两种。

- 一种是因为程序开发者不了解语言标准规范，
- 一种是程序开发者编写了含有未定义行为（Undefined Behavior）或未指定行为（Unspecified Behavior）或实现定义行为（Implementation-Defined Behavior）的源程序。

◎第一章 计算机系统概述

考点7 计算机系统核心层之间的关联

2. 目标代码与ISA和ABI规范之间的关系

编译程序的后端处理应根据ISA和应用程序二进制接口（Application Binary Interface, ABI）规范进行设计实现。

ABI是为运行在特定ISA及特定操作系统平台上的应用程序规定的一种机器级目标代码接口，包含了在生成特定平台上的目标代码时所必须遵循的一些约定。

3. ISA与硬件、ABI规范和操作系统之间的关系

在ISA层之上，操作系统向应用程序提供的运行时环境需要符合ABI规范，同时，操作系统也需要根据ISA规范来使用硬件提供的接口，包括硬件提供的各种控制寄存器和状态寄存器、原子操作、中断机制等。

◎第一章 计算机系统概述

考点8 计算机的性能测试

- 吞吐率表示在单位时间内所完成的工作量，类似的概念是带宽（Bandwidth），它表示单位时间内所传输的信息量。
- 响应时间是指从作业提交开始到作业完成所用的时间，类似的概念是执行时间（Execution Time）和等待时间（Latency），它们都是用来表示一个任务所用时间的度量值。

◎第一章 计算机系统概述

考点8 计算机的性能测试

通常把用户感觉到的执行时间分成以下两部分：

- CPU时间指CPU用于程序执行的时间，它又包括以下两部分：
 - （1）用户CPU时间，指真正用于运行用户程序代码的时间
 - （2）系统CPU时间，指为了执行用户程序而需要CPU运行操作系统程序的时间。
- 其他时间指等待I/O操作完成的时间或CPU用于执行其他用户程序的时间。

◎第一章 计算机系统概述

考点8 计算机的性能测试

计算机系统的性能评价主要考虑的是CPU性能。

系统性能和CPU性能不等价，两者有一些区别。

- 系统性能是指系统的响应时间，它与CPU外的其他部分也有关系
- CPU性能是指用户CPU时间，它只包含CPU运行用户程序代码的时间

◎第一章 计算机系统概述

考点8 计算机的性能测试

1. 时钟周期

计算机执行一条指令的过程被分成若干步骤，由每一步中相应的操作来完成指令功能。每一步操作都要有相应的控制信号进行控制，用于对控制信号进行定时的同步信号就是CPU的时钟信号，其宽度为一个时钟周期（clock cycle, tick, clock tick, clock）。

2. 时钟频率

CPU的主频就是CPU时钟信号的时钟频率（clock rate），是CPU时钟周期的倒数。

时钟频率的单位通常为MHz或GHz。

◎第一章 计算机系统概述

考点8 计算机的性能测试

3. CPI

CPI (cycles per instruction) 表示执行一条指令所需的时钟周期数。

由于不同指令的功能不同，因而执行不同指令所需的时钟周期数也不同，因此，对于一条特定指令而言，其CPI指执行该条指令所需的时钟周期数，此时CPI是一个确定的值；

◎第一章 计算机系统概述

考点8 计算机的性能测试

CPU执行时间=程序所含时钟周期数÷时钟频率
=程序所含时钟周期数×时钟周期

◎第一章 计算机系统概述

考点8 计算机的性能测试

如果已知程序总的指令条数和综合CPI，则可用如下公式计算程序的总时钟周期数。

程序总时钟周期数=程序所含指令条数 $\times$ CPI

◎第一章 计算机系统概述

考点8 计算机的性能测试

如果已知程序中共有n种不同类型的指令，第i种指令的条数和CPI分别为 C_i 和 CPI_i ，则

$$\text{程序总时钟周期数} = \sum_{i=1}^n (CPI_i \times C_i)$$

◎第一章 计算机系统概述

考点8 计算机的性能测试

程序的综合CPI也可由以下公式求得，其中， F_i 表示第*i*种指令在程序中所占的比例

$$\text{综合CPI} = \sum_{i=1}^n (CPI_i \times F_i)$$

=程序总时钟周期数/程序指令条数

程序总时钟周期数=程序所含指令条数×CPI

◎第一章 计算机系统概述

考点9 用执行速度进行性能评估

指令速度所用的计量单位为MIPS (Million Instructions Per Second)，其含义是平均每秒钟执行多少百万（ 10^6 ）条指令

相对MIPS (Relative MIPS) 是根据某个公认的参考机型来定义的相应MIPS值，其值的含义是被测机型相对于参考机型MIPS的多少倍。

◎第一章 计算机系统概述

考点10 用基准程序进行性能评估

基准程序（benchmarks）是进行计算机性能评测的一种重要工具。基准程序是专门用来进行性能评价的一组程序，能够很好地反映机器在运行实际负载时的性能，可以通过在不同机器上运行相同的基准程序来比较在不同机器上的运行时间，从而评测其性能。

基准程序最好是用户经常使用的一些实际程序，或是某个应用领域的一些典型的简单程序。

对于不同的应用场合，应该选择不同的基准程序。

第二章 数据的表示和运算

◎第二章 数据的表示和运算

考点1 信息的二进制编码

在计算机内部，所有信息都用二进制数字表示。这是因为：

- (1) 二进制只有两种基本状态，而使用有两个稳定状态的物理器件可以容易地表示二进制数的每一位；
- (2) 二进制的编码、计数和运算规则都很简单，可用开关电路实现，简便易行；
- (3) 两个符号“1”和“0”正好与逻辑命题的两个值“真”和“假”相对应，为计算机中实现逻辑运算和程序中的逻辑判断提供了便利的条件，特别是能通过逻辑门电路方便地实现算术运算。

◎第二章 数据的表示和运算

考点1 信息的二进制编码

指令所处理的基本数据类型分为两种：数值数据和非数值数据。

- 数值数据可用来表示数量的多少，可比较其大小，分为整数和实数。
- 整数又分为无符号整数和带符号整数。在计算机内部，整数用定点数表示，实数用浮点数表示。
- 非数值数据没有大小之分，不表示数量的多少，主要包括字符数据和逻辑数据。

◎第二章 数据的表示和运算

考点1 信息的二进制编码

采用十进制数表示数值数据的话，也要将十进制数编码成二进制数，即采用二进制编码的十进制数（Binary Coded Decimal Number，简称BCD）表示。

十进制数	8421码	5421码	2421码
0	0000	0000	0000
1	0001	0001	0001
2	0010	0010	0010
3	0011	0011	0011
4	0100	0100	0100
5	0101	1000	1011
6	0110	1001	1100
7	0111	1010	1101
8	1000	1011	1110
9	1001	1100	1111

◎第二章 数据的表示和运算

考点2 进位计数制

$$V(D) = d_n \times 10^n + d_{n-1} \times 10^{n-1} + \dots + d_1 \times 10^1 + d_0 \times 10^0 + d_{-1} \times 10^{-1} + d_{-2} \times 10^{-2} \dots + d_{-m} \times 10^{-m}$$

- d_i 可以是0, 1, 2, 3, 4, 5, 6, 7, 8, 9这十个数字符号中的任何一个, “10”称为基数 (base)
- 数位 i : 小数点的左边是0, 1, 2...; 右边是-1, -2...
- 10^i 表示位权
- 逢十往高位进一

◎第二章 数据的表示和运算

考点2 进位计数制

在计算机系统中，常用的几种进位记数制有下列几种。

二进制 $R=2$ ，基本符号为0和1。

八进制 $R=8$ ，基本符号为0，1，2，3，4，5，6，7。

十六进制 $R=16$ ，基本符号为0，1，2，3，4，5，6，7，8，9，A，B，C，D，E，F。

十进制 $R=10$ ，基本符号为0，1，2，3，4，5，6，7，8，9。

◎第二章 数据的表示和运算

考点2 进位计数制

书写方式：

- (1) 用下标2, 10, 8, 16进行表示
- (2) 使用后缀字母标识该数的进位数制，一般用B表示二进制，用O表示八进制，用D表示十进制（十进制数的后缀可以省略），而H则是十六进制数的后缀
- (3) 有时也在一个十六进制数之前用0x作为前缀

◎第二章 数据的表示和运算

考点2 进位计数制

1. R进制数转换成十进制数

任何一个R进制数转换成十进制数时，只要“按权展开”即可。

2. 十进制数转换成R进制数

任何一个十进制数转换成R进制数时，要将整数和小数部分分别进行转换。

(1) 整数部分的转换为

整数部分的转换方法是“除基取余，上右下左”。

(2) 小数部分的转换

方法是“乘基取整，上左下右”

◎第二章 数据的表示和运算

考点2 进位计数制

(4) 二进制和八进制，十六进制的转换

总结：

二进制	----->	八进制	三合一
二进制	----->	十六进制	四合一
八进制	----->	二进制	一拆三
十六进制	----->	二进制	一拆四

不够的，左右两边补0

◎第二章 数据的表示和运算

考点3 定点数的编码表示

定点数编码表示方法主要有以下4种：原码、补码、反码和移码。

- 将数值数据在计算机内部编码表示的数称为机器数，
- 机器数真正的值（即现实世界中带有正负号的数）称为机器数的真值。
- $X_T = \pm X_{n-2} \dots X_1 X_0$ （定点整数）
- $X_T = \pm 0.X_{n-2} \dots X_1 X_0$ （定点小数）

◎第二章 数据的表示和运算

考点3 定点数的编码表示

一个数的原码表示由符号位直接跟数值位构成，因此，也称“符号—数值”表示法。

符号（占1位）	数值（其余）
---------	--------

符号位为1：表示负数

符号位为0：表示正数

原码表示法中，正数和负数的编码表示仅符号位不同，数值部分完全相同。

优点是，与真值的对应关系直观、方便，因此与真值的转换简单。

缺点是零的表示不唯一，给使用带来不便，并且原码加/减运算规则复杂。

◎第二章 数据的表示和运算

考点3 定点数的编码表示

正数：反码=原码

负数：反码：最高位为1，其余就是对原码取反（0变成1，1变成0）

反码表示存在以下几个方面的不足：

0的表示不唯一；表数范围比补码少一个最小负数；运算时必须考虑循环进位。因此，反码在计算机中很少被使用，有时用作数码变换的中间表示形式。

◎第二章 数据的表示和运算

考点3 定点数的编码表示

补码的定义

- 正数的补码是它本身；
- 负数的补码等于模与该负数绝对值之差（也可以通过反码+1得到）

因此，数X，的补码可用如下公式表示。

- ①当X为正数时， $[X_T]_{\text{补}} = [X_T] = M + X_T \pmod{M}$ 。
- ②当X为负数时， $[X_T]_{\text{补}} = M - [X_T] = M + X_T \pmod{M}$ 。

综合①和②，得到以下结论：对于任意一个数

$$[X_T]_{\text{补}} = M + X_T \pmod{M}$$

◎第二章 数据的表示和运算

考点3 定点数的编码表示

从上述结果可知，补码0的表示是唯一的。优点：

- ①减少了+0和-0之间的转换。
- ②少占用一个编码表示，使补码比原码能多表示一个最小负数。

在n位原码定点数中，100...0用来表示-0，

在n位补码表示中，-0和+0都用00...0表示，而100...0用来表示最小负整数 -2^{n-1}

◎第二章 数据的表示和运算

考点3 定点数的编码表示

由补码，得到正真的方法，将下面过程反过来即可：

正数：原码=反码=补码

负数：数值部分，先取反后+1，或先-1，后取反

◎第二章 数据的表示和运算

考点3 定点数的编码表示

假设用来表示阶E的移码的位数为n，则

$[E]_{\text{移}} = \text{偏置常数} + E$ 。通常，偏置常数取 2^{n-1} 或 $2^{n-1}-1$ 。

数符	阶码E	尾数（用移码表示）
----	-----	-----------

◎第二章 数据的表示和运算

考点4 无符号整数的表示

二进制整数分为无符号整数（unsigned integer）和带符号整数（signed integer）两种。

一般在全部是正数运算且不出现负值结果的情况下，使用无符号整数表示。

n位无符号整数可表示的数的范围为 $0 \sim (2^n - 1)$ 。

◎第二章 数据的表示和运算

考点5 有符号整数的表示

带符号整数也被称为有符号整数，它必须用一个二进位来表示符号，虽然前面介绍的各种二进制定点数编码表示（包括原码、补码、反码和移码）都可以用来表示带符号整数。

例如，8位带符号整数的表示范围为 $-128 \sim +127$

◎第二章 数据的表示和运算

考点5 有符号整数的表示

但是补码表示有其突出的优点，主要体现在以下几方面。

- (1) 与原码和反码相比，数0的补码表示形式唯一。
- (2) 与原码和移码相比，补码运算系统是一种模运算系统，因而可用加法实现减法运算，且符号位可以和数值位一起参加运算。
- (3) 与原码和反码相比，它比原码和反码多表示一个最小负数。
- (4) 与反码相比，不需要通过循环进位来调整结果。

◎第二章 数据的表示和运算

考点6 浮点数的表示格式

任意的一个实数X，可以表示为

$$X = (-1)^s \times M \times R^E$$

其中S取值为0或1，用来决定数X的符号，一般用0表示正，1表示负；

M是一个二进制定点小数，称为数X的尾数；

E是一个二进制定点整数，称为数X的阶或指数；

R是基数，可以约定为2、4、16等。

要确定一个实数的值，只要在默认基数R下，确定数符S、尾数M和阶E就可以了

◎第二章 数据的表示和运算

考点6 浮点数的表示格式



第0位为数符S；

第1～8位为8位移码表示的阶码E（偏置常数为128）；

第9～31 位为24位二进制原码小数表示的尾数。

基数为2，规格化尾数形式为 $\pm 0.1bb\cdots b$ ，其中小数点后面第一位“1”不显式地表示出来，这样可用23个数位表示24位尾数。

◎第二章 数据的表示和运算

考点6 浮点数的表示格式

上述格式的规格化浮点数的表示范围如下。

正数最大值： $0.11\dots1 \times 2^{11\dots1} = (1-2^{-24}) \times 2^{127}$

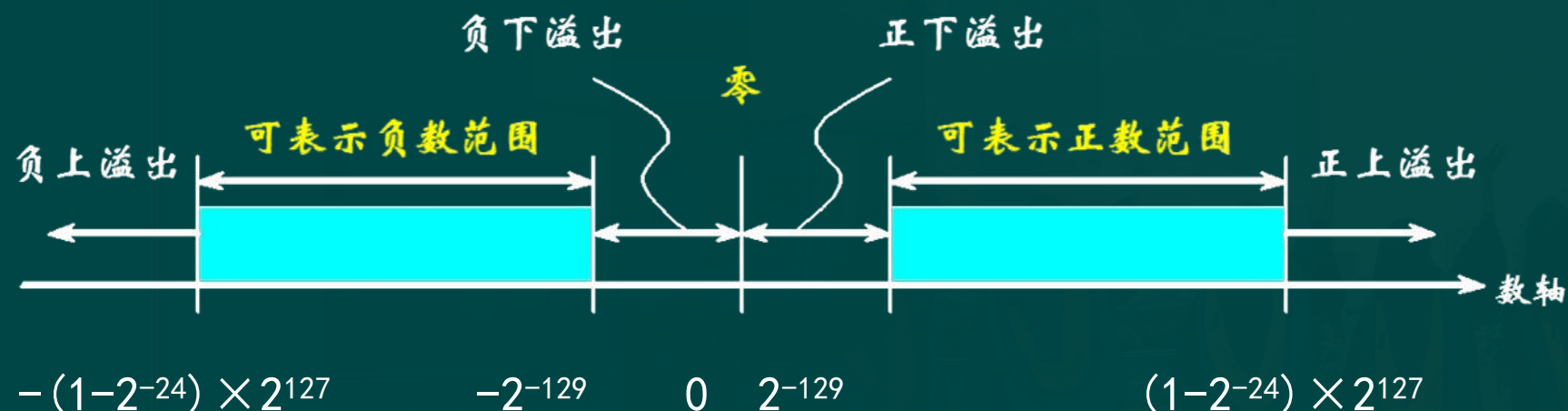
正数最小值： $0.10\dots0 \times 2^{00\dots0} = (1/2) \times 2^{-128} = 2^{-129}$

0	1	8	9	31
0	10010001	000	0000 1000 0011 0000 0000	

◎第二章 数据的表示和运算

考点6 浮点数的表示格式

因为原码是关于原点对称的，故该浮点格式的范围是关于原点对称的，如图所示



数轴上有4个区间的数不能用浮点数表示。这些区间称为溢出区，接近0 的区间为下溢区，向无穷大方向延伸的区间为上溢区。

◎第二章 数据的表示和运算

考点7 浮点数的规格化

若浮点数的基数为2，则尾数规格化的浮点数形式应为 $\pm 0.1bb\cdots b \times 2^E$ （这里b是0或1）。

规格化操作有两种：“左规”和“右规”。

左规

- 当有效数位进到小数点前面时，需要进行右规。右规时，尾数每右移一位，阶码加1，直到尾数变成规格化形式为止，右规时阶码会增加，因此阶码有可能溢出；

右规

- 当出现形如 $\pm 0.0\cdots 0bb\cdots b \times 2^E$ 的运算结果时，需要进行左规，左规时，尾数每左移一位，阶码减1，直到尾数变成规格化形式为止。

◎第二章 数据的表示和运算

考点8 IEEE 754浮点数标准

目前几乎所有计算机都采用IEEE 754标准表示浮点数。基数默认为2在这个标准中，提供了两种基本浮点格式：

32位单精度和64位双精度格式，如图

1位	8位	23位
符号S	阶码E	尾数M

1位	11位	52位
符号S	阶码E	尾数M

◎第二章 数据的表示和运算

考点8 IEEE 754浮点数标准

尾数用原码表示，默认第1位为1，称为隐藏位；

阶码用移码表示，偏置常数为 $(2^{n-1}-1)$ ，所以32位的移码为127，64位移码为1023，
因此32位阶码 $E=127+\text{阶}$ ，64位为 $E=1023+\text{阶}$

1位	8位	23位
符号S	阶码E	尾数M

1位	11位	52位
符号S	阶码E	尾数M

◎第二章 数据的表示和运算

考点8 IEEE 754浮点数标准

对于IEEE754标准格式的数，一些特殊的位序列（如阶码为全0或全1）有其特别的解释。
表给出了对各种形式的数的解释。以单精度为例。

值的类型	单精度（32）				值的类型	单精度（32）			
	符号	阶码	尾数	值		符号	阶码	尾数	值
正零	0	0	0	0	规格化非0正数	0	$0 < e < 255$	f	$2^{e-127}(1.f)$
负零	1	0	0	0	规格化非0负数	1	$0 < e < 255$	f	$-2^{e-127}(1.f)$
正无穷大	0	255（全1）	0	∞	非规格化正数	0	0	$f \neq 0$	$2^{-126}(0.f)$
负无穷大	1	255（全1）	0	$-\infty$	非规格化负数	1	0	$f \neq 0$	$-2^{-126}(0.f)$
无定义数	0或1	255（全1）	$\neq 0$	NaN					

◎第二章 数据的表示和运算

考点9 C语言中的浮点数类型

当在int、float和double等类型数据之间进行强制类型转换时，程序将得到以下数值转换结果（假定int为32位）。

- （1）从int转换为float时，不会发生溢出，但可能有有效数字被舍入。
- （2）从int或float转换为double时，因为double的有效位数更多，故能保留精确值。
- （3）从double转换为float时，因为 float 表示范围更小，故可能发生溢出，此外，由于有效位数变少，故可能被舍入。

◎第二章 数据的表示和运算

考点9 C语言中的浮点数类型

(4) 从 float 或 double 转换为 int 时，因为 int 没有小数部分，所以数据可能会向 0 方向被截断。

例如，1.9999 9 被转换为 1，-1.999 9 被转换为 -1。

此外，因为 int 的表示范围更小，故可能发生溢出。将大的浮点数转换为整数可能会导致程序错误

◎第二章 数据的表示和运算

考点10 逻辑值

当数据以这种方式看待时，就被认为是逻辑数据。

因此 n 位二进制数可表示 n 个逻辑值。逻辑数据只能参加逻辑运算，并且是按位进行的，如按位“与”、按位“或”、逻辑左移、逻辑右移等。

◎第二章 数据的表示和运算

考点11 西文字符

美国标准信息交换码 (American Standard Code for Information Interchange)
ASCII 字符编码共占7位；一般采用1B来存储，对于最高位可以设置为0，也可以设置奇偶检验码，此时最高位为奇偶检验位。

常用的字符ASCII十进制值为：

空格：32

数字：0为48，1为49...

大写字母：A为65，B为66...

小写字母：a为97，b为98...

◎第二章 数据的表示和运算

考点12 汉字字符

1. 汉字的输入码，又称外码。

每个汉字用一个或几个键来表示，这种对每个汉字用相应的按键进行的编码表示就称为汉字的“输入码”，又称外码。

目前使用的汉字输入码有基于读音的拼音码和基于字形的五笔码等。

◎第二章 数据的表示和运算

考点12 汉字字符

2. 字符集与汉字内码

汉字被输入到计算机内部后，就按照一种称为“内码”的编码形式在系统中进行存储、查找、传送等处理。

对于西文字符，它的内码就是ASCII码。

◎第二章 数据的表示和运算

考点12 汉字字符

区位码

码表由94行（区号0~93，）、94列（位号0~93）组成。每一个汉字或符号在码表中都有各自的位置，因此各有一个唯一的位置编码，区号和位号各占一个字节。

◎第二章 数据的表示和运算

考点12 汉字字符

国标码

1981年我国颁布了《信息交换用汉字编码字符集基本集》（GB2312—80）。该标准选出6763个常用汉字，为每个汉字规定了标准代码，以供汉字信息在不同计算机系统之间交换使用。这个标准称为国标码，又称国标交换码。

国标码= 区位码 + 2020H

◎第二章 数据的表示和运算

考点12 汉字字符

机内码

汉字在存储和处理时，使用的编码，称为机内码，简称内码；

将国标码两个字节的最高位（b，）置“1”；

应当注意，汉字的区位码和国标码是唯一的、标准的，而汉字内码可能随系统的不同而有差别。

机内码 = 8080H + 国标码 = A0A0H + 区位码

◎第二章 数据的表示和运算

考点12 汉字字符

字形码

汉字在显示和打印时，使用的编码；

汉字的字形主要有两种描述方法：字模点阵描述和轮廓描述。

◎第二章 数据的表示和运算

考点12 汉字字符

字模点阵描述是将字库中的各个汉字或其他字符的字形（即字模），用一个其元素由“0”和“1”组成的方阵（如 16×16 、 24×24 、 32×32 甚至更大）来表示，汉字或字符中有黑点的地方用“1”表示，空白处用“0”表示，我们把这种用来描述汉字字模的二进制点阵数据称为汉字的字模点阵码。

汉字的轮廓描述方法比较复杂，它把汉字笔画的轮廓用一组直线和曲线来勾画，记下每一直线和曲线的数学描述公式。

◎第二章 数据的表示和运算

考点13 数据的宽度和单位

二进制数据的每一位（0或1）是组成二进制信息的最小单位，称为一个“比特”（bit），或称“位元”，简称“位”。

比特是计算机中处理、存储和传输信息的最小单位。

计算机中运算和处理二进制信息时使用的单位除了比特和字节之外，还经常使用“字”（word）作为单位。

必须注意，对于不同的计算机，字的长度可能不同。

◎第二章 数据的表示和运算

考点13 数据的宽度和单位

在考察计算机性能时，一个很重要的性能参数就是机器的“字长”。

所谓“字长”通常是指CPU内部用于整数运算的数据通路的宽度。

“字长”等于CPU内部用于整数运算的运算器位数和通用寄存器宽度。

◎第二章 数据的表示和运算

考点13 数据的宽度和单位

存储容量的单位主要有

1KB=1024字节= 2^{10} 字节;

1MB= 2^{10} KB;

1GB= 2^{10} MB;

1TB= 2^{10} GB;

1PB= 2^{10} TB;

1EB= 2^{10} PB;

1ZB= 2^{10} EB。

◎第二章 数据的表示和运算

考点14 数据的存储和排列顺序

字节为一个排列基本单位，那么 LSB表示最低有效字节（Least Significant Byte），MSB表示最高有效字节（Most Significant Byte）。

◎第二章 数据的表示和运算

考点14 数据的存储和排列顺序

如图所示，根据数据中各字节在连续存储单元中的排列顺序的不同，可有大端和小端两种排列方式。

	0800H	0801H	0802H	0803H		
大端方式	...	01H	23H	45H	67H	...

	0800H	0801H	0802H	0803H		
小端方式	...	76H	45H	23H	01H	...

◎第二章 数据的表示和运算

考点15 全加器和加法器

的算术逻辑部件（简称 ALU）来完成基本逻辑运算和定点数加减运算；
各类定点乘除运算和浮点数运算可利用加法器或ALU和移位器来实现

◎第二章 数据的表示和运算

考点16 带标志加法器

在无符号数加法器的基础上增加相应的逻辑门电路，使得加法器不仅能计算和/差，还要能够生成相应的标志信息。

溢出标志的逻辑表达式为 $OF = C_n \oplus C_{n-1}$ ；

符号标志就是和的符号，即 $SF = F_{n-1}$ ；

零标志 $ZF = 1$ 当且仅当 $F=0$ ；

进位/借位标志 $CF = Cout \oplus Cin$ ，即当 $Cin=0$ 时， CF 为进位 $Cout$ ，当 $Cin=1$ 时， CF 为进位 $Cout$ 取反。

◎第二章 数据的表示和运算

考点17 补码加减运算器

$[x+y]_{\text{补}}$ 和 $[x-y]_{\text{补}}$ 的运算表达式如下：

$$[x+y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}} \quad (\text{mod } 2^n)$$

$$[x-y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} \quad (\text{mod } 2^n)$$

求一个数的负数的补码由其补码“各位取反、末位加1”得到

◎第二章 数据的表示和运算

考点17 补码加减运算器

该电路可实现补码加减运算。

当控制端Sub为1时，做减法，实现 $X + \bar{Y} + 1 = [x]_{\text{补}} + [-y]_{\text{补}}$ ；

当控制端Sub为0时，做加法，实现 $X + Y = [x]_{\text{补}} + [y]_{\text{补}}$ 。

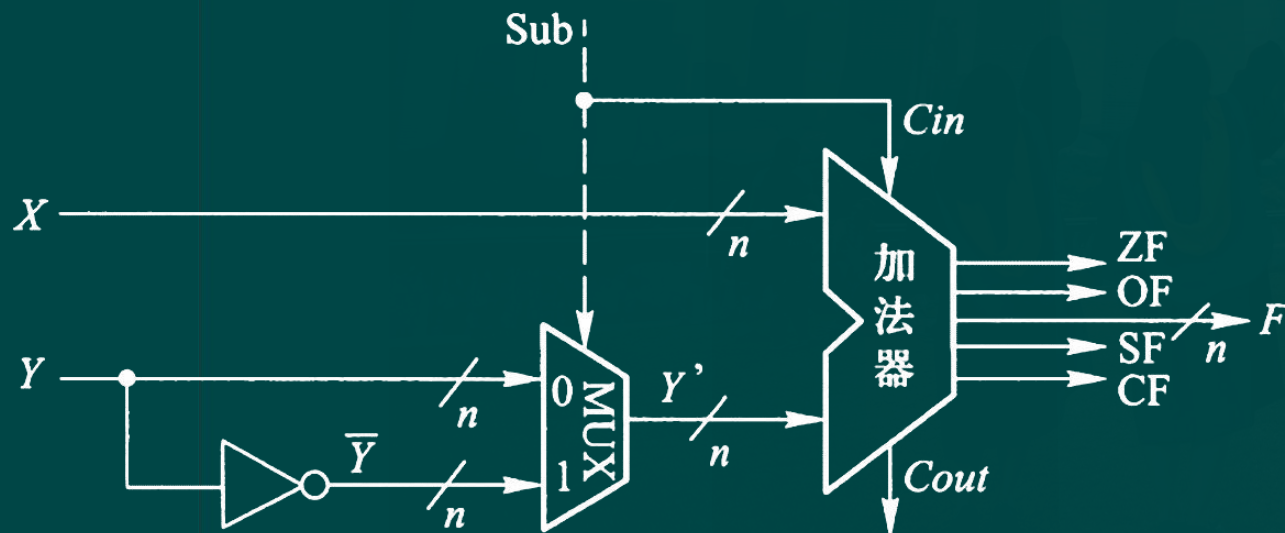


图 2.11 补码加减运算部件

◎第二章 数据的表示和运算

考点17 补码加减运算器

可通过标志信息来区分带符号整数运算结果和无符号整数运算结果。

- 零标志ZF=1表示结果F为0。不管作为无符号数还是带符号整数来运算，ZF都有意义。
- 符号标志SF表示结果的符号，即F的最高位。对于无符号数运算，SF没有意义。

◎第二章 数据的表示和运算

考点17 补码加减运算器

可通过标志信息来区分带符号整数运算结果和无符号整数运算结果。

➤ 进/借位CF表示无符号数加/减运算时的进位/借位。

- 加法时，若CF=1表示无符号数加法溢出；
- 减法时，若CF=1表示有借位，即不够减。
- 因此，加法时CF就应等于进位输出Cout；减法时，就应将进位输出Cout取反来作为借位标志。
- 综合起来，可得 $CF = Sub \oplus Cout$ 。
- 对于带符号整数运算，CF没有意义。

◎第二章 数据的表示和运算

考点17 补码加减运算器

有以下两种溢出判断逻辑表达式。

①若符号位产生的进位 C_n 。与最高数值位向符号位的进位 C_{n-1} 不同，则产生溢出，即：

$$Overflow = C_n \oplus C_{n-1}$$

②若两个加数的符号位 X_{n-1} 和 Y_{n-1} 相同，且与和的符号位 F_{n-1} 不同，则产生溢出，即：

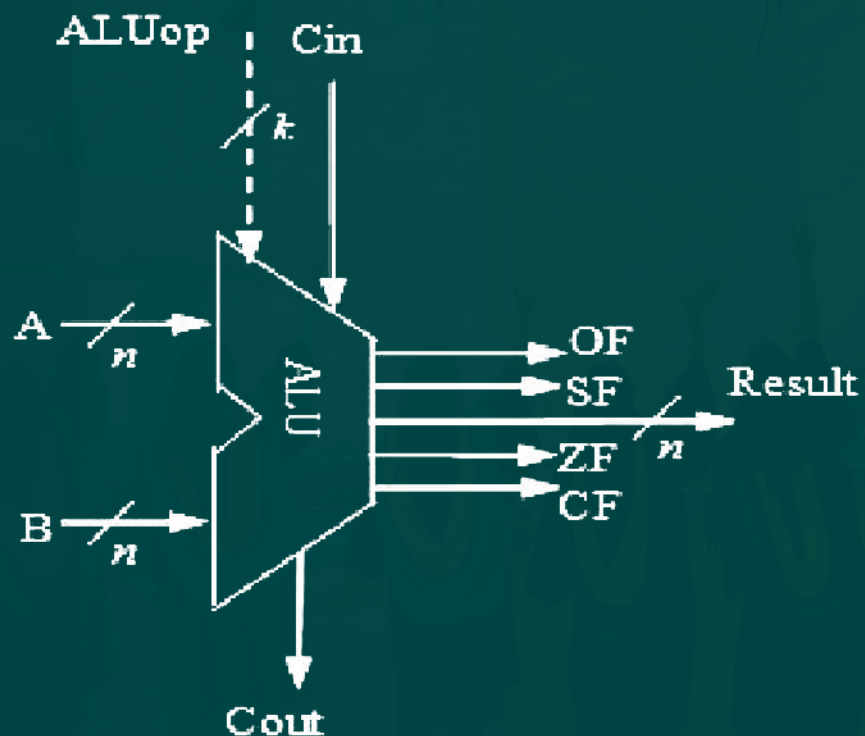
$$Overflow = X_{n-1} \cdot Y_{n-1} \cdot \overline{F_{n-1}} + \overline{X_{n-1}} \cdot \overline{Y_{n-1}} \cdot F_{n-1}$$

根据上述溢出判断逻辑表达式，可以很容易实现溢出判断电路

◎第二章 数据的表示和运算

考点18 算术逻辑部件

算术逻辑部件ALU是一种能进行多种算术运算与逻辑运算的组合逻辑电路，其核心部件是带标志加法器，多采用先行进位方式。通常用图所示的符号来表示。



◎第二章 数据的表示和运算

考点19 无符号数乘法运算

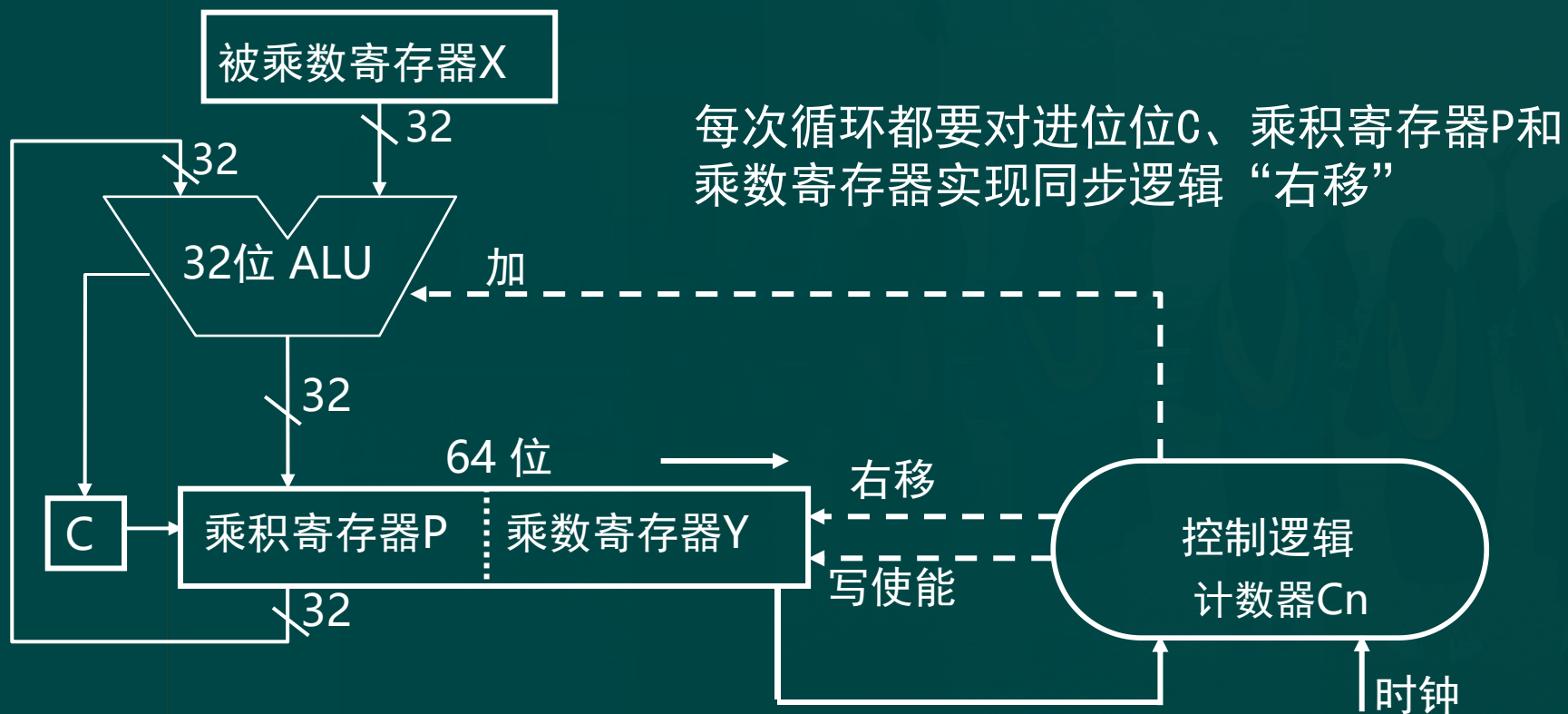
两个无符号数相乘，类似手算乘法，主要思想包括以下几个方面：

- ①每将乘数 Y 的一位乘以被乘数得 $X \times y_i$ ，后，就将该结果与前面所得的结果累加，得到 P_i ，称之为部分积。因为没有等到全部计算后一次求和，所以减少了保存每次相乘结果 $X \times y_i$ 的开销。
- ②每次求得 $X \times y_i$ ，后，不是将它左移与前次部分积 P_i 相加，而是将超分积 P_i 右移一位，然后与 $X \times y_i$ ，相加。

◎第二章 数据的表示和运算

考点19 无符号数乘法运算

图2.13是实现两个32位无符号数乘法的逻辑结构图。



◎第二章 数据的表示和运算

考点20 原码乘法运算

用原码实现乘法运算时，符号位与数值位分开计算，因此，原码乘法运算分为两步。

- ①确定乘积的符号位。由两个乘数的符号异或得到。
- ②计算乘积的数值位。乘积的数值部分为两个乘数的数值部分之积。

◎第二章 数据的表示和运算

考点21 补码乘法运算

补码作为机器中带符号整数的表示形式，需要计算机能实现定点补码整数的乘法运算。

根据每次部分积是一位相乘得到还是两位相乘得到，

- 补码一位乘法
- 补码两位乘法

A. D. Booth提出了一种补码相乘算法，可以将符号位与数值位合在一起参与运算，直接得出用补码表示的乘积，且正数和负数同等对待。这种算法被称之为Booth（布斯）乘法。

布斯乘法的算法过程为 n 次“判断—加减—右移”循环，在布斯乘法中，遇到连续的1或连续的0时，可跳过加法运算直接进行右移操作，因此，布斯算法的运算效率较高。

◎第二章 数据的表示和运算

考点21 补码乘法运算

- ①乘数最低位增加一位辅助位 $y_{-1}=0$ 。
- ②若 $y_i y_{i-1}=01$ ，则“ $+ [x]_{\text{补}}$ ”；若 $y_i y_{i-1}=10$ ，则“ $+ [-x]_{\text{补}}$ ”。
- ③算术右移一位，得到部分积。
- ④重复第②和第③步 n 次，结果得 $[x \times y]_{\text{补}}$

◎第二章 数据的表示和运算

考点21 补码乘法运算

补码乘法也可以采用两位一乘的方法，把乘数分成两位一组，根据两位代码的组合决定加或减被乘数的倍数，形成的部分积每次右移两位，总循环次数为 $n/2$ 。该算法可将部分积的数目压缩一半，从而提高运算速度。两位补码乘法称为改进的布斯乘法（Modified BoothAlgorithm, MBA），也称为基4布斯乘法。

◎第二章 数据的表示和运算

考点22 无符号数除法运算

通常的判断操作如下：

- （1）若被除数为0、除数不为0，或者定点整数除法时 $|被除数| < |除数|$ ，则说明商为0，余数为被除数，不再继续执行。
- （2）若被除数不为0、除数为0，对于整数，则发生“除数为0”异常；对于浮点数，则结果为无穷大。
- （3）若被除数和除数都为0，对于整数，则发生除法错异常；对于浮点数，则有些机器产生一个不发信号的NaN，即“quiet NaN”。

只有当被除数和除数都不为0，并且商也不可能溢出（例如，补码中最大负数除以-1时会发生溢出）时，才进一步进行除法运算。

◎第二章 数据的表示和运算

考点22 无符号数除法运算

参考手工除法过程，得到计算机中两个无符号数除法的运算步骤和算法要点如下。

- ①操作数预置：在确认被除数和除数都不为0后，将被除数（必要时进行0扩展）置于余数寄存器R和余数/商寄存器Q中，除数置于除数寄存器Y中。
- ②做减法试商：根据 $R-Y$ 得到的结果的符号来判断两数的大小。若结果为正，则上商1；若结果为负，则上商0。
- ③上商为0时恢复余数：把减掉的除数再加回来，恢复原来的中间余数。
- ④中间余数左移，以便继续试商；

◎第二章 数据的表示和运算

考点22 无符号数除法运算

上述给出的算法要点③中，采用了“上商为0时恢复余数”的方式，所以，把上述这种方法称为“恢复余数法”。也可以不这样做，而是在下一步运算时把当前多减的除数补回来。这种方法称为“不恢复余数法”，又称“加减交替法”。

由此可见，根据余数恢复方式的不同，分为：

- “恢复余数除法”
- “不恢复余数除法”

◎第二章 数据的表示和运算

考点23 原码除法运算

两数符号相“异或”得到商的符号，而通过无符号小数除法方式求出数值部分的商。

在恢复余数除法中，第 i 次余数为 $R_i = 2R_{i-1} - Y$ 。根据下次中间余数的计算方法，有以下两种不同情况。

- 1) 若 $R_i \geq 0$ ，则上商1，不需恢复余数，左移一位后试商得下次余数 R_{i+1} ，即 $R_{i+1} = 2R_i - Y$
- 2) 若 $R_i < 0$ ，则上商0，恢复余数后左移一位再试商得下次余数 R_{i+1} ，即 $R_{i+1} = 2(R_i + Y) - Y = 2R_i + Y$ 。

从上述结果可知，当第 i 次中间余数为负时，可以跳过恢复余数这一步，直接求第 $i+1$ 次中间余数。这种算法称为不恢复余数法。

◎第二章 数据的表示和运算

考点23 原码除法运算

从上述推导可以发现，不恢复余数法的算法要点就是6个字：“正、1、减，负、0、加”。其含义是，

若中间余数为正数，则上商1，下次做减法；

若中间余数为负数，则上商0，下次做加法。这样运算中每次循环内的步骤都是规整的，差别仅在做加法还是减法，这种方法也称为“加减交替法”。采用这种方法有一点要注意，即如果在最后一步上商为0，则必须恢复余数，把试商时减掉的除数加回去。

◎第二章 数据的表示和运算

考点24 补码除法运算

表2. 6给出了判断是否够减的规则。

中间余数R	除数Y	新中间余数R-Y		新中间余数R+Y	
		0	1	0	1
0	0	够减	不够减	够减 不够减	不够减 够减
0	1	不够减			
1	0				
1	1				

◎第二章 数据的表示和运算

考点25 整数的乘除运算

1. 无符号整数乘的溢出判断

对于 n 位无符号整数 x 和 y 的乘法运算，若取 $2n$ 位乘积中的低 n 位为乘积，则相当于取模 2^n 。若丢弃的高 n 位乘积为非0，则发生溢出。

例如，对于上述例子，1101与1110相乘得到的8位乘积为10110110，高4位为非0，因而发生了溢出，说明低4位0110不是正确的乘积。

◎第二章 数据的表示和运算

考点25 整数的乘除运算

无符号整数乘运算可用公式表示如下，式中p是指取低n位乘积时对应的值。

$$p = \begin{cases} x \times y & (x \times y < 2^n) \text{正常} \\ x \times y \bmod 2^n & (x \times y \geq 2^n) \text{溢出} \end{cases}$$

如果无符号数乘法指令能够将高n位保存到一个寄存器中，则编译器可以根据该寄存器的内容采用相应的比较指令来进行溢出判断。

◎第二章 数据的表示和运算

考点25 整数的乘除运算

2. 带符号整数乘的溢出判断

对于带符号整数乘法，可使用2.7.3节介绍的补码乘法实现，带符号整数乘运算得到的结果是 $2n$ 位乘积的补码表示。

例如，对于上述例子， -3 和 -2 对应的补码 1101 和 1110 在补码乘法器中运算，得到乘积的 $2n$ 位补码表示为 00000110 (对应真值为 6)。

◎第二章 数据的表示和运算

考点25 整数的乘除运算

3. 整数除法的溢出判断

对于带符号整数除法，只有当 $-2147483648/-1$ 时会发生溢出，其他情况下，因为商的绝对值不可能比被除数的绝对值更大，因而肯定不会发生溢出。

但是，在不能整除时需要进行舍入，通常按照朝0方向舍入。即正数商取比自身小的最接近整数，负数商取比自身大的最接近整数。

除数不能为0，否则根据C语言标准，其结果是未定义的。在IA-32系统中，除数为0会发生“异常”，此时，需要调出操作系统中的异常处理程序来处理。

◎第二章 数据的表示和运算

考点25 整数的乘除运算

4. 变量和常数之间的乘除运算

从上述介绍的定点运算部件可以看出，乘除运算比移位和加减运算复杂得多，所需时间也更多。因此，编译器在处理变量与常数相乘或相除时，往往以移位、加法和减法的组合运算来代替乘除运算。

◎第二章 数据的表示和运算

考点26 浮点数加减法

设两个规格化浮点数 x 和 y 表示为 $x = Mx \times 2^{Ex}$, $y = My \times 2^{Ey}$, Mx 、 My 分别是浮点数 x 和 y 的尾数, Ex 和 Ey 分别是浮点数 x 和 y 的阶, 不失一般性, 设 $Ex \leq Ey$, 那么

$$x + y = (Mx \times 2^{Ex - Ey} + My) \times 2^{Ey}$$

$$x - y = (Mx \times 2^{Ex - Ey} - My) \times 2^{Ey}$$

计算机中实现上述计算过程需要经过对阶、尾数加减、规格化和舍入4个步骤, 此外, 还必须考虑溢出判断和溢出处理问题。

◎第二章 数据的表示和运算

考点26 浮点数加减法

假定在下面的讨论中 $x \pm y$ 未经规格化的结果表示为 $M_b \times 2^{Eb}$

1. 对阶

对阶的目的是使 x 和 y 的阶码相等，以使尾数可以相加减。

对阶的原则是：小阶向大阶看齐，阶小的那个数的尾数右移，右移的位数等于两个阶的差的绝对值。

◎第二章 数据的表示和运算

考点26 浮点数加减法

假设 $\Delta E = E_x - E_y$ ，则对阶操作可以表示如下，

若 $\Delta E \leq 0$ ，则 $E_x \leftarrow E_y$, $M_x \leftarrow M_x \times 2^{E_x - E_y}$, $E_b \leftarrow E_y$

若 $\Delta E > 0$ ，则 $E_y \leftarrow E_x$, $M_y \leftarrow M_y \times 2^{E_y - E_x}$, $E_b \leftarrow E_x$

◎第二章 数据的表示和运算

考点26 浮点数加减法

2. 尾数加减

对阶后两个浮点数的阶码相等，此时，可以进行对阶后的尾数相加减。

因为IEEE754 采用定点原码小数表示尾数，所以，尾数加减实际上是定点原码小数的加减运算。

因为IEEE754浮点数尾数中有一个隐藏位，所以，在进行尾数加减时，必须把隐藏位还原到尾数部分。

◎第二章 数据的表示和运算

考点26 浮点数加减法

3. 尾数规格化

IEEE 754的规格化尾数形式为： $\pm 1. bb\cdots b$ 。在进行尾数相加减后可能会得到各种形式的结果，例如，

$$\pm 1. bb\cdots b + 1. bb\cdots b = \pm 1b. bb\cdots b$$

$$\pm 1. bb\cdots b - 1. bb\cdots b = \pm 0. 00\cdots 01b\cdots b$$

(1) 对于上述结果为 $\pm 1b. bb\cdots b$ 的情况，需要进行右规：尾数右移一位，阶码加1。右规操作可以表示为 $M_b \leftarrow M_b \times 2^{-1}$ ， $E_b \leftarrow E_b + 1$ 。尾数右移时，最高位“1”被移到小数点前一位作为隐藏位，最后一位移出时，要考虑舍入。阶码加1时，直接在末位加1。

(2) 对于上述结果为 $\pm 0. 00\cdots 01b\cdots b$ 的情况，需要进行左规：数值位逐次左移，阶逐次减1，直到将第一位“1”移到小数点左边。

◎第二章 数据的表示和运算

考点26 浮点数加减法

4. 尾数的舍入处理

在对阶和尾数右规时，可能会对尾数进行右移，为保证运算精度，一般将低位移出的位保留下来，参加中间过程的运算，最后再将运算结果进行舍入，还原表示成IEEE754格式。

这两位附加位中，紧跟在浮点数尾数右边那一位为保护位或警戒位（guard），用以保护尾数右移的位，紧跟保护位右边的是舍入位（round），左规时可以根据其值进行舍入。

为了更进一步提高计算精度，在保护位和舍入位后面还引入了额外的一个数位，称为粘位（sticky），只要舍入位的右边有任何非0数字，粘位就被置1；否则，粘位被置为0。

◎第二章 数据的表示和运算

考点26 浮点数加减法

IEEE 754提供了以下可选的4种模式。

①就近舍入。

舍入为最近可表示的数。当运算结果是两个可表示数的非中间值时，实际上是“0舍1入”方式；当运算结果正好在两个可表示数中间时，根据“就近舍入”的原则就无法操作了。

②朝 $+\infty$ 方向舍入。总是取右边最近可表示数，也称为正向舍入或朝上舍入。

③朝 $-\infty$ 方向舍入。总是取左边最近可表示数，也称为负向舍入或朝下舍入。

④朝0方向舍入。直接截取所需位数，丢弃后面所有位，也称为截取、截断或恒舍法。这种舍入处理最简单。对正数或负数来说，都是取更靠近原点的那个可表示数，是一种趋向原点的舍入，因此，又称为趋向零舍入。

◎第二章 数据的表示和运算

考点26 浮点数加减法

5. 阶的溢出判断

在进行尾数规格化和尾数舍入时，可能会对结果的阶码执行加、减运算。因此，必须考虑结果的阶的溢出问题。若一个正阶超过了最大允许值（127或1023），则发生“阶上溢”，机器产生“阶上溢”异常。

◎第二章 数据的表示和运算

考点27 浮点数乘除运算

已知两个浮点数 $x = M_x \times 2^{E_x}$, $y = M_y \times 2^{E_y}$, 则乘、除运算的结果如下。

$$x \times y = (M_x \times 2^{E_x}) \times (M_y \times 2^{E_y}) = (M_x \times M_y) \times 2^{E_x + E_y}$$

$$x/y = (M_x \times 2^{E_x}) / (M_y \times 2^{E_y}) = (M_x/M_y) \times 2^{E_x - E_y}$$

下面分别给出浮点数乘法和浮点数除法的运算步骤。

◎第二章 数据的表示和运算

考点27 浮点数乘除运算

1. 浮点数乘法运算

假定 x 和 y 是两个IEEE754标准规格化浮点数，其相乘结果为 $M_o \times 2^{E_b}$ ，则求 M_o 和 E_b 的过程如下。

(1) 尾数相乘、阶相加

尾数的乘法运算 $M_o = M_x \times M_y$ ，可以采用定点原码小数乘法。在运算时，需要将隐藏位1还原到尾数中，并注意乘积的小数点位置。

(2) 尾数规格化

对于IEEE 754标准的规格化尾数 M_x 和 M_y 来说，一定满足条件： $|M_x| \geq 1$ ， $|M_y| \geq 1$ ，因此，两数乘积的绝对值应该满足： $1 \leq |M_x \times M_y| < 4$ 。

◎第二章 数据的表示和运算

考点27 浮点数乘除运算

(3) 尾数舍入处理

对 $M_x \times M_y$ 规格化后得到的尾数形式为 $\pm 01. bb \cdots b$ ，其中小数点后面有 $(2n-2)$ 位尾数积，而最终的结果肯定只能有24位尾数（单精度）或53位尾数（双精度）。

(4) 阶码溢出判断

在进行阶相加、右规和舍入时，要对阶进行溢出判断。

右规和舍入时的溢出判断与浮点数加减运算中的溢出判断方法相同。

◎第二章 数据的表示和运算

考点27 浮点数乘除运算

2. 浮点数除法运算

假定 x 和 y 是两个IEEE754标准规格化浮点数，其相除结果为 $M_o \times 2^{E_b}$ ，则求 M_b 和 E_b 的过程如下。

(1) 尾数相除、阶码相减

尾数的除法运算 $M_b = M_x / M_y$ 可以采用定点原码小数除法。

运算时需将隐藏位1还原到尾数中。

阶的相减运算 $E_b = E_x - E_y$ ，采用移码相减运算算法。

◎第二章 数据的表示和运算

考点27 浮点数乘除运算

(2) 尾数规格化

对于IEEE 754标准的规格化尾数 M_x 和 M_y 来说，一定满足条件： $|M_x| \geq 1$ ， $|M_y| \geq 1$ ，因此，两数相除的绝对值应该满足： $1/2 \leq |M_x/M_y| < 2$ 。

(3) 尾数舍入处理

对 M_x/M_y 规格化后得到的尾数形式为 $\pm 1. bb \cdots b$ ，其中小数点后面有 $n-1$ 位尾数商，因此，需要对商的低位部分进行舍入，其处理方法同浮点数加减运算中的舍入操作。

(4) 阶码溢出判断

在进行阶相减、左规和舍入时，要对阶进行溢出判断。左规和舍入时的溢出判断与浮点数加/减运算中的溢出判断方法相同。

第三章 程序的转换及机器级表示

◎第三章 程序的转换及机器级表示

考点1 机器指令及汇编指令

为了能直观地表示机器语言程序，引入了一种与机器语言一一对应的符号化表示语言。称为汇编语言。

在汇编语言中，通常用容易记忆的英文单词或缩写来表示指令操作码的含义，用标号、变量名称、寄存器名称、常数等表示操作数或地址码。

计算机硬件不能直接执行汇编指令而只能执行机器指令。用来将汇编语言源程序中的汇编指令翻译成机器指令的程序称为汇编程序。而将机器指令反过来翻译成汇编指令的程序称为反汇编程序

◎第三章 程序的转换及机器级表示

考点2 指令集体系结构

ISA定义了机器语言级虚拟机的属性和功能特性，主要包括如下信息。

- 可执行的指令的集合，包括指令格式、操作种类以及每种操作对应的操作数的相应规定。
- 指令可以接受的操作数的类型。
- 操作数或其地址所能存放的寄存器组的结构，包括每个寄存器的名称、编号、长度和用途。
- 操作数所能存放的存储空间的大小和编址方式。
- 操作数在存储空间存放时按照大端还是小端方式存放。
- 指令获取操作数以及下一条指令的方式，即寻址方式。
- 指令执行过程的控制方式，包括程序计数器、条件码定义等。

◎第三章 程序的转换及机器级表示

考点3 指令系统设计风格

现代计算机都采用通用寄存器型指令系统，使用通用寄存器而不是累加器来存放运算过程中所用的临时数据，其指令的操作数可以是立即数（I）或来自通用寄存器（R）或来自存储单元（S）。

指令类型可以是

- RR型（两个操作数都来自寄存器）、
- RS型（两个操作数分别来自寄存器和存储单元）、
- SI型（两个操作数分别来自存储单元和立即数）、
- SS型（两个操作数都来自存储单元）等。

按指令格式的复杂度来分，可分为CISC与RISC两种类型指令系统。

◎第三章 程序的转换及机器级表示

考点3 指令系统设计风格

(1) CISC风格指令系统

在设计指令系统时增加了越来越多功能强大的复杂指令，以使指令的功能接近高级语言语句的功能，给软件提供较好的支持。

◎第三章 程序的转换及机器级表示

考点3 指令系统设计风格

(2) RISC风格指令系统

RISC的着眼点不是简单地放在简化指令系统上，而是通过简化指令使计算机结构更加简单合理，从而提高机器的性能。

与CISC相比、RISC指令系统的主要特点如下：

- ①指令数目少；
- ②指令格式规整，采用定长指令字方式，操作码和操作数地址等字段的长度固定；
- ③只有Load/Store指令中的数据需要访存、这种称为Load/Store型指令风格；
- ④采用大量通用寄存器。

◎第三章 程序的转换及机器级表示

考点4 生成机器代码的过程

这个转换过程分为以下4个步骤。

- ①预处理。例如、在C语言源程序中有一些以#开头的语句，可以在预处理阶段对这些语句进行处理，在源程序中插入所有用#include命令指定的文件和用#define声明指定的宏。
- ②编译。将预处理后的源程序文件编译生成相应的汇编语言程序。
- ③汇编。由汇编程序将汇编语言源程序文件转换为可重定位的机器语言目标代码文件。
- ④链接。由链接器将多个可重定位的机器语言目标文件以及库例程（如printf（）库函数）链接起来，生成最终的可执行文件。

◎第三章 程序的转换及机器级表示

考点5 数据类型及其格式

IA-32，已经有8位（字节）、16位（字）、32位（双字）等不同长度，因而用来表示操作数长度至少要有两位。

在对应的汇编指令中，通过在指令助记符后面加一个长度后缀，或通过专门的数据长度指示符来指出操作数长度。

C语言声明	Intel操作类型	汇编指令长度后缀	存储长度/位
(ungisned) char	整数/字节	b	8
(unsigned) short	整数/字	w	16
(unsigned) int	整数/双字	l	32
...	...	...	...

◎第三章 程序的转换及机器级表示

考点5 数据类型及其格式

IA-32中大部分指令需要区分操作数类型。

例如。指令`fdivs`的操作数为`float`类型，指令`fdivl`的操作数为`double`类型，指令`imulw`的操作数为带符号整数（`short`）类型，指令`mull`的操作数为无符号整数（`unsigned int`）类型。

◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

IA-32指令的操作数有三类：立即数、寄存器操作数和存储器操作数。

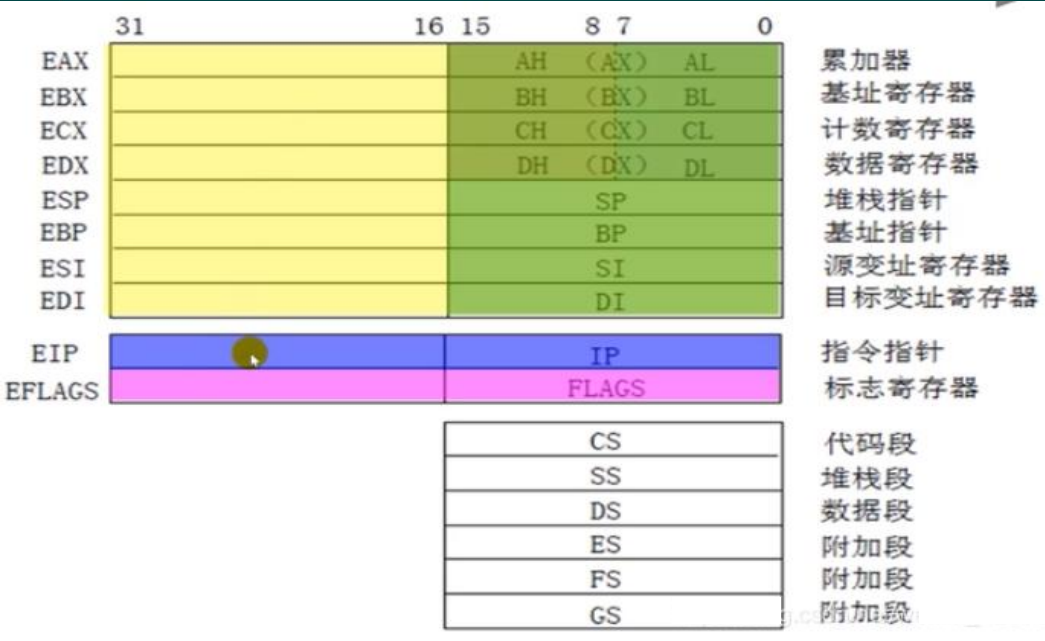
- ①立即数就在指令中，无须指定其存放位置；
- ②寄存器操作数，需要指定操作数所在寄存器的编号，例如，图3.1中的指令指定了源操作数寄存器的编号为001；
- ③当操作数为存储单元内容时，需要指定操作数所在存储单元的地址

◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

1. 定点寄存器组

IA-32由最初的8086/8088向后兼容扩展而来，因此，寄存器的结构也体现了逐步扩展的特点。图给出了定点（整数）寄存器组的结构。



◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

EFLAGS寄存器主要用于记录机器的状态和控制信息，如图

31-22	21	20	19	18	17	16	15	14	13-12	11	10	9	8	7	6	5	4	3	2	1	0
保留	ID	VIP	VIF	AC	VM	RF	0	NT	IOPL	0	D	I	T	S	Z	0	A	0	P	I	C

EFLAGS寄存器中第0~11位中的9个标志位是从最早的8086架构延续下来的，按功能分为6个条件标志和3个控制标志。其中，

- 条件标志用来存放运行的状态信息，由硬件自动设定，条件标志有时也称为条件码（Condition Code，CC）；
- 控制标志由软件设定，用于中断响应、串操作和单步执行等控制。

◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

常用条件标志的含义说明如下。

- 1) OF (Overflow Flag) : 溢出标志。反映带符号数的运算结果是否超过相应数值范围。例如, 字节运算结果超出 $-128 \sim +127$ 或字运算结果超出 $-32768 \sim +32767$ 时, 称为溢出, 此时OF=1; 否则OF=0。
- 2) SF (Sign Flag) : 符号标志。反映带符号数运算结果的符号。负数时, SF=1; 否则SF=0。
- 3) ZF (Zero Flag) : 零标志。反映运算结果是否为0。若结果为0, ZF=1; 否则ZF=0。

◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

4) CF (Carry Flag)：进/借位标志。反映无符号整数加（减）运算后的进（借）位情况。有进（借）位则CF=1；否则CF=0。

综上所述，OF和SF对于无符号数运算来说没有意义，而CF对于带符号整数运算来说没有意义。

◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

控制标志的含义说明如下。

- 1) DF (Direction Flag) : 方向标志。用来确定串操作指令执行时变址寄存器SI (ESI) 和DI (EDI) 中的内容是自动递增还是递减。若DF=1, 则为递减; 否则为递增。可用std指令和cld指令分别将DF置1和清0。
- 2) IF (Interrupt Flag) : 中断允许标志。若IF=1, 表示允许响应中断; 否则禁止响应中断。IF对非屏蔽中断和内部异常不起作用, 仅对外部可屏蔽中断起作用。可用sti指令和cli指令分别将IF置1和清0。

◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

3) TF (Trap Flag) : 陷阱标志。用来控制单步执行操作。TF=1时, CPU按单步方式执行指令, 此时, 可以控制在每执行完一条指令后, 就把该指令执行得到的机器状态 (包括各寄存器和存储单元的值等) 显示出来。没有专门的指令用于对该标志的修改, 但可用栈操作指令 (如pushf/pushfd和popf/popfd) 来改变其值。

EFLAGS寄存器的第12~31位中的其他状态或控制信息是从80286以后逐步添加的。包括用于表示当前程序的I/O特权级 (IOPL)、当前任务是否是嵌套任务 (NT)、当前处理器是否处于虚拟8086方式 (VM) 等一些状态或控制信息。

◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

2. 寻址方式

根据指令给定信息得到操作数或操作数地址的方式称为寻址方式。其中，

- 立即寻址指指令中直接给出操作数；
- 寄存器寻址指指令中给出操作数所存放的寄存器的编号。

除了立即寻址和寄存器寻址外，其他寻址方式下的操作数都在存储单元中，称为存储器操作数。

◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

图给出了IA-32在保护模式下的各种寻址方式，

寻 址 方 式	说 明
立即寻址	指令直接给出操作数
寄存器寻址	指定的寄存器 R 的内容为操作数
位移	$LA = (SR) + A$
基址寻址	$LA = (SR) + (B)$
基址加位移	$LA = (SR) + (B) + A$
比例变址加位移	$LA = (SR) + (I) \times S + A$
基址加变址加位移	$LA = (SR) + (B) + (I) + A$
基址加比例变址加位移	$LA = (SR) + (B) + (I) \times S + A$
相对寻址	$LA = (PC) + A$

◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

有效地址由指令中给出的寻址方式来确定如何计算。有比例变址和非比例变址两种变址方式。

- 比例变址时，变址值等于变址寄存器内容乘以比例系数 S （也称为比例因子）， S 的含义为操作数的字节个数，在IA-32中， S 的取值可以是1、2、4或8。例如，对数组元素访问时，若数组元素的类型为`short`，则比例系数就是2；若数组元素类型为`float`，则比例系数就是4。
- 非比例变址相当于比例系数为1的比例变址情况，即变址值就是变址寄存器的内容，无须乘以比例系数。例如，若数组元素类型为`char`，则比例系数就是1，即非比例变址方式。

◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

3. 浮点寄存器栈和多媒体扩展寄存器组

IA-32的浮点处理架构有两种。

一种是与x86配套的浮点协处理器x87架构，它是一种栈结构，x87 FPU中进行运算的浮点数来源于浮点寄存器栈的栈顶：

另一种是由MMX发展而来的SSE架构，采用单指令多数据（Single Instruction Multi Data, SIMD）技术，SIMD技术可实现单条指令同时并行处理多个数据元素的功能，其操作数来源于专门新增的8个128位寄存器XMM0 ~ XMM7。

◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

综上所述，IA-32中的通用寄存器共有三类：

- 8个8/16/32位定点通用寄存器、
- 8个MMX指令/x87 FPU使用的64位/80位寄存器MM0/ST（0）～MM7/ST（7）、
- 8个SSE指令使用的128位寄存器XMM0～XMM7。

这些寄存器编号如表3.2所示。

◎第三章 程序的转换及机器级表示

考点6 寄存器组织和寻址方式

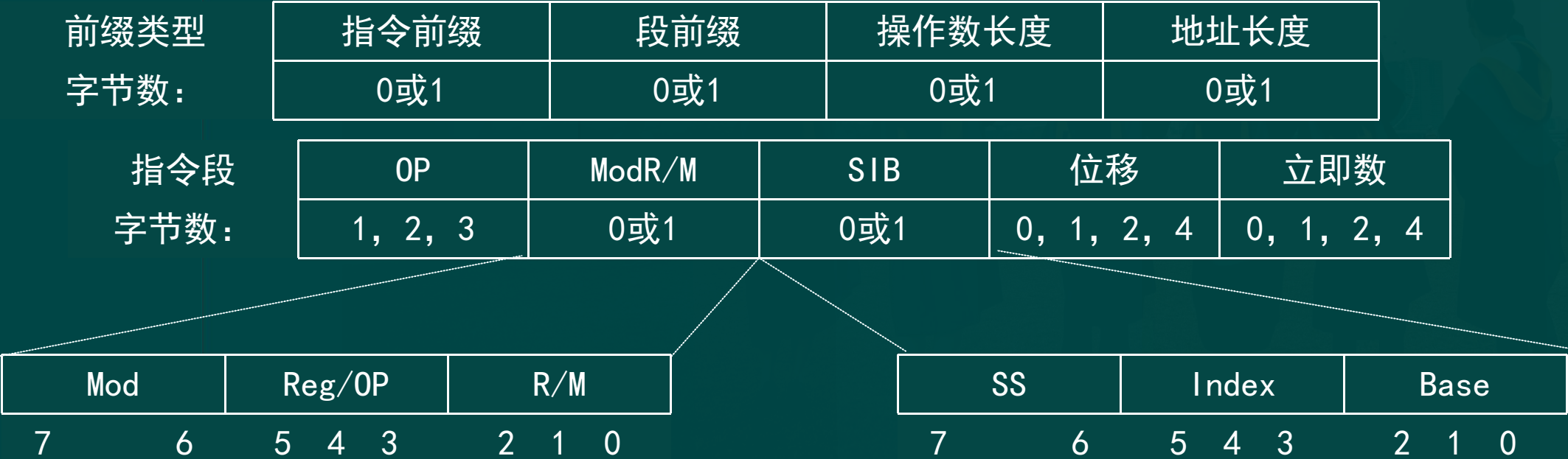
表3. 2 IA-32中通用寄存器的编号

编号	8 位寄存器	16 位寄存器	32 位寄存器	64 位寄存器	128 位寄存器
000	AL	AX	EAX	MM0 / ST(0)	XMM0
001	CL	CX	ECX	MM1 / ST(1)	XMM1
010	DL	DX	EDX	MM2 / ST(2)	XMM2
011	BL	BX	EBX	MM3 / ST(3)	XMM3
100	AH	SP	ESP	MM4 / ST(4)	XMM4
101	CH	BP	EBP	MM5 / ST(5)	XMM5
110	DH	SI	ESI	MM6 / ST(6)	XMM6
111	BH	DI	EDI	MM7 / ST(7)	XMM7

◎第三章 程序的转换及机器级表示

考点7 机器指令格式

机器指令（Instruction）是用0和1表示的一串0/1序列，用来指示CPU完成一个特定的原子操作。图是Intel 64和IA-32体系结构的机器指令格式，包含前缀（Prefix）和指令本身的代码部分。



◎第三章 程序的转换及机器级表示

考点8 传送指令

1. 通用数据传送指令

通用数据传送指令传送的是寄存器或存储器中的数据。

主要有以下几种。

- MOV：一般的传送指令。包括movb、movw和movl等。
- MOVS：符号扩展传送指令。将短的源数据高位符号扩展后传送到目的地址，如movsbw表示把一个字节进行符号扩展后送到一个16位寄存器中。
- MOVZ：零扩展传送指令，将短的源数据高位零扩展后传送到目的地址，如movzwl表示把一个字的高位进行零扩展后送到一个32位寄存器中。

◎第三章 程序的转换及机器级表示

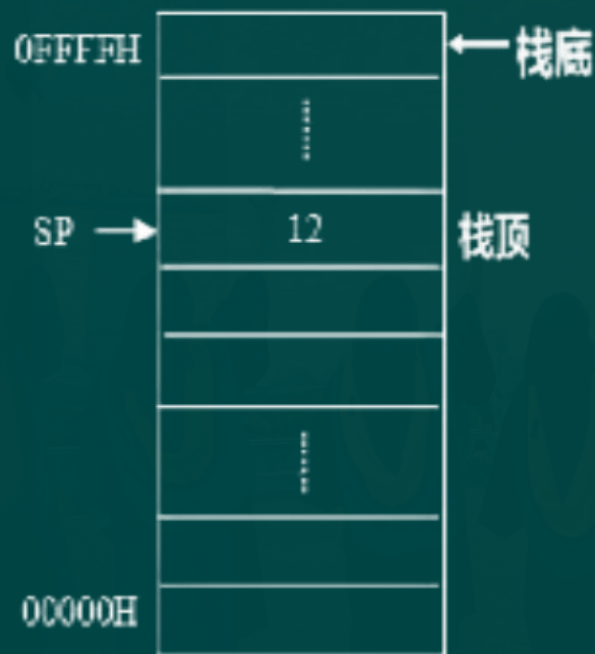
考点8 传送指令

- XCHG：数据交换指令，将两个寄存器内容互换。例如，xchgb表示字节交换。
- PUSH：先执行 $R[sp] \leftarrow R[sp] - 2$ 或 $R[esp] \leftarrow R[esp] - 4$ ，然后将一个字或双字从指定寄存器送到SP或ESP指示的栈单元中。如pushl表示双字压栈，pushw表示字压栈。
- POP：将一个字或双字从SP或ESP指示的栈单元送入指定寄存器，再执行 $R[sp] \leftarrow R[sp] + 2$ 或 $R[esp] \leftarrow R[esp] + 4$ 。如popl表示双字出栈，popw表示字出栈。

◎第三章 程序的转换及机器级表示

考点8 传送指令

栈（Stack）是一种采用“先进后出”方式进行访问的一块存储区，在处理过程调用时非常有用。大多数情况下，栈是从高地址向低地址增长的，在IA-32中，用ESP寄存器指向当前栈顶。而栈底通常在一个固定的高地址上。



◎第三章 程序的转换及机器级表示

考点8 传送指令

2. 地址传送指令

地址传送指令传送的是操作数的存储地址，指定的目的寄存器不能是段寄存器，且源操作数必须是存储器寻址方式。注意，这些指令均不影响标志位。

其中，加载有效地址（Load Effect Address, LEA）指令用于将源操作数的存储地址送到目的寄存器中

◎第三章 程序的转换及机器级表示

考点8 传送指令

3. 输入/输出指令

输入/输出指令专门用于在累加寄存器AL/AX/EAX和I/O端口之间进行数据传送。

例如，in指令用于将I/O端口内容送累加器，out指令将累加器内容送I/O端口。

◎第三章 程序的转换及机器级表示

考点8 传送指令

4. 标志传送指令

标志传送指令专门用于对标志寄存器进行操作。

如pushf指令用于将标志寄存器的内容压栈，popf指令将栈顶内容送标志寄存器，因而popf指令可能会改变标志。

◎第三章 程序的转换及机器级表示

考点9 定点算术运算指令

1. 加/减运算指令

加/减类指令（ADD/SUB）用于对给定长度的两个位串进行相加或相减。两个操作数中最多只能有一个是存储器操作数，不区分是无符号数还是带符号整数，产生的和/差送到目的地。

生成的标志信息送标志寄存器FLAGS/EFLAGS。

◎第三章 程序的转换及机器级表示

考点9 定点算术运算指令

2. 增/减运算指令

增/减类（INC/DEC）指令对给定长度的一个位串加1或减1，给定操作数既是源操作数也是目的操作数，不区分是无符号数还是带符号整数，生成的标志信息送标志寄存器FLAGS/EFLAGS，注意不生成CF标志。

◎第三章 程序的转换及机器级表示

考点9 定点算术运算指令

3，取负指令

取负类指令NEG用于求操作数的负数，也即，将给定长度的一个位串“各位取反、末位加1”，也称之为取补指令。给定操作数既是源操作数也是目的操作数，生成的标志信息送标志寄存器FLAGS/EFLAGS。

◎第三章 程序的转换及机器级表示

考点9 定点算术运算指令

4，比较指令

比较指令CMP用于两个寄存器操作数的比较。用目的操作数减去源操作数。结果不送回目的操作数，即两个操作数保持原值不变，只是标志位做相应改变。因而功能类似SUB指令。通常，该指令后面跟条件跳转指令或条件设置指令。

◎第三章 程序的转换及机器级表示

考点9 定点算术运算指令

5. 乘/除运算指令

乘法指令分成MUL（无符号整数乘）和IMUL（带符号整数乘）两类。

- 对于IMUL指令，可以显式地给出一个、两个或三个操作数。
- 对于MUL指令，只须显式给出一个操作数。

◎第三章 程序的转换及机器级表示

考点10 按位运算指令

1. 逻辑运算指令

以下5类逻辑运算指令中，仅NOT指令不影响条件标志位，其他指令执行后，OF=CF=0，而ZF和SF则根据运算结果来设置：若结果为全0，则ZF=1；若最高位为1，则SF=1。

- NOT：单操作数的取反指令。它将操作数每一位取反，然后把结果送回对应位。
- AND：对双操作数按位逻辑“与”，主要用来实现“掩码”操作。
- OR：对双操作数按位逻辑“或”，常用于使目的操作数的特定位置1。
- XOR：对双操作数按位进行逻辑“异或”，常用于判断两个操作数中哪些位不同或用于改变指定位的值。
- TEST：根据两个操作数相“与”的结果来设置条件标志，常用于需检测某种条件但不能改变原操作数的场合。

◎第三章 程序的转换及机器级表示

考点10 按位运算指令

2. 移位指令

- SHL：逻辑左移，每左移一次，最高位送入CF，并在低位补0。
- SHR：逻辑右移，每右移一次，最低位送入CF，并在高位补0。
- SAL：算术左移，操作与SHL指令类似，每次移位，最高位送入CF，并在低位补0。执行SAL指令时，如果移位前后符号位发生变化，则OF=1，表示左移后结果溢出。这是SAL与SHL的不同之处。
- SAR：算术右移，每右移一次，操作数的最低位送入CF，并在高位补符号。
- ROL：循环左移，每左移一次，最高位移到最低位，并送入CF。
- ROR：循环右移，每右移一次，最低位移到最高位，并送入CF。
- RCL：带循环左移，将CF作为操作数的一部分循环左移。
- RCR：带循环右移，将CF作为操作数的一部分循环右移。

◎第三章 程序的转换及机器级表示

考点11 程序执行流控制指令

有直接跳转和间接跳转两种方式。

- 直接跳转指跳转目标地址由出现在指令机器码中的立即数作为偏移量而计算得到：
- 间接跳转则是指跳转目标地址间接存储在某寄存器或存储单元中。

◎第三章 程序的转换及机器级表示

考点11 程序执行流控制指令

跳转目标地址的计算方法有两种。

- 一种是通过将当前EIP的值加偏移量计算得到。因为偏移量是带符号整数。因此跳转目标地址为EIP内容增加或减少某一个数值得到。也就是采用相对寻址方式得到。可以看成是以当前EIP内容为基准往前或往后跳转。称为相对跳转；
- 另一种是直接将指令中设置的目标地址设置到EIP中，称为绝对跳转。

◎第三章 程序的转换及机器级表示

考点11 程序执行流控制指令

1. 无条件跳转指令

无条件跳转指令JMP的执行结果就是直接跳转到目标地址处执行。

2. 条件跳转指令

条件跳转指令Jcc（其中cc为条件助记符）以标志位或标志位组合作为跳转依据。

3. 条件设置指令

条件设置指令用来将条件标志组合得到的条件值设置到一个8位通用寄存器中，其格式为：

SETcc DST

DST通常是一个8位寄存器。

◎第三章 程序的转换及机器级表示

考点11 程序执行流控制指令

4. 条件传送指令

该类指令的功能是。如果符合条件就进行传送操作，否则什么都不做。其格式为：

`CMOVcc DST, SRC`

源操作数SRC可以是16位或32位寄存器或存储器操作数，传送目的地DST必须是16位或32位寄存器。

◎第三章 程序的转换及机器级表示

考点11 程序执行流控制指令

5. 调用和返回指令

(1) 调用指令

调用指令CALL是一种无条件跳转指令，跳转方式与JMP指令类似。它包含两个操作

- ①将返回地址入栈（相当于PUSH操作）；
- ②跳转到指定地址处执行。

(2) 返回指令

返回指令RET也是一种无条件跳转指令，通常放在子程序的末尾，使子程序执行后返回主程序继续执行。

◎第三章 程序的转换及机器级表示

考点11 程序执行流控制指令

6. 陷阱指令

陷阱也称为自陷或陷入。它是预先安排的一种“异常”事件、就像预先设定的“陷阱”一样。当执行到陷阱指令（也称自陷指令）时、CPU就调出特定的程序进行相应处理，处理结束后返回到陷阱指令的下一条指令执行。

◎第三章 程序的转换及机器级表示

考点11 程序执行流控制指令

陷阱是一种特殊的中断当前程序运行的“异常”事件，以下是IA-32/x86-64中提供的部分异常/中断类指令，其中，INT、into和sysenter为陷阱指令。

- INT n: n为中断类型号，取值范围为0~255。
- iret: 中断返回指令，执行后将回到被中断的程序继续运行。
- into: 溢出中断指令，若OF=1，产生类型号为4的异常，进入相应的溢出异常处理。
- sysenter: 快速进入系统调用指令。
- sysexit: 快速退出系统调用指令。

◎第三章 程序的转换及机器级表示

考点12 过程调用的机器级表示

1. IA-32中用于过程调用的指令

在3.3.4节中提到的调用指令CALL和返回指令RET是用于过程调用的主要指令，它们都属于一种无条件跳转指令，都会改变程序执行的顺序。

◎第三章 程序的转换及机器级表示

考点12 过程调用的机器级表示

2. 过程调用的执行步骤

假定过程P调用过程Q，则P称为调用者（Caller），Q称为被调用者（Callee）。

过程调用的执行步骤如下。

- 1) P将入口参数（实参）放到Q能访问到的地方。
- 2) P将返回地址存到特定的地方，然后将控制转移到Q。
- 3) Q保存P的现场，并为自己的非静态局部变量分配空间。
- 4) 执行Q的过程体（函数体）。
- 5) Q恢复P的现场，并释放局部变量所占空间。
- 6) Q取出返回地址，将控制转移到P。

◎第三章 程序的转换及机器级表示

考点12 过程调用的机器级表示

3. 过程调用所使用的栈

从上述执行步骤来看，在过程调用中，需要为入口参数、返回地址、调用过程执行时用到的寄存器、被调用过程中的非静态局部变量、过程返回时的结果等数据找到存放空间。如果有足够的寄存器，最好把这些数据都保存在寄存器中，这样，CPU 执行指令时，可以快速地从寄存器取得这些数据进行处理。

◎第三章 程序的转换及机器级表示

考点12 过程调用的机器级表示

4. IA-32的寄存器使用约定

寄存器EBX、ESI、EDI是被调用者保存寄存器（Callee Saved Register），Q必须先将它们的值保存到栈中再使用它们，并在返回P之前先恢复它们的值。

两个寄存器EBP和ESP则分别是帧指针寄存器和栈指针寄存器，分别用来指向当前栈帧的底部和顶部。

◎第三章 程序的转换及机器级表示

考点12 过程调用的机器级表示

5. IA-32的栈、栈帧及其结构

IA-32使用栈来支持过程的嵌套调用，过程的入口参数、返回地址、被保存寄存器的值、被调用过程中的非静态局部变量等都会入栈保存。IA-32中可通过执行MOV、PUSH和POP指令存取栈中元素，用ESP寄存器指示栈顶，栈从高地址向低地址增长。

◎第三章 程序的转换及机器级表示

考点12 过程调用的机器级表示

每个过程都有自己的栈区，称为栈帧（Stack Frame），因此，一个栈由若干栈帧组成，每个栈帧用专门的帧指针寄存器EBP指定起始位置。因而，当前栈帧的范围在帧指针EBP和栈指针ESP指向区域之间。

每个过程都有自己的栈区，称为栈帧（Stack Frame），因此，一个栈由若干栈帧组成，每个栈帧用专门的帧指针寄存器EBP指定起始位置。因而，当前栈帧的范围在帧指针EBP和栈指针ESP指向区域之间。

◎第三章 程序的转换及机器级表示

考点12 过程调用的机器级表示

6. 变量的作用域和生存期

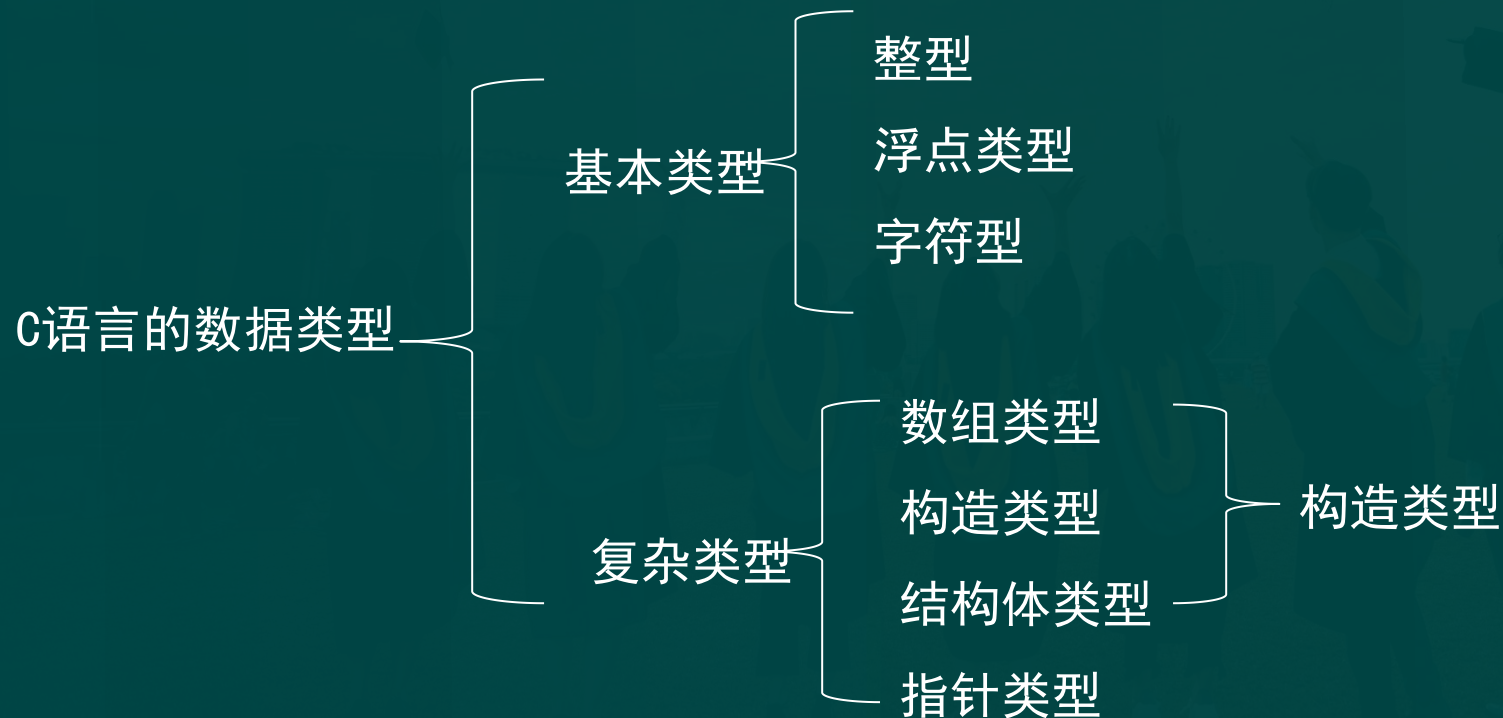
C语言中的auto型变量就是过程（函数）内的非静态局部变量, 因为它是通过执行指令而动态、自动地在栈中分配并在过程结束时释放的, 因而其作用域仅限于过程内部且具有的仅是“局部生存期”

◎第三章 程序的转换及机器级表示

考点12 过程调用的机器级表示

8，按值传递参数和按地址传递参数

使用参数传递数据是C语言函数间传递数据的主要方式。C语言中的数据类型分为基本数据类型和复杂数据类型，而复杂数据类型中又分为构造类型和指针类型。C语言的数据类型如图



◎第三章 程序的转换及机器级表示

考点12 过程调用的机器级表示

C函数中的形式参数可以是基本类型变量名、构造类型变量名和指针类型变量名。对于不同类型的形式参数。其传递参数的方式不同，总体来说分为两种：

- 按值传递：形参是基本类型变量名时。采用按值传递方式
- 按地址传递：形参是指针类型变量名或构造类型变量名时，采用按地址传递方

◎第三章 程序的转换及机器级表示

考点12 过程调用的机器级表示

9. 递归过程调用

过程调用的时间开销也应该考虑。每个过程包含准备阶段和结束阶段，因而每增加一次过程调用。就要增加许多条包含在准备阶段和结束阶段的额外指令，这些额外指令的执行时间开销对程序的性能影响很大。因而，应该尽量避免不必要的过程调用。特别是递归调用。

◎第三章 程序的转换及机器级表示

考点13 选择语句的机器级表示

1. 条件运算表达式的机器级表示

C语言中的三目运算符由“?”和“:”组成。

```
x=cond_expr ?then_expr: else_expr;
```

2. if ~else语句的机器级表示

if~(then)、if~(then)~else选择结构根据判定条件来控制一些语句是否被执行。其通用形式如下。

```
if (condexpr)
    then_statement
else
    statement
```

◎第三章 程序的转换及机器级表示

考点13 选择语句的机器级表示

通常，编译后得到的对应汇编代码可以有如下两种不同的结构，如图。

```
c=cond_expr,  
if (!c)  
    goto false_label  
    then_statement  
    goto done,  
false_label:  
    else_statement  
done:
```

```
c=cond_expr,  
if (c)  
    goto true_label  
    else_statement  
    goto done,  
true_label:  
    then_statement  
done:
```

◎第三章 程序的转换及机器级表示

考点13 选择语句的机器级表示

3. switch语句的机器级表示

解决多分支选择问题可以用连续的if~else~if语句，不过，这种情况下，只能按顺序一一测试条件，直到满足条件时才执行对应分支的语句，因而，通常用switch语句来实现多分支选择功能，它可以直接跳到某个条件处的语句执行，而不用一一测试条件。

◎第三章 程序的转换及机器级表示

考点13 选择语句的机器级表示

左图是一个含有switch语句的过程，右图是对应过程体的汇编表示和跳转表。

```
int switch_test(int a, int b, int c) {  
    int result;  
    switch(a) {  
        case 15:  
            c=b&0x0f;  
        case 10:  
            result=c+50;  
            break:  
        case 12:  
        case 17:  
            result=b+50;  
            break:  
        case 14:  
            result=b;  
            break:  
        default:  
            result=a;  
    }  
    return result;  
}
```

movl	8(%ebp), %eax	.section .rodata
subl	\$10, %eax	.align 4
cmpl	\$7, %eax	.L8
ja	.L5	.long .L2
jmp	*.L8(, %eax4)	.long .L5
.L1		.long .L3
movl	12(%ebp), %eax	.long .L5
addl	\$15, %eax	.long .L4
movl	%eax, 16(%ebp)	.long .L1
.L2		.long .L5
movl	16(%ebp), %eax	.long .L3
addl	\$50, %eax	
jmp	.L7	
.L3		
movl	12(%ebp), %eax	
addl	\$50, %eax	
jmp	.L7	
.L4		
movl	12(%ebp), %eax	
jmp	.L7	
.L5		
movl	12(%ebp), %eax	
addl	\$10, %eax	
.L7		

◎第三章 程序的转换及机器级表示

考点14 循环结构的机器级表示

1. do~while循环的机器级表示

C语言中的do~while语句形式如下。

```
do {  
    Loop body-statement;  
} while (cond_expr)
```

该循环结构的执行过程可以用以下更接近机器级语言的低级行为来描述。

```
loop: Loop_ body_statement  
    c=cond_expr;  
    if (c) goto loop;
```

◎第三章 程序的转换及机器级表示

考点14 循环结构的机器级表示

2. while循环的机器级表示

C语言中的while语句形式如下。

```
while (cond_expr)
    loop_body_statement
```

该循环结构的执行过程可以用以下更接近于机器级语言的低级行为来描述。

```
    c=cond_expr;
    if (!c) goto done;
loop: loop_body_statement
    c=cond_expr;
    if (c) goto loop;
done:
```

◎第三章 程序的转换及机器级表示

考点14 循环结构的机器级表示

3. for循环的机器级表示

C语言中的for语句形式如下。

```
for (begin_expr; cond_expr; update_expr)  
    loop_body_statement;
```

for循环结构的执行过程大多可以用以下更接近于机器级语言的低级行为来描述。

◎第三章 程序的转换及机器级表示

考点14 循环结构的机器级表示

```
begin_expr;  
c=cond_expr;  
if (!c) goto done;  
loop: loop_body_statement  
update_expr;  
c=cond_expr;  
if (c) goto loop;  
done:
```

◎第三章 程序的转换及机器级表示

考点15 数组的分配和访问

1. 数组元素在存储空间的存放和访问

在程序中使用数组，必须遵循定义在前，使用在后的原则。一维数组定义的一般形式如下。

存储类型 数据类型 数组名[元素个数];

其中，存储类型可以缺省。

◎第三章 程序的转换及机器级表示

考点15 数组的分配和访问

2. 数组的存储分配和初始化

数组可以定义为静态存储型（static）、外部存型（extern）、自动存储型（auto）或者定义为全局静态区数组，其中，只有auto型数组被分配在栈中，其他存储型数组都分配在静态数据区。

数组的初始化就是在定义数组时给数组元素赋初值。

例如，以下声明可以对数组A的4个元素初始化。

```
static short A[4] = { 13, 80, 90, 65 } ;
```

◎第三章 程序的转换及机器级表示

考点15 数组的分配和访问

3. 数组与指针

在指针变量的目标数据类型与数组元素的数据类型相同的前提条件下，指针变量可以指向数组或者数组中的任意元素。

序号	表达式	类型	值的计算方式	汇编代码
1	A	int *	SA	leal (%ecx), %eax
2	A[0]	int	M[SA]	movl (%ecx), %eax
3	A[i]	int	M[SA+4*i]	movl (%ecx, %edx, 4), %eax
4	&A[3]	int *	SA+12	leal 12(%ecx), %eax
5	&A[i]-A	int	(SA+4*i-SA) /4=i	movl %edx, %eax
6	*(A+i)	int	M[SA+4*i]	movl (%ecx, %edx, 4), %eax
7	*(&A[0]+i-1)	int	M[SA+4*i-4]	movl -4(%ecx, %edx, 4), %eax
8	A+i	int *	SA+4*i	movl (%ecx, %edx, 4), %eax

◎第三章 程序的转换及机器级表示

考点16 结构体数据的分配和访问

1. 结构体成员在存储空间的存放和访问

结构体中的数据成员存放在存储器中一段连续的存储区中，指向结构的指针就是其第一个字节的地址。编译器在处理结构型数据时，根据每个成员的数据类型获得相应的字节偏移量，然后通过每个成员的字节偏移量来访问结构成员。

◎第三章 程序的转换及机器级表示

考点16 结构体数据的分配和访问

2. 结构体数据作为入口参数

- 如果采用按值传递方式。则结构的每个成员都要被复制到栈中参数区。这既增加时间开销，又增加空间开销。
- 把指向结构的指针作为参数。这样，在执行call指令之前。就无须把结构成员复制到栈中的参数区，而只要把相应的结构体首地址送到参数区，也即仅传递指向结构体的指针而不复制每个成员。

◎第三章 程序的转换及机器级表示

考点17 联合体数据的分配和访问

联合体的各个成员共享存储空间。

也就是说。在其一时刻，联合体的存储空间中仅在有一个成员数据。因此。联合体也称为共用体。

因为联合体的每个成员所占的存储空间大小可能不同。因而分配给它的存储空间总是按最大数据长度成员所需空间大小为目标。

◎第三章 程序的转换及机器级表示

考点18 数据的对齐方式

最简单的对齐策略是，要求不同的基本类型按照其数据长度进行对齐，例如，`int`型长度是4B，因此规定`int`型数据的地址是4的倍数，称为4字节边界对齐，简称4字节对齐。

◎第三章 程序的转换及机器级表示

考点18 数据的对齐方式

对于由基本数据类型构造而成的struct结构体数据，为了保证其中每个字段都满足对齐要求，i386 System V ABI对struct结构体数据的对齐方式有如下几条规则：

- ①整个结构体变量的对齐方式与其中对齐方式最严格的成员相同：
- ②每个成员在满足其对齐方式的前提下，取地址最小的可用位置作为成员在结构体中的偏移量。这可能导致内部插空；
- ③结构体大小应为对齐边界长度的整数倍。这可能会导致尾部插空。

前两条规则是为了保证结构体中的任意成员都能以对齐的方式访问。

第四章 可执行文件的生成与加载

执行

◎第四章 可执行文件的生成与加载执行

考点1 预处理、编译和汇编

这种将一个程序的所有关联模块对应的目标代码文件结合在一起，以形成一个可执行文件的过程称为链接，由专门的链接程序（Linker，也称为链接器）来实现。链接生成的可执行文件可以被加载并在计算机中执行，计算机能自动逐条取出程序中的指令并执行。

链接的功能是将所有关联的可重定位目标文件组合起来，以生成一个可执行目标文件。

◎第四章 可执行文件的生成与加载执行

考点1 预处理、编译和汇编

1. 预处理

预处理是从源程序变成可执行文件的第一步，C预处理程序为cpp（即C Preprocessor），主要用于C语言编译器对各种预处理命令进行处理，包括对头文件的包含、宏定义的扩展、条件编译的选择等。

◎第四章 可执行文件的生成与加载执行

考点1 预处理、编译和汇编

2. 编译

C编译器在进行具体的程序翻译前，会先对源程序进行词法分析、语法分析和语义分析，然后根据分析的结果进行代码优化和存储分配，最终把C语言源程序翻译成汇编语言程序。编译器通常采用对源程序进行多次扫描的方式进行处理，每次扫描集中完成一项或几项任务，也可以将一项任务分散到几次扫描去完成。

例如，可以按照以下四趟扫描进行处理：

第一趟扫描进行词法分析；

第二趟扫描进行语法分析；

第三趟扫描进行代码优化和存储分配；

第四趟扫描生成代码。

◎第四章 可执行文件的生成与加载执行

考点1 预处理、编译和汇编

3. 汇编

汇编的功能是将编译生成的汇编语言代码转换为机器语言代码。因为通常最终的可执行文件由多个不同模块对应的机器语言目标代码组合而成，所以，在生成单个模块的机器语言目标代码时，不可能确定每条指令或每个数据最终的地址，即单个模块的机器语言目标代码需要重新定位，因此，通常把汇编生成的机器语言目标文件称为可重定位目标文件。

◎第四章 可执行文件的生成与加载执行

考点2 程序的链接过程

可重定位文件和可执行文件都是机器语言目标文件，所不同的是前者是单个模块生成的，而后者是多个模块组合而成的。

因而，对于前者，代码总是从0开始，而对于后者，代码在ABI规范规定的虚拟地址空间中产生。

◎第四章 可执行文件的生成与加载执行

考点2 程序的链接过程

实际上，可重定位文件和可执行文件都不是可以直接显示的文本文件，而是不可显示的二进制文件，它们都按照一定的格式以二进制字节序列构成一种目标文件，其中包含二进制代码区、只读数据区、初始化数据区和未初始化数据区等，每个信息区称为一个节（Section），如代码节（.text）、只读数据节（.rodata）、已初始化全局数据节（.data）和未初始化全局数据节（.bss）等。

◎第四章 可执行文件的生成与加载执行

考点2 程序的链接过程

链接器在将多个可重定位文件组合成一个可执行文件时，主要完成符号解析和重定位两个任务。

1. 符号解析

符号解析的目的是将每个符号的引用与一个确定的符号定义建立关联。符号包括全局静态变量名和函数名，而非静态局部变量名则不是符号。

◎第四章 可执行文件的生成与加载执行

考点2 程序的链接过程

2. 重定位

可重定位文件中的代码区和数据区都是从地址0开始的。链接器需要将不同模块中相同的节合并起来生成一个新的单独的节，并将合并后的代码区和数据区按照ABI规范确定的虚拟地址空间划分（也称存储器映像）来重新确定位置。

◎第四章 可执行文件的生成与加载执行

考点3 ELF目标文件格式

ELF目标文件既可用于程序的链接，也可用于程序的执行。图说明了ELF目标文件格式的基本框架。

ELF头
程序头表（可选）
节1
...
节n
...
...
节头表

a) 链接视图

ELF头
程序头表
段1
段2
节n
...
节头表（可选）

b) 执行视图

◎第四章 可执行文件的生成与加载执行

考点3 ELF目标文件格式

ELF文件中的节头表包含其中各节的说明信息，每个节在该表中都有一个与之对应的项，每一项都指定了节名和节大小之类的信息。用于链接的目标文件必须具有节头表。

1. ELF头

ELF头位于目标文件的起始位置，包含文件结构说明信息。

◎第四章 可执行文件的生成与加载执行

考点4 可重定位目标文件格式

2. 节

节是ELF文件中的主体信息，包含了链接过程所用的目标代码信息，包括指令、数据符号表和重定位信息等。一个典型的ELF可重定位目标文件中包含下面几个节。

- `.text`：目标代码部分。
- `.rodata`：只读数据，如`printf`语句中的格式串、开关语句（如`switch-case`）的跳转表等。
- `data`：已初始化且初值不为0的全局变量和静态变量。

◎第四章 可执行文件的生成与加载执行

考点4 可重定位目标文件格式

- `.bss`：所有未初始化或初始化为0的全局变量和静态变量。因为未初始化变量没有具体的值，所以无须在目标文件中分配用于保存值的空间，即它在目标文件中不占据实际的盘空间，仅是一个占位符。运行时在存储器中再为这些变量分配空间，并设定初始值为0。目标文件中区分初始化和未初始化变量是为了提高盘空间利用率。

◎第四章 可执行文件的生成与加载执行

考点4 可重定位目标文件格式

对于auto型局部变量，它们在运行时被分配在栈中，因此既不出现在.data节，也不出现在.bss节。

- .symtab: 符号表 (Symbol Table)。在程序中被定义的函数名和全局静态变量名都属于符号，与这些符号相关的信息被保存在符号表中。每个可重定位目标文件都有一个symtab节。
- .rel.text: .text节相关的可重定位信息。当链接器将某个目标文件和其他目标文件组合时，.text节中的代码被合并后，一些指令中引用的操作数地址信息或跳转目标指令位置信息等都可能要被修改，因此需要说明指令如何进行重定位。

◎第四章 可执行文件的生成与加载执行

考点4 可重定位目标文件格式

- `.rel.data`: `.data`节相关的可重定位信息。当链接器将某个目标文件和其他目标文件组合时，`.data`节被合并后，一些全局变量的地址可能被修改，因此需要说明数据如何进行重定位。
- `.debug`: 调试用符号表，有些表项对定义的局部变量和类型定义进行说明，有些表项对定义和引用的全局静态变量进行说明。只有使用带-g选项的gcc命令才会得到这张表。
- `.line`: C源程序中的行号和`.text`节中机器指令之间的映射。只有使用带-g选项的gcc命令才会得到这张表。
- `.strtab`: 字符串表，包括`.symtab`节和`.debug`节中的符号以及节头表中的节名。字符串表就是以null结尾的字符串序列。

◎第四章 可执行文件的生成与加载执行

考点4 可重定位目标文件格式

3. 节头表

节头表由若干个表项组成，每个表项描述相应节的节名、在文件中的偏移、大小、访问属性、对齐方式等，目标文件中的每个节都有一个表项与之对应。除ELF头之外，节头表是ELF可重定位目标文件中最重要的一部分内容。

◎第四章 可执行文件的生成与加载执行

考点5 可执行目标文件格式

与ELF可重定位文件格式相比，ELF可执行文件的不同点主要有以下几个方面。

- 1) ELF头中字段e_entry给出程序执行入口地址，可重定位文件中此字段为0。
- 2) 通常会有.init节和.fini节，其中，init节定义一个init函数，用于可执行文件开始执行时的初始化工作，当程序开始运行时，系统会在进程进入主函数main之前，先执行这个节中的指令代码。，fini节中包含进程终止时要执行的指令代码，当程序退出时，系统会执行这个节中的指令代码。
- 3) 少了.rel, text和.rel, data等重定位信息节。因为可执行文件中的指令和数据已被重定位，故可去掉用于重定位的节。
- 4) 多了一个程序头表，也称段头表（Segment Header Table），它是一个结构数组。

◎第四章 可执行文件的生成与加载执行

考点5 可执行目标文件格式

64位系统对应的数据结构为Elf64_Phdr，其中描述的成员与Elf32_Phdr类似，出于对齐考虑，Elf64_Phdr将p_flags移到了p_offset之前。

P_type描述存储段的类型或特殊节的类型。例如，

- 是否为可装入段（PT_LOAD），
- 是否是特殊的动态节（PT_DYNAMIC），
- 是否是特殊的解释程序节（PT_INTERP）。
- p_offset指出本段的首字节在文件中的偏移地址。
- p_vaddr指出本段首字节的虚拟地址。
- p_paddr指出本段首字节的物理地址，因为物理地址由操作系统根据情况动态确定，因而该信息通常是无效的。

◎第四章 可执行文件的生成与加载执行

考点5 可执行目标文件格式

- `p_filesz`指出本段在文件中所占的字节数，可以为0。
- `p_memsz`指出本段在存储器中所占字节数，也可以为0。
- `p_flags`指出存取权限。
- `p_align`指出对齐方式，用一个模数表示，为2的正整数幂，通常模数与页面大小相关，若页面大小为4KB，则模数为 2^{12} 。

◎第四章 可执行文件的生成与加载执行

考点6 符号和符号表

目标文件中有一个符号表。表中包含了在程序模块中定义的所有符号的相关信息。对于某个C程序模块m来说。包含在符号表中的符号有以下三种不同类型。

- 1) 在m中定义并被其他模块引用的全局符号 (Global Symbol)。这类符号包括非静态的函数名和全局变量名。
- 2) 由其他模块定义并被m引用的全局符号，称为m的外部符号 (External Symbol)，包括在m中引用的在其他模块定义的外部函数名和外部变量名。
- 3) 在m中定义并在m中引用的本地符号 (Local Symbol)。这类符号包括带static属性的函数名和全局变量名。

◎第四章 可执行文件的生成与加载执行

考点6 符号和符号表

符号类型可以是未指定（NOTYPE）、变量（OBJECT）、函数（FUNC）、节（SECTON）等。

当类型为“节”时，其表项主要用于重定位。绑定属性可以是本地（LOCAL）、全局（GLOBAL）、弱（WEAK）等。其中，本地符号指在包含其定义的目标文件的外部是不可见的。名称相同的本地符号可存在于多个文件中而不会相互干扰。全局符号对于合并的所有目标文件都可见。

弱符号是通过CCC扩展的属性指示符attribute（（weak））指定的符号，它与全局符号一样，对于所有被合并目标文件都可见。

◎第四章 可执行文件的生成与加载执行

考点7 符号解析和静态链接

符号解析的目的是将每个模块中引用的符号与某个目标模块中的定义符号建立关联。每个定义符号在代码段或数据段中都被分配了存储空间，因此，将引用符号与对应的定义符号建立关联后，就可以在重定位时将引用符号的地址重定位为相关联的定义符号的地址。

◎第四章 可执行文件的生成与加载执行

考点7 符号解析和静态链接

1. 全局符号的解析规则

编译器在对源程序编译时，会把每个全局符号的定义输出到汇编代码文件中，汇编器通过对汇编代码文件的处理，在可重定位文件的符号表中记录全局符号的特性，以供链接时全局符号的符号解析所用。

◎第四章 可执行文件的生成与加载执行

考点7 符号解析和静态链接

- 规则3：若同时出现COMMON符号定义和弱符号定义，则按COMMON符号定义为准。
- 规则4：若一个COMMON符号出现多次定义。则以其中占空间最大的一个为准。因为符号表中仅记录COMMON符号的最小长度，而不会记录变量的类型，因此在链接器确定多重COMMON符号的唯一定义时，以最小长度中的最大值为准进行符号解析。能够保证满足所有同名COMMON符号的空间要求。
- 规则5：若使用编译选项-fno-common，则不考虑COMMON符号，相当于将COMMON符号作为强符号处理。

◎第四章 可执行文件的生成与加载执行

考点7 符号解析和静态链接

2. 与静态库的链接

编译系统通常会提供一种将多个目标模块打包成一个单独的库文件的机制，这个库文件就是静态库（Static Library）。在构建可执行文件时只需指定静态库文件名，链接器会自动到库中寻找那些在应用程序中用到的目标模块，并且只把用到的模块从库中提取出来，和应用程序模块进行链接。

◎第四章 可执行文件的生成与加载执行

考点8 重定位过程

重定位的目的是在符号解析的基础上将所有关联的目标模块合并，并确定每个定义符号在ABI规范规定的虚拟地址空间中的地址，在定义符号的引用处重定位引用的地址。

（1）节和定义符号的重定位

链接器将相互关联的所有可重定位文件中相同类型的节合并，生成一个同一类型的新节。

（2）引用处符号的重定位

链接器对合并后的新代码节（.text）和新数据节（.data）中所有符号引用处进行重定位，使其指向对应的定义符号的起始位置。

◎第四章 可执行文件的生成与加载执行

考点9 动态链接

共享目标文件（Shared Object File），也称为共享库文件。它是一种特殊的可重定位目标文件，其中记录了相应的代码、数据、重定位和符号表信息，能在可执行文件装入或运行时被动态地装入内存并自动被链接，这个过程称为动态链接（Dynamic Link）。

由一个称为动态链接器（DynamicLinker）的程序来完成。

- 类UNIX系统中共享库文件扩展名为.so，
- Windows系统中为动态链接库（Dynamic Link Libraries, DLLs），文件扩展名为.dll。

◎第四章 可执行文件的生成与加载执行

考点9 动态链接

共享库的动态链接有两个方面的特点：一是“共享性”；二是“动态性”。

- “共享性”是指共享库中的代码段在内存只有一个副本，当应用程序在其代码中需要引用共享库中的符号时，在引用处通过某种方式确定指向共享库中对应定义符号的地址即可。
- “动态性”是指共享库只在使用它的程序被加载或执行时才加载到内存，因而在共享库重新后并不需要重新对程序进行链接，每次加载或执行程序时所链接的共享库总是最新的。可以利用共享库的这个特性来实现软件分发或生成动态Web网页等。动态链接有两种方式，一种是在程序加载过程中加载和链接共享库，另一种是在程序执行过程中加载并链接共享库。

◎第四章 可执行文件的生成与加载执行

考点10 程序和进程的概念

程序（Program）就是代码和数据的集合，程序的代码是一个机器指令序列，因而程序是一种静态的概念，它可以作为文件存放在硬盘中。

进程（Process）可以看成是程序的一次运行过程，因此进程是一个具有一定独立功能的程序关于某个数据集合的一次运行活动，因而进程具有动态的含义。

一个可执行文件可以被多次加载执行，也就是说，一个程序可能对应多个不同的进程。

◎第四章 可执行文件的生成与加载执行

考点11 `execve` 函数和 `main` 函数

`execve` 函数的功能是在当前进程的上下文中加载并运行一个新程序。`execve` 函数的用法如下。

```
int execve (char *filename, char *argv[], *envp[]);
```

该函数用来加载并运行可执行目标文件`filename`，可带参数列表；`argv` 和环境变量列表`envp`。

◎第四章 可执行文件的生成与加载执行

考点11 `execve` 函数和 `main` 函数

通常，主函数 `main` 的原型有如下两种形式：

- `int main (int argc, char * *argv, char **envp) ;`
- `int main (int argc, char *argv[], char *envp[]) ;`

其中，参数列表 `argv` 可用一个以 `null` 结尾的指针数组表示。每个数组元素都指向一个用字符串表示的参数。

◎第四章 可执行文件的生成与加载执行

考点12 fork函数和程序的启动加载

在父进程中可通过fork函数创建一个子进程，fork函数的原型如下：

```
pid_t fork (void) ;
```

在Linux系统中，返回值类型pid_t在头文件sys/types.h中定义为int型，fork函数原型在头文件unistd.h中定义。通常用一个唯一的正整数标识一个进程，称为进程ID、简写为PID。这里的返回值实际上就是一个PID。

◎第四章 可执行文件的生成与加载执行

考点12 fork函数和程序的启动加载

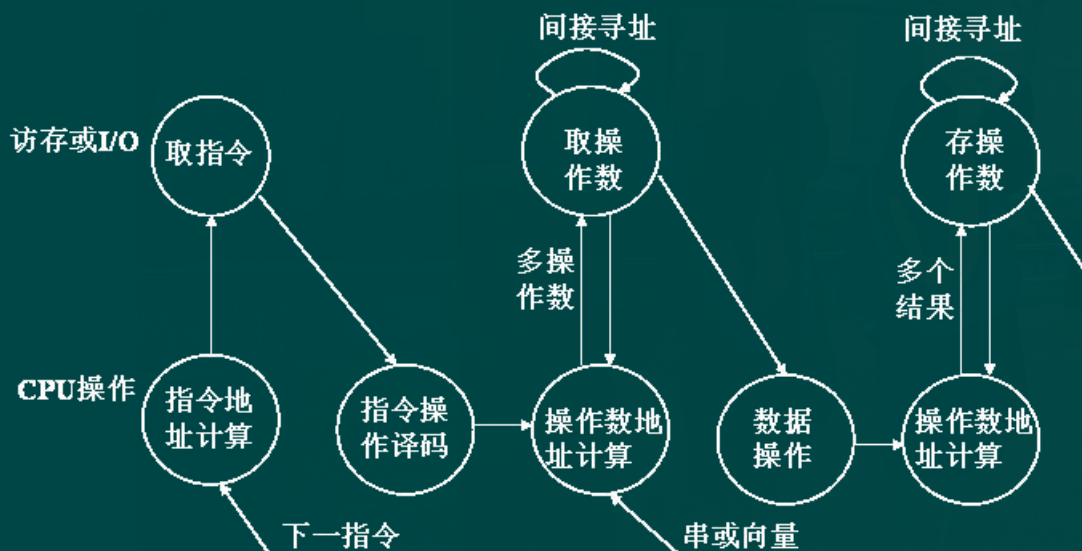
以下说明通过shell命令行输入可执行文件名a.out进行程序加载的过程，大致如下

- 1) shell命令行解释器输出一个命令行提示符（如：unix>），并开始接受用户输入的命令行。
- 2) 当用户在命令行提示符后输入命令行“./a.out[enter]”后，shell命令行程序开始对命令行进行解析，获得各个命令行参数并构造传递给函数execve的参数列表argv和参数个数argc。
- 3) 调用fork函数，创建一个子进程。

◎第四章 可执行文件的生成与加载执行

考点13 程序及指令的执行过程

CPU执行一条指令的大致过程如所示，分成取指令、指令译码、计算源操作数地址并取操作数、执行数据操作、计算目的操作数地址并存结果、计算下条指令地址这几个步骤。



◎第四章 可执行文件的生成与加载执行

考点13 程序及指令的执行过程

- 1) 取指令。马上将要执行的指令的地址总是在程序计数器（PC）中，因此，取指令的操作就是从PC所指出的存储单元中取出指令送到指令寄存器（IR）。
- 2) 对IR中的指令操作码进行译码。不同指令其功能不同，即指令涉及的操作过程不同，因而需要不同的操作控制信号。
- 3) 源操作数地址计算并取操作数。根据寻址方式确定源操作数地址计算方式。若是存储器数据，则需要一次或多次访存。

◎第四章 可执行文件的生成与加载执行

考点13 程序及指令的执行过程

- 4) 执行数据操作。在ALU或加法器等运算部件中对取出的操作数进行运算。
- 5) 目的操作数地址计算并存结果。根据寻址方式确定目的操作数地址计算方式，若是存储器数据，则需要一次或多次访存（间接寻址时）；若是寄存器数据，则在进行数据操作时直接将结果存到寄存器。
- 6) 指令地址计算并将其送PC。顺序执行时，下条指令地址的计算比较简单，只要将PC加上当前指令长度即可，

◎第四章 可执行文件的生成与加载执行

考点13 程序及指令的执行过程

每条指令的功能总是通过对以下4种基本操作进行组合来实现的，也即，每条指令的执行可以分解成若干个以下基本操作。

- 1) 读取某存储单元内容（可能是指令或操作数或操作数地址），并将其装入某个寄存器
- 2) 把某个寄存器中的数据存储在给定的存储单元中。
- 3) 把一个数据从某个寄存器传送到另一个寄存器或者ALU的输入端。
- 4) 在ALU中进行某种算术运算或逻辑运算，并将结果传送到某个寄存器。

◎第四章 可执行文件的生成与加载执行

考点14 CPU的基本功能和组成

CPU的基本职能是周而复始地执行指令。

基本组成：

- 1) 程序计数器（PC）。PC又称指令计数器或指令指针寄存器（IP），用来存放即将执行指令的地址。
- 2) 指令寄存器（IR）。IR用以存放现行指令。
- 3) 指令译码器（ID）。ID对IR中的操作码部分进行译码，产生的译码信号提供给操作控制信号形成部件，以产生控制信号。

◎第四章 可执行文件的生成与加载执行

考点14 CPU的基本功能和组成

- 4) 启停控制逻辑。脉冲源产生一定频率的脉冲信号作为CPU的时钟信号。
- 5) 时序信号产生部件。该部件以时钟信号为基础，产生不同指令对应的时序信号，以实现机器指令执行过程的时序控制。
- 6) 操作控制信号形成部件。该部件综合时序信号、指令译码信号和执行部件反馈的条件标志（如CF、SF、ZF和OF）等。
- 7) 总线控制逻辑。实现对总线传输的控制，包括对数据和地址信息的缓冲与控制。
- 8) 中断机构。

◎第四章 可执行文件的生成与加载执行

考点14 打断程序正常执行的事件

当然，程序并不总是能按正常顺序执行。有时CPU会遇到一些特殊情况而无法继续执行当前程序。

- 对指令操作码进行译码时，发现是不存在的“非法操作码”，因此，CPU不知道如何实现当前指令而无法继续执行。
- 在访问指令或数据时，发现页故障，如段错误（Segmentation Fault）、缺页（PageFault）等，因此，CPU没有访问到正确的指令或数据而无法继续执行当前指令。
- 在ALU中运算的结果发生溢出，或者整数除法指令的除数为0等，因此，CPU发现运算结果不正确而无法继续执行程序。
- 在程序执行过程中，CPU接收到外部发送来的中断请求信号。

◎第四章 可执行文件的生成与加载执行

考点14 打断程序正常执行的事件

CPU中相应的异常和中断处理逻辑称为中断机构。

所有这些打断程序正常执行的事件被分成两大类：内部异常和外部中断。

(1) 内部异常

内部异常 (Exception) 是指由CPU在执行某条指令时引起的与该指令相关的意外事件。

(2) 外部中断

程序执行过程中，若CPU外部发生了采样计时时间到、网络数据包到达、用户按下<Ctrl+C>等外部事件，要求CPU中止当前程序的执行，则会向CPU发中断请求信号，要求CPU对这些情况进行处理

◎第四章 可执行文件的生成与加载执行

考点15 异常和中断的响应过程

CPU对异常和中断的响应过程可分为三个步骤：

- 保护断点和程序状态、
- 关中断、
- 识别异常和中断事件并转相应处理程序。

◎第四章 可执行文件的生成与加载执行

考点15 异常和中断的响应过程

异常和中断源的识别可以采用软件识别或硬件识别两种方式。

- 软件识别通常是在CPU中设置一个原因寄存器，该寄存器中有一些标识异常原因或中断类型的标志信息。
- 硬件识别称为向量中断方式。这种方式下，通常将不同异常/中断处理程序的首地址称为中断向量，所有中断向量存放在一个表中，称为中断向量表。

◎第四章 可执行文件的生成与加载执行

考点16 指令流水线的基本概念

通常把数据通路中专门进行数据运算的部件称为执行元件（Execution Unit）或功能元件（Function Unit），数据通路由控制元件（也称控制器）进行控制。

CPU中最基本的电路主要有数据通路和控制器两部分组成。

◎第四章 可执行文件的生成与加载执行

考点16 指令流水线的基本概念

所有这些指令的处理过程可以归纳为以下4个阶段。

- 1) 取指令并PC加1 (IF)：根据PC的值从存储器取出指令，并 $PC \leftarrow PC+1$ 。
- 2) 译码并读寄存器 (ID)：对指令操作码进行译码并生成控制信号，同时读取寄存器 rs 和 rt 的内容。
- 3) 运算或读存储器 (EX)：在ALU中对寄存器操作数进行运算，或者根据 $addr$ 读存储器
- 4) 结果写回 (WB)：将结果写入目的寄存器 rt ，或写入主存单元 $addr$ 中。



◎第四章 可执行文件的生成与加载执行

考点16 指令流水线的基本概念

在指令的流水线执行方式下，流水线CPU设计的原则是：
指令流水段个数以最复杂指令所用的阶段数为准；
流水段执行时间以最复杂操作所用时间为准。

第五章 程序的存储访问

◎第五章 程序的存储访问

考点1 存储器的分类

存储元件必须具有两个截然不同的物理状态，才能被用来表示二进制代码0和1。

按存储元件分类

- 半导体器件构成的存储器称为半导体存储器；
- 磁性材料存储器是磁表面存储器，如磁盘存储器和磁带存储器
- 光介质存储器称为光盘存储器。

◎第五章 程序的存储访问

考点1 存储器的分类

随机存取存储器（RAM）

特点是按地址访问存储单元，因为每个地址译码时间相同，所以，在不考虑芯片内部缓冲的前提下，每个单元的访问时间是一个常数，与地址无关。

半导体存储器属于随机存取存储器，可用作cache和主存储器。

◎第五章 程序的存储访问

考点1 存储器的分类

按信息的可更改性分类

按信息的可更改性分为读写存储器（RW Memory）和只读存储器（ROM）。

- 读写存储器中的信息可以读出和写入，RAM芯片是一种读写存储器；
- ROM中的信息一旦确定，通常情况下只读不写

RAM芯片和ROM芯片都采用随机存取方式进行信息的访问。

◎第五章 程序的存储访问

考点1 存储器的分类

按断电后信息的可保存性分类

按断电后信息的可保存性分成非易失性存储器和易失性存储器

- 非易失性存储器的信息可一直保留，不需电源维持，如ROM、磁表面存储器、光存储器等；
- 易失性存储器在电源关闭时信息自动丢失，如RAM、cache等。

◎第五章 程序的存储访问

考点1 存储器的分类

指令直接面向的存储器是主存储器，简称主存，由DRAM芯片组成。

高速缓存（Cache）由静态RAM（Static RAM，SRAM）芯片组成，位于主存和CPU之间，存取速度接近CPU的工作速度，用来存放当前CPU经常使用到的指令和数据。

CPU执行指令时给出的存储器地址是主存地址，主存是存储器层次结构中的核心存储器，用来存放系统中被启动运行的程序代码及其数据。

◎第五章 程序的存储访问

考点2 主存储器的组成和基本操作

存储阵列也被称为存储体、存储矩阵。

对存储单元编号，所编号码就是地址。

编址单位是指具有相同地址的那些位元构成的一个单位，可以是一个字节或一个字。

对各存储单元进行编号的方式称为编址方式，可以按字节编址，也可以按字编址。

现在大多数通用计算机都采用按字节编址方式，此时，存储体内一个地址中有一个字节。

每一个存储单元由8个记忆单元组成，因此，是按字节编址方式。

◎第五章 程序的存储访问

考点3 存储器的层次化结构

存储器容量指它能存放的二进制位数或字节数。

存储器的访问时间也称存取时间（Access Time），是指访问一次数据所用的时间。

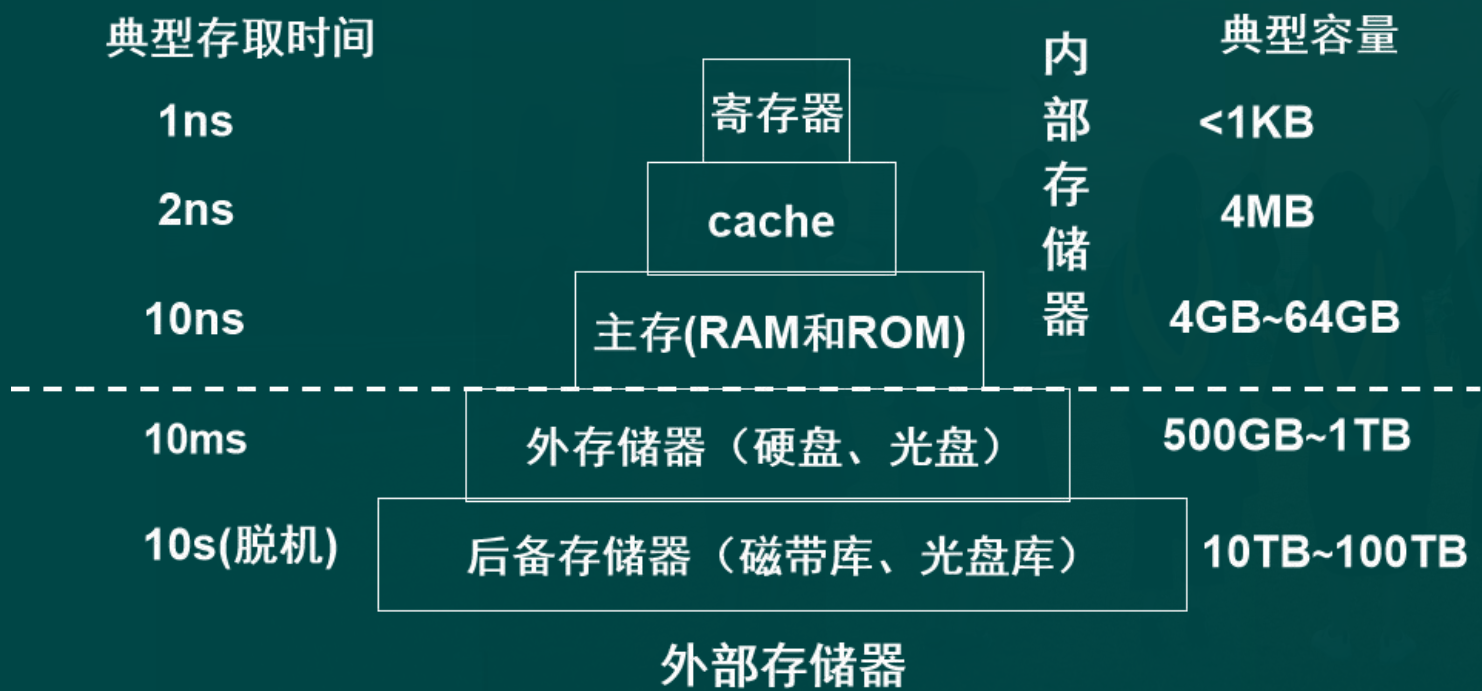
存储器容量和访问时间应能随着处理器速度的提高而同步提高，以保持系统性能的平衡。

然而，随着时间的推移，处理器和存储器在性能上的差异越来越大。为了缩小存储器和处理器两者之间的差距，通常在计算机系统中采用层次化存储结构。

◎第五章 程序的存储访问

考点3 存储器的层次化结构

在计算机中把各种不同容量和不同存取速度的存储器按一定的结构有机结合。以形成层次化存储结构，使得整个存储系统在速度、容量和价格等方面具有较好的综合指标。



◎第五章 程序的存储访问

考点4 程序访问的局部性

CPU和主存之间设置高速缓存（cache）也可以提高CPU访问指令和数据的速度。

工作原理：程序访问的局部性

- 时间局部性是指被访问的某个存储单元在一个较短的时间间隔内很可能又被访问。
- 空间局部性是指被访问的某个存储单元的邻近单元在一个较短的时间间隔内很可能也被访问。

◎第五章 程序的存储访问

考点5 主存芯片技术

SDRAM的工作方式与传统的DRAM有很大不同。

- 传统DRAM与CPU之间采用异步方式交换数据，CPU发出地址和控制信号后，经过一段延迟时间，数据才读出或写入。在这段时间里，CPU不断采样DRAM的完成信号，在没有完成之前，CPU插入等待状态而不能做其他工作。
- SDRAM芯片则不同，其读/写受外部系统时钟（即前端总线时钟CLK）控制，因此与CPU之间采用同步方式交换数据。它将CPU或其他主设备发出的地址和控制信息锁存起来，经过确定的几个时钟周期后给出响应。因此，主设备在这段时间内，可以安全地进行其他操作。

◎第五章 程序的存储访问

考点5 主存芯片技术

突发传输（Burst）方式。只要在第一次存取时给出首地址，以后按地址顺序读/写即可，而不再需要地址建立时间和行、列预充电时间，就能连续快速地从行缓冲器中输出一连串数据。

内部的工作方式寄存器（也称模式寄存器）可用来设置传送数据的长度以及从收到读命令（与列地址选通信号CAS同时发出）到开始传送数据的延迟时间等，前者称为突发长度（Burst Lengths, BL），后者称为CAS延迟（CAS Latency, CL）。

◎第五章 程序的存储访问

考点5 主存芯片技术

DDR (Double Data Rate) SDRAM是对标准SDRAM的改进设计，通过芯片内部的预取缓冲区提供的双字预取功能，并利用存储器总线上时钟信号的上升沿与下降沿，实现一个时钟内传送两个存储字的功能。

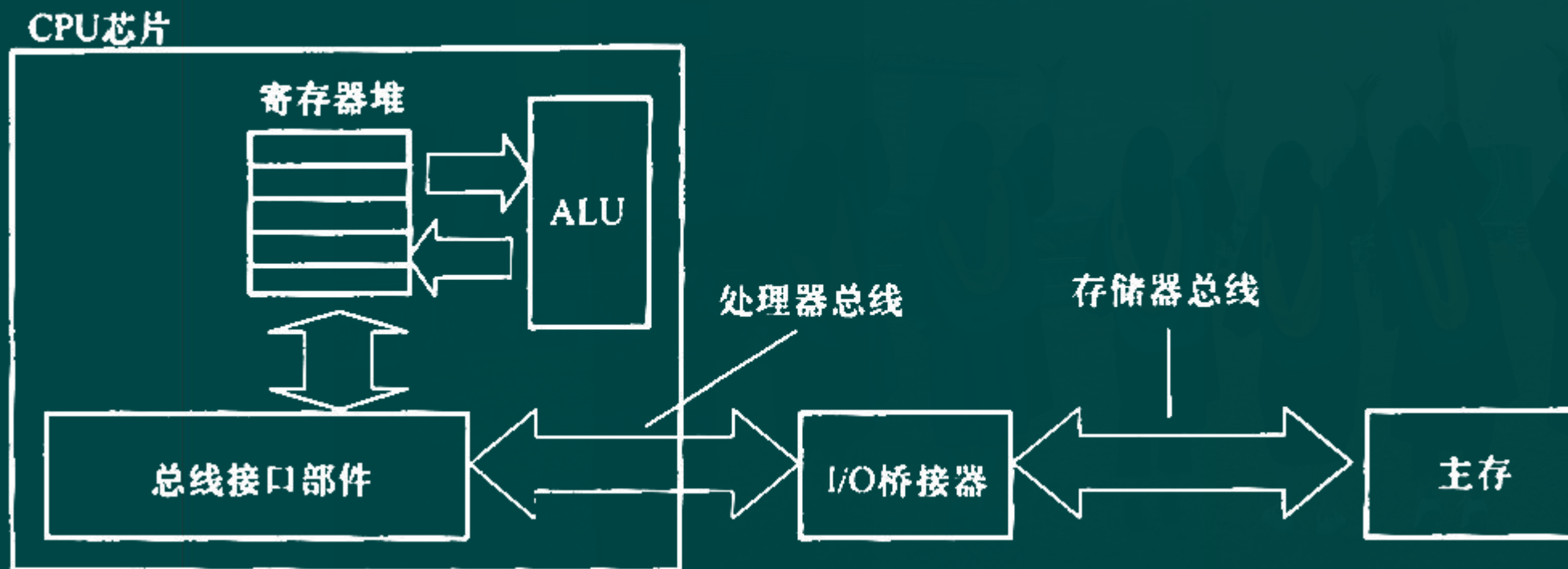
DDR2 SDRAM内存条芯片内部预取缓冲区采用4位预取、

DDR3 SDRAM芯片内部缓冲则采用8位预取

◎第五章 程序的存储访问

考点6 主存与CPU的连接

主存与CPU之间的连接如图所示总线是连接其上的各部件共享的传输介质，通常总线由控制线、数据线和地址线构成。



◎第五章 程序的存储访问

考点6 主存与CPU的连接

由若干个存储器芯片可构成一个内存条，此时，需要在字方向和位方向上进行扩展。
用若干片位数较少的存储器芯片构成给定存储字长的内存条时，需要进行位扩展。

字扩展是容量的扩充，位数不变。

◎第五章 程序的存储访问

考点7 装入/存储指令的操作过程

访存指令主要有两类：

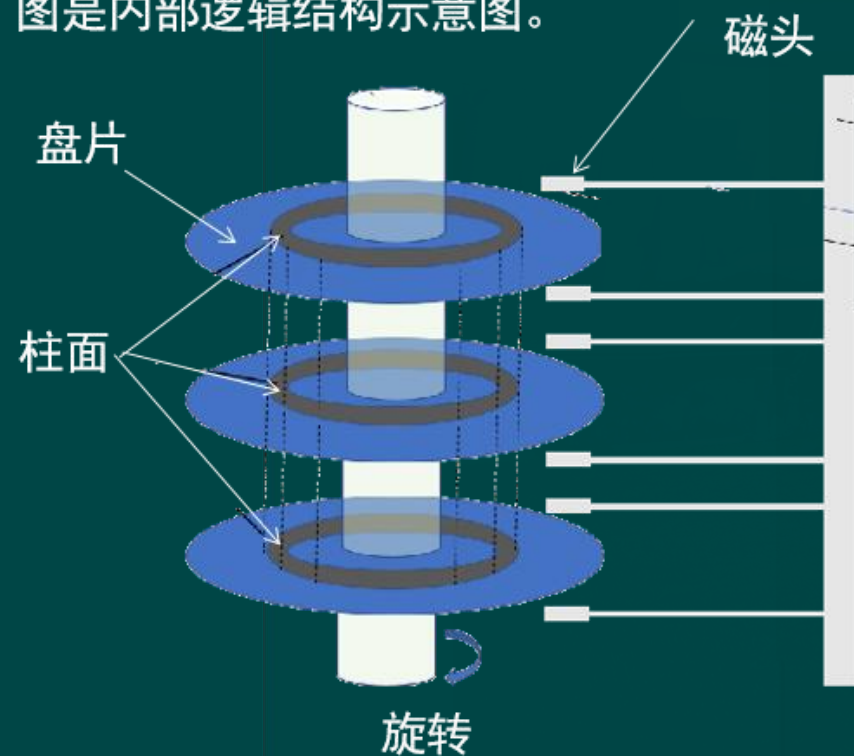
- 取数指令（Load）用于将存储单元内容装入CPU的寄存器中，如1.1.3节给出的8位模型机中的“loadr 0, 6#”指令，IA-32中的“movl 8(%ebp), %eax”指令等；
- 存数指令（Store）用于将CPU寄存器的内容存储到存储单元中，如1.1.3节给出的8位模型机中的“store 10#, r0”指令，IA-32中的“movl %eax, 8(%ebp)”指令等。

◎第五章 程序的存储访问

考点8 磁盘存储器的结构

逻辑结构：

图是内部逻辑结构示意图。



◎第五章 程序的存储访问

考点8 磁盘存储器的结构

磁盘读/写三个步骤。

- 1) 寻道操作：磁盘控制器把盘地址送到盘驱动器的磁盘地址寄存器后，便产生寻道命令，启动磁头定位伺服系统，根据磁头号 and 柱面号，选择指定的磁头移动到指定的柱面。此操作完成后，发出寻道结束信号给磁盘控制器。
- 2) 旋转等待操作：要读/写的信息已经转到磁头下方
- 3) 读/写操作：

◎第五章 程序的存储访问

考点8 磁盘存储器的结构

每个磁道由若干个扇区（也称扇段）组成，每个扇区记录一个数据块，每个扇区有头空（间隙1）、ID域、间隙2、数据域和尾空（间隙3）组成。

头空 (17B)	ID域 (7B)					间隙2 (41B)	数据域 (515B)			尾空 (20B)
	同步 字节 (1B)	道号 (2B)	头号 (2B)	段号 (2B)	CRC (2B)		同步 字节 (1B)	数据 (512B)	CRC (2B)	

◎第五章 程序的存储访问

考点9 磁盘存储器的性能指标

磁盘存储器的性能指标包括记录密度、存储容量、数据传输率和平均存取时间等。

1. 记录密度

道密度指在沿磁道分布方向上单位长度内的磁道数。

位密度指在沿磁道方向上单位长度内存放的二进制信息量。

2. 存储容量

存储容量指整个存储器存放的二进制信息量

◎第五章 程序的存储访问

考点9 磁盘存储器的性能指标

3. 数据传输速率

数据传输速率（Data Transfer Rate）指磁盘存储器完成磁头定位和旋转等待以后，单位时间内读/写存储介质的二进制信息量。为区别于外部数据传输率，通常称之为内部传输速率（Internal Transfer Rate），也称为持续传输率（Sustained Transfer Rate）。

外部传输速率（External Transfer Rate）指主机中的外设控制接口读/写外存储器缓存的速度，由外设采用的接口类型决定，也称为突发数据传输率（Burst Data Transfer Rate）或接口传输率。

◎第五章 程序的存储访问

考点9 磁盘存储器的性能指标

4. 平均存取时间

磁盘响应读/写请求的过程如下：

首先将读/写请求在队列中排队，出队列后由磁盘控制器解析请求命令，然后进行寻道、旋转等待和读/写数据三个过程。

因此，总响应时间的计算公式如下：

响应时间=排队延迟+控制器时间+寻道时间+旋转等待时间+数据传输时间

磁盘上的信息以扇区为单位进行读写，

◎第五章 程序的存储访问

考点9 磁盘存储器的性能指标

上式中后面三个时间之和称为存取时间T。即：

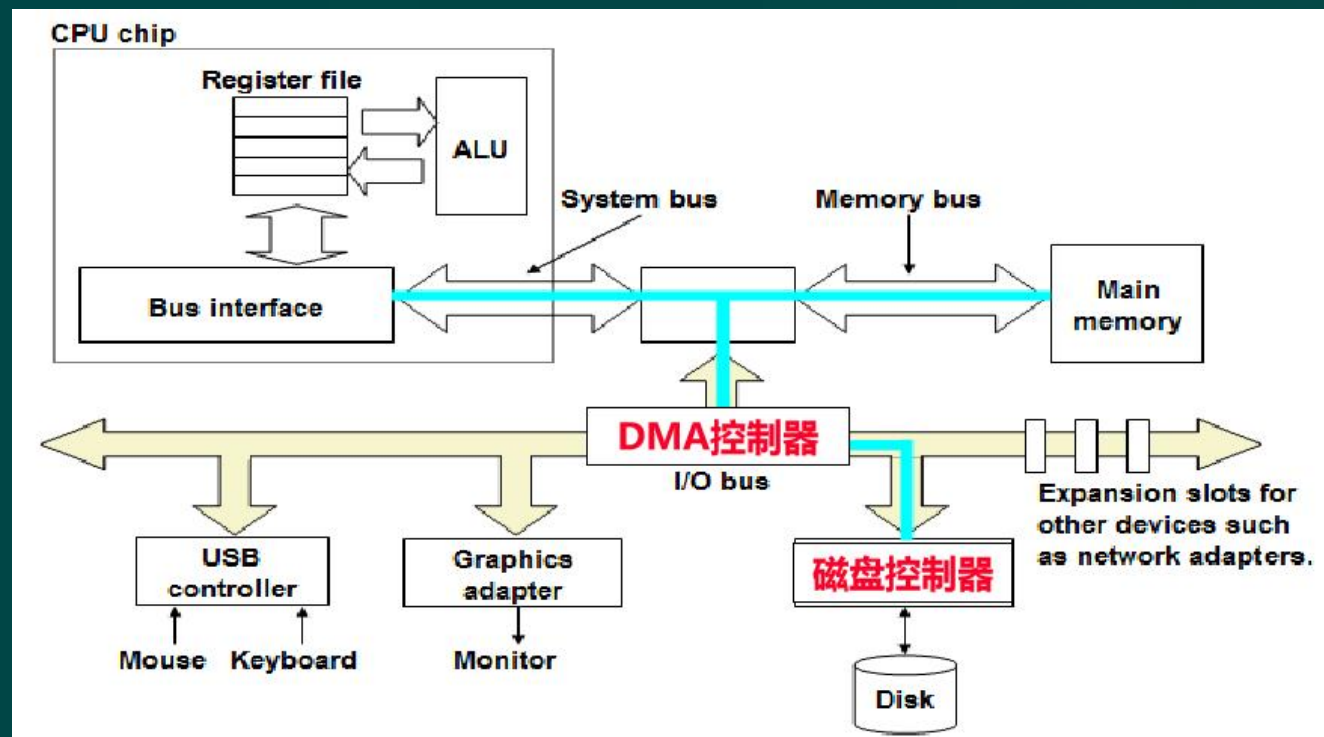
存取时间=寻道时间+旋转等待时间+数据传输时间

- 寻道时间为磁头移动到指定磁道所需时间；
- 旋转等待时间指指定扇区旋转到磁头下方所需时间；
- 数据传输时间（Transfer Time）指传输一个扇区的时间（大约0.01ms/扇区）。

◎第五章 程序的存储访问

考点10 磁盘存储器的连接

图是磁盘驱动器（简称磁盘）通过磁盘控制器与CPU、主存连接的示意图。磁盘控制器连接在I/O总线上，I/O总线与其他系统总线（如处理器总线、存储器总线）之间用桥接器连接。



◎第五章 程序的存储访问

考点11 磁盘存储器的连接

- MROM在芯片生产过程中制造，生产后不可编程，故可靠性高，但生产周期长、不灵活；
- PROM只可编程一次，不灵活；
- EPROM可擦除可编程多次，但采用MOS工艺，且擦除时只能抹除所有信息，不灵活且速度慢；
- EEPROM可以字为单位擦除，擦除次数可达数千次，且数据可保持一二十年。

◎第五章 程序的存储访问

考点11 磁盘存储器的连接

闪速存储器也称闪存或Flash存储器，是一种非易失性读/写存储器，兼有RAM和ROM的优点，且功耗低、集成度高，无需后备电源。

这种器件沿用了EPROM的简单结构和浮栅/热电子注入的编程写入方式，又兼备EEPROM的可擦除特点，且可在计算机内进行擦除和编程写入，因此又称为快擦型EEPROM。目前广泛使用的U盘和存储卡等都属于闪存。

◎第五章 程序的存储访问

考点12 固态硬盘

固态硬盘，也称为电子硬盘。它并不是一种磁表面存储器，而是使用NAND闪存组成的外存，与U盘没有本质差别，只是容量更大，存取性能更好。它用闪存颗粒代替了磁盘作为存储介质，以区块写入和擦除的方式进行数据的读取和写入。

◎第五章 程序的存储访问

考点13 cache的基本工作原理

除了提高DRAM芯片本身的速度和采用并行结构技术以外，加快CPU访存速度的主要方式之一是在CPU和主存之间增加高速缓冲存储器（简称高速缓存或cache）。

为便于cache和主存交换信息，一般将cache和主存空间划分为大小相等的区域。主存中的区域称为块（Block），也称为主存块，它是cache和主存之间的信息交换单位；cache中存放一个主存块的区域称为行（Line）或槽（Slot）。

◎第五章 程序的存储访问

考点13 cache的基本工作原理

1. cache的有效位

在系统启动或复位时，每个cache行都为空，其中的信息无效，只有装入了主存块后信息才有效。为了说明cache行中的信息是否有效，每个cache行需要一个有效位（ValidBit）。

有了有效位，就可通过将有效位清0来淘汰某cache行中的主存块，称为冲刷（Flush），装入一个新主存块时，再将有效位置1。

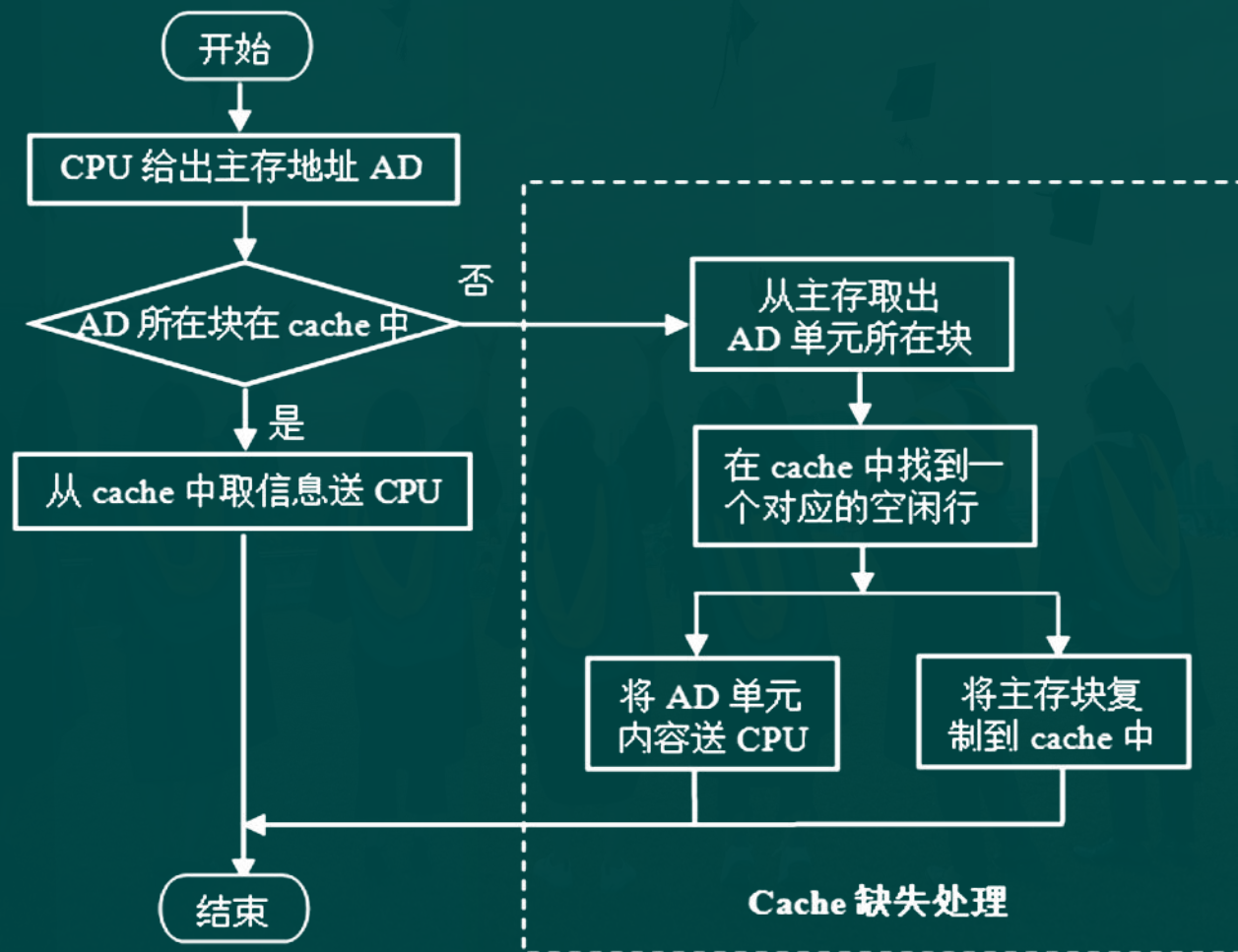
◎第五章 程序的存储访问

考点13 cache的基本工作原理

2. CPU在cache中的访问过程

CPU执行程序时需要从主存取指令或读/写数据，此时先检查cache中是否有要访问的信息。

图给出了带cache的CPU执行一次访存操作的过程。



◎第五章 程序的存储访问

考点13 cache的基本工作原理

3. cache-主存层次的平均访问时间

在访存过程中需要判断所访问信息是否在cache中。

- 若CPU访问单元所在的块在cache中，则称cache命中（Hit），命中概率称为命中率p（Hit Rate），它等于命中次数与访问总次数之比；
- 若不在cache中，则为不命中或缺失（Miss），其概率称为缺失率（Miss Rate），它等于不命中次数与访问总次数之比。

CPU在cache-主存层次的平均访问时间为

$$T_a = p \times T_c + (1-p) \times (T_c + T_m) = T_c + (1-p) \times T_m$$

◎第五章 程序的存储访问

考点14 cache的基本工作原理

- (1) 直接映射：每个主存块映射到cache的固定行中。
- (2) 全相联映射：每个主存块映射到cache的任意行中。
- (3) 组相联映射：每个主存块映射到cache的固定组的任意行中。

◎第五章 程序的存储访问

考点14 cache的基本工作原理

1. 直接映射

直接映射的基本思想是把主存的每一块映射到固定的一个cache行中，也称模映射，其映射关系如下：

$$\text{cache行号} = \text{主存块号} \bmod \text{cache 行数}$$

◎第五章 程序的存储访问

考点14 cache的基本工作原理

2. 全相联映射

全相联的基本思想是主存块可装入任意cache行中。因此，全相联cache需比较所有cache行标记才能判断是否命中，同时，主存地址中无需cache行索引，只有标记和块内地址两个字段。

全相联方式下，只要有空闲cache行，就不会发生冲突，因而块冲突概率低。

标记	块内地址
----	------

◎第五章 程序的存储访问

考点14 cache的基本工作原理

3. 组相联映射

直接映射和全相联的优缺点正好相反，二者结合可以取长补短，从而形成组相联方式。组相联的主要思想是，将cache所有行分成 2^q 个大小相等的组，每组有 2^s 行。

每个主存块映射到cache固定组中的任意一行，即采用组间模映射、组内全映射的方式，映射关系如下。

$$\text{cache组号} = \text{主存块号} \bmod \text{cache组数}$$

◎第五章 程序的存储访问

考点14 cache的基本工作原理

假设主存地址有 n 位，块内地址占 b 位，有 2^t 个组群，则 $n=t+q+b$ ，主存地址被划分为以下三个字段：

标记 (t)	Cache组号 (q)	块内地址 (b)
------------	-----------------	-----------------

其中，高 t 位为标记，中间 q 位为组号（也称组索引），剩下的 b 位为块内地址。

◎第五章 程序的存储访问

考点15 cache的基本工作原理

(1) 先进先出算法 (FIFO) 的基本思想是：

总是选择最早装入cache的主存块被替换掉。这种算法实现起来较方便，但不能正确反映程序的访问局部性，因为最先进入的主存块也可能是目前经常要用的，因此，这种算法有可能产生较大的缺失率。

◎第五章 程序的存储访问

考点15 cache的基本工作原理

(2) 最近最少用算法 (LRU) 的基本思想是：

总是选择近期最少使用的主存块被替换掉。这种算法能比较正确地反映程序的访问局部性，因为当前最少使用的块一般来说也是将来最少被访问的。但是，它的实现比FIFO算法要复杂一些。

LRU算法用计数值来记录主存块的使用情况，通过硬件修改计数值，并根据计数值选择淘汰某个cache行中的主存块。这个计数值称为LRU位，其位数与cache组大小有关。

◎第五章 程序的存储访问

考点15 cache的基本工作原理

- (3) 最不经常用算法（LFU）的基本思想是
替换掉cache中引用次数最少的块。LFU也用与每个行相关的计数器来实现。这种算法与LRU有点类似，但不完全相同。

◎第五章 程序的存储访问

考点15 cache的基本工作原理

(4) 随机替换算法

从候选行的主存块中随机选取一个淘汰掉，与使用情况无关。模拟试验表明，随机替换算法在性能上只稍逊于基于使用情况的算法，而且代价低。

◎第五章 程序的存储访问

考点16 cache的基本工作原理

1. 全写法

全写法的基本做法是：若写命中，则同时写 cache 和主存；若写不命中，则有以下两种处理方式。

（1）写分配法。先在主存块中更新相应存储单元，然后分配一个cache行，将更新后的主存块装入到分配的cache行中。这种方式可以充分利用空间局部性，但每次写不命中都要从主存读一个块到cache中，增加了读主存块的开销。

（2）非写分配法。仅更新主存单元而不装入主存块到cache中。这种方式可以减少读入主存块的时间，但没有很好利用空间局部性。

◎第五章 程序的存储访问

考点16 cache的基本工作原理

2. 回写法

回写法的基本做法是：

若写命中，则信息只被写入 cache 而不被写入主存；

若写不命中，则在cache中分配一行，将主存块调入该cache行中并更新相应单元的内容。

该方式下在写不命中时，通常采用写分配法进行写操作。

◎第五章 程序的存储访问

考点17 虚拟地址空间

图给出了在IA-32+Linux系统中一个进程（如hello程序对应的进程）的虚拟地址空间映像。虚拟地址空间分为两大部分：操作系统内核区和用户虚拟存储空间，分别简称为内核空间（Kernel Space）和用户空间（User Space）。



◎第五章 程序的存储访问

考点17 虚拟地址空间

虚拟存储管理机制为程序提供了一个极大的虚拟地址空间（也称逻辑地址空间），

- ①每个进程具有一致的虚拟地址空间，从而可以简化存储管理，简化链接器的设计和实现，也简化了程序的加载过程；
- ②把主存看成是硬盘存储器的一个缓存，在主存中仅保存当前活动的程序段和数据区，并根据需要在硬盘和主存之间进行信息交换，通过这种方式，使有限的主存空间得到了有效利用；
- ③每个进程的虚拟地址空间是私有的、独立的，因此，可以保护各自进程不被其他进程破坏。

◎第五章 程序的存储访问

考点18 虚拟地址空间

1. 段式虚拟存储器

根据程序的模块化特性，可按程序的逻辑结构将其划分成多个相对独立的部分，这些相对独立的部分称为段（Segment）。

分段方式下，将主存空间按实际程序中的段来划分，并通过段表中的段表项记录每个段在主存中的基址、段长、访问权限、使用和装入情况等。每个进程有一个段表，指令给出的虚拟地址即为段内偏移。可将其加上对应段的基址得到实际访问的主存物理地址。

段式虚拟存储器实现机制较简单，硬件实现成本低。适合简单的嵌入式系统和实时系统。由于段的粒度较大，不易管理，且易产生主存碎片。因此现代操作系统通常不使用段式虚拟存储管理方式。

◎第五章 程序的存储访问

考点18 虚拟地址空间

2. 页式虚拟存储器

现代操作系统主要采用页式虚拟存储管理方式。在页式虚拟存储系统中，虚拟地址空间被划分成大小相等的页，外存和主存之间按页（Page）为单位交换信息。

- 虚拟地址空间中的页称为虚拟页、逻辑页或虚页，简称VP（Virtual Page）；
- 主存空间也被划分成同样大小的页框（页帧），也称为物理页或实页，简称PF（Page Frame）或PP（Physical Page）。

◎第五章 程序的存储访问

考点18 虚拟地址空间

页式虚拟存储管理采用请求分页思想，仅将当前需要的页从外存调入主存，而其他不活跃的页保留在外存。当访问信息所在页不在主存时，CPU抛出缺页异常，此时操作系统从外存将缺失页装入主存。

◎第五章 程序的存储访问

考点18 虚拟地址空间

(1) 页表

进程中每个虚拟页在页表中都有一个对应表项，称为页表项。如图所示，页表项内容包括该虚拟页的存放位置、装入位（Valid）、修改位（Dirty）、使用位、访问权限位和禁止缓存位等。



◎第五章 程序的存储访问

考点18 虚拟地址空间

(2) 地址转换

对于采用虚存机制的系统，指令中给出的地址是虚拟地址，因此，CPU执行指令时首先要将虚拟地址转换为主存物理地址。才能到主存取指令和数据。

地址转换（Address Translation）工作由CPU中的存储器管理部件（MMU）完成。

◎第五章 程序的存储访问

考点18 虚拟地址空间

虚拟地址分两个字段：高位字段为虚拟页号（即虚页号或逻辑页号），低位字段为页内偏移地址（简称页内地址）。

虚拟页号	页内偏移地址
------	--------

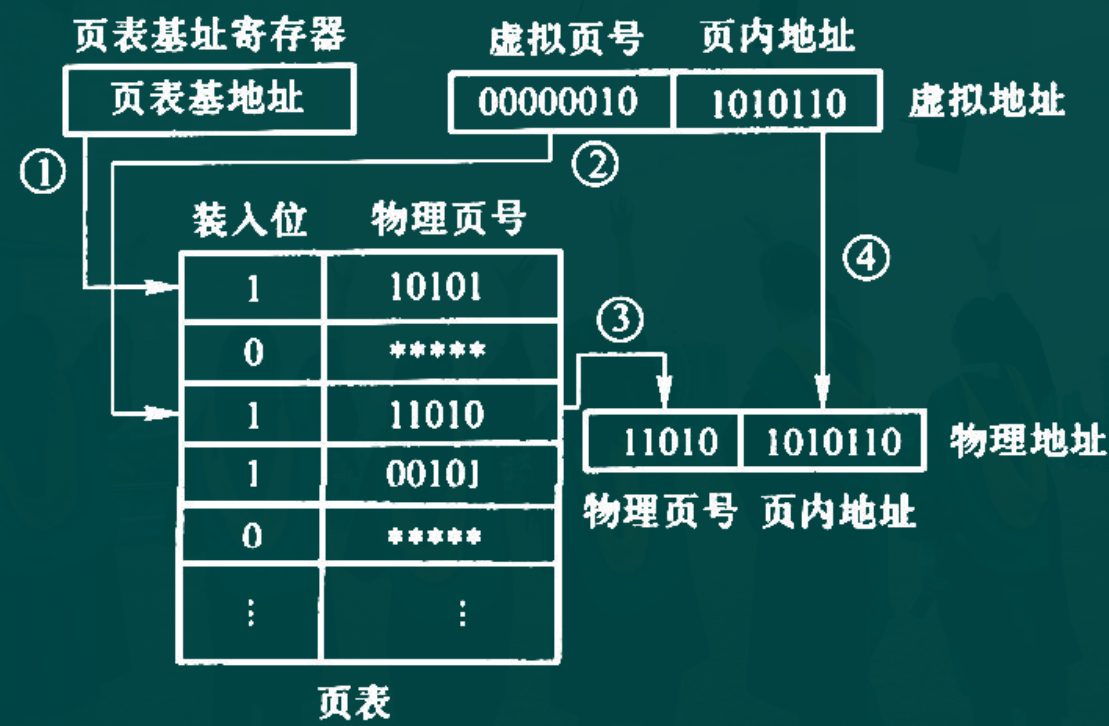
主存物理地址也分两个字段：高位字段为物理页号，低位字段为页内偏移地址。由于虚拟页和物理页大小一样。所以两者的页内偏移地址相等。

物理页号	页内偏移地址
------	--------

◎第五章 程序的存储访问

考点18 虚拟地址空间

分页式虚存方式下，地址转换过程如图所示。首先根据页表基址寄存器的内容找到主存中对应的页表起始位置。然后将虚拟页号作为索引，找到对应页表项，若装入位为1，则取出物理页号和虚拟地址中的页内地址拼接，形成访问主存时实际的物理地址；若装入位为0，则说明缺页，需要操作系统进行缺页处理。

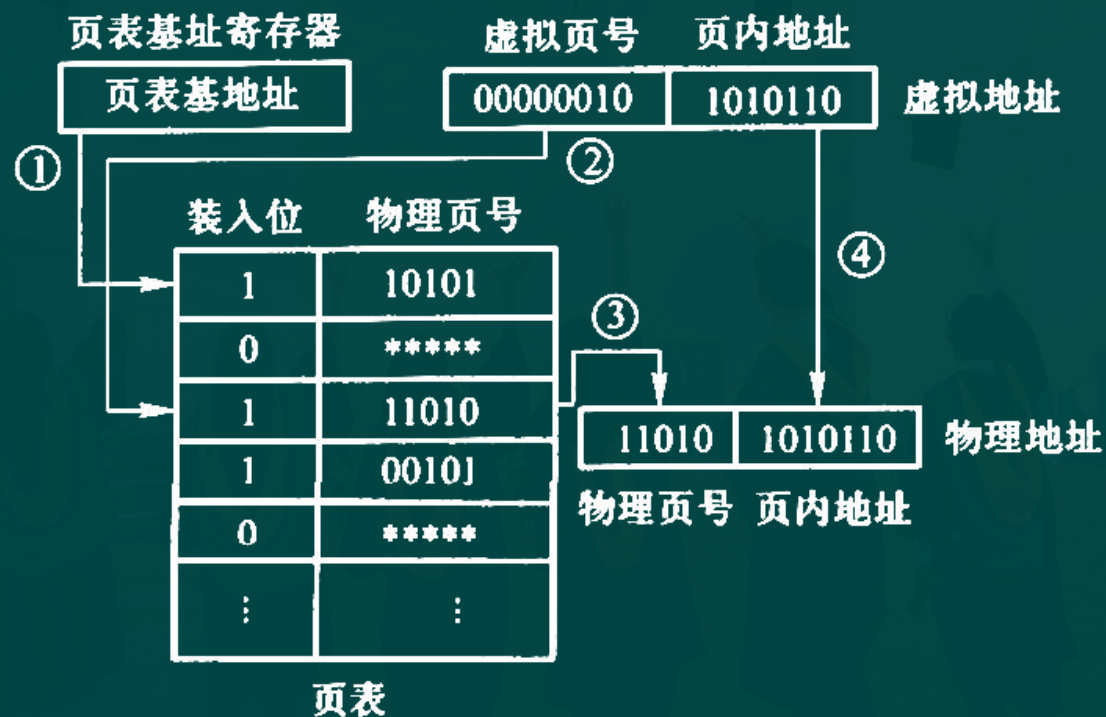


◎第五章 程序的存储访问

考点18 虚拟地址空间

(3) 快表

从地址转换过程可看出，访存时首先要到主存查页表，然后才能根据转换得到页表的物理地址再访问主存。如果缺页，则还要进行页面替换、页表修改等，访问主存的次数就更多。显然，采用虚拟存储器机制后，CPU执行一条指令的访存次数反而增加了。



◎第五章 程序的存储访问

考点18 虚拟地址空间

为了减少访存次数，通常利用程序访问的局部性特性，将页表中最活跃的几个页表项复制到高速缓存中，这种在高速缓存中的页表项组成的页表称为转换后备缓冲器（Translation Lookaside Buffer, TLB）或快表，相应地称主存中的页表为慢表。

这样，MMU进行地址转换时，首先查询快表，若命中，则直接取出快表中的页表项进行地址转换；若缺失，则访问主存中的慢表。因此，快表是加速地址转换的有效方法。

◎第五章 程序的存储访问

考点18 虚拟地址空间

从上图可看出，CPU访存过程中存在以下三种缺失情况。

- 1) TLB缺失 (TLB Miss)：要访问的虚拟页对应的页表项不在TLB中。
- 2) cache缺失 (Cache Miss)：要访问的主存块不在cache中。
- 3) 缺页 (Page Miss)：要访问的虚拟页不在主存中。

◎第五章 程序的存储访问

考点18 虚拟地址空间

3. 段页式虚拟存储器

在段页式虚拟存储器中，程序按模块分段，段内再分页，用段表和页表（每段一个页表）进行两级定位管理。段表中每个表项对应一个段，每个段表项中包含一个指向该段页表起始位置的指针，以及该段其他的控制和存储保护信息，由页表指明该段各页在主存中的位置以及是否装入、修改等状态信息。

◎第五章 程序的存储访问

考点19 虚拟地址空间

为了支持操作系统实现存储保护，指令集架构和硬件必须具有以下三种基本功能。

- 1) 使部分CPU状态只能由操作系统内核程序访问而用户进程只能读不能写，或者根本不能访问。
- 2) 支持至少两种特权模式。操作系统内核程序比用户程序具有更多的特权，例如，内核程序可以执行用户程序不能执行的特权指令，内核程序可以访问用户程序不能访问的存储空间等，为此，需要让内核程序 and 用户程序运行在不同的特权级别或特权模式。
- 3) 提供在不同特权模式之间相互切换的机制。

◎第五章 程序的存储访问

考点19 虚拟地址空间

1. 访问权限保护

访问权限保护检查是否发生了访问越权。若实际访问操作与访问权限不符，则发生存储保护错。

2. 存储区域保护

存储区域保护检查是否发生了地址越界，也即是否访问了不该访问的区域。例如、程序是否访问了“空洞”区域，访问区间是否超越了最大段长规定的地址。

第六章 程序中I/O操作的实现

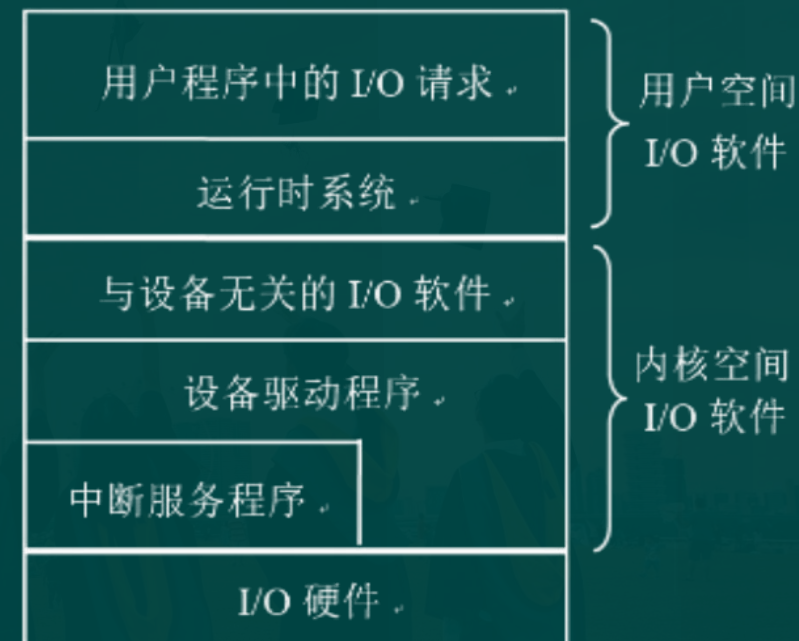
◎第六章 程序中的I/O操作实现

考点1 I/O子系统概述

与计算机系统一样，I/O子系统也采用层次结构。

I/O子系统包含I/O软件和I/O硬件两大部分。

- I/O软件包括最上层提出I/O请求的用户空间I/O软件(称为用户I/O软件)和在底层操作系统中对I/O进行具体管理和控制的内核空间I/O软件(称为系统I/O软件)。系统I/O软件又分三个层次，分别是与设备无关的I/O软件层、设备驱动程序层和中断服务程序层。
- I/O硬件在操作系统内核空间I/O软件的控制下完成具体的I/O操作。



◎第六章 程序中的I/O操作实现

考点1 I/O子系统概述

操作系统在I/O子系统中承担极其重要的作用，这主要是由I/O子系统以下三个特性决定的。

- 共享性。
- 复杂性。
- 异步性。

◎第六章 程序中的I/O操作实现

考点1 I/O子系统概述

I/O子系统工作的大致过程如下：首先，CPU在用户态下运行用户进程，当CPU执行到系统调用封装函数对应的指令序列中的陷阱指令时，会从用户态陷入内核态；转到内核态执行后，CPU会根据陷阱指令执行时EAX寄存器中的系统调用号，选择执行相应的系统调用服务例程；在系统调用服务例程的执行过程中，可能需要调用具体设备的驱动程序；在设备驱动程序执行过程中启动外设工作，外设准备好数据或准备好接收数据后，就发出中断请求：CPU响应中断后，就调出中断服务程序执行。在中断服务程序中控制主机与设备进行具体的数据交换。

◎第六章 程序中的I/O操作实现

考点1 用户程序中的I/O函数

在用户空间I/O软件中，用户程序可以通过调用特定的I/O函数的方式提出I/O请求。

在UNIX/Linux系统中，用户程序使用的I/O函数可以是C标准I/O库函数或系统调用封装函数，前者如文件I/O函数fopen、fread、fwrite和fclose或控制台I/O函数printf、scanf等；后者如open、read、write和close等。

◎第六章 程序中的I/O操作实现

考点1 用户程序中的I/O函数

在Windows系统中，用户程序同样可以调用C标准I/O库函数，此外，还可以调用Windows提供的API函数，如文件I/O函数CreateFile、ReadFile、WriteFile、CloseHandle和控制台I/O函数ReadConsole、WriteConsole等。

◎第六章 程序中的I/O操作实现

考点2 文件的基本概念

Linux操作系统是一个类UNIX系统，其文件格式和文件操作相关的系统调用等与UNIX类似。在UNIX系统中，所有I/O操作都通过读写文件实现，所有外设，包括网络(套接字socket)、终端设备(键盘和显示器)等，都被看成文件。

◎第六章 程序中的I/O操作实现

考点2 文件的基本概念

根据文件中的每个字节是否为可读的ASC II 码，可将文件分成ASC II 文件和二进制文件两类。

- ASC II 文件也称为文本文件，由多个正文行组成，每行以换行符('\n')结束，其中每个字节是一个字符。通常，终端设备上的标准输入文件和标准输出文件是ASC II
- 文件硬盘上的普通文件则可能是文本文件或二进制文件，如可重定位文件和可执行文件都是二进制文件，而源程序文件则是ASC II 文件。

◎第六章 程序中的I/O操作实现

考点2 文件的基本概念

1. 创建文件

通常，用户程序在读写一个文件前，必须告知系统将要对该文件进行何种操作，是读、写、添加还是可读可写，该告知操作通过打开或创建一个文件来实现。

2. 打开文件

打开文件时，系统会检测文件是否存在、用户是否有访问权限等。若成功，则系统会返回一个非负整数作为文件描述符。

◎第六章 程序中的I/O操作实现

考点2 文件的基本概念

3. 设置文件读/写位置

每个文件都有一个当前读/写位置，表示相对于文件最开始处的字节偏移量，初始时为0。用户程序中可通过系统调用封装函数 `lseek()` 设置文件读/写位置。

4. 读文件和写文件

用户程序可以向被创建的新文件中写入信息，也可以从一个已存在且打开后的文件中读或写信息。写文件操作将从当前读/写位置 k ($k \geq 0$) 处写入 n ($n > 0$) 个字节，因而写入后文件当前读/写位置为 $k+n$ 。

◎第六章 程序中的I/O操作实现

考点2 文件的基本概念

5. 关闭文件

完成文件读/写等操作后，用户程序需要通知系统关闭文件，表示用户程序不再对该文件进行任何操作。关闭文件时，系统将释放文件创建或打开相关的数据结构所在存储区，并回收文件描述符。无论一个进程因为何种原因终止，系统都会关闭其打开的所有文件，以释放相应的存储资源。

◎第六章 程序中的I/O操作实现

考点3 系统级I/O函数

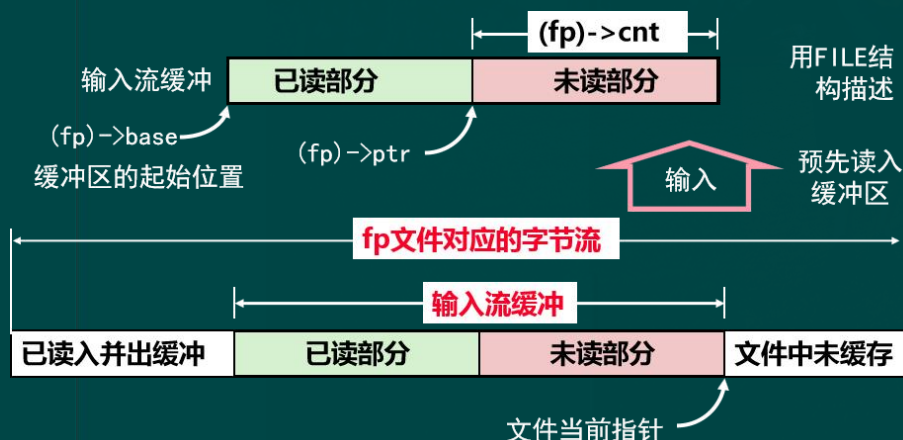
前面提到，与I/O操作相关的系统调用封装函数属于系统级I/O函数。在UNIX/Linux系统中，常用的这类函数有`creat`、`open`、`read`、`write`、`lseek`、`stat/fstat`、`close`等，它们的调用形式及其功能说明如下。使用以下函数时必须包含相应的头文件(如`unistd.h`等)。

◎第六章 程序中的I/O操作实现

考点4 C标准I/O库函数

1. 输入/输出流缓冲区

C标准I/O库函数将一个打开的文件抽象为一个类型为FILE的“流”模型。上面曾提到，C标准I/O库函数中，文件是用一个指向特定结构的指针来标识的，这个特定结构就是FILE结构，它描述了包含文件描述符fd在内的一组信息。FILE结构在头文件stdio.h中描述，此外，stdio.h文件中还对其他与标准I/O有关的常量、数据结构、函数和宏等进行了定义。



◎第六章 程序中的I/O操作实现

考点4 C标准I/O库函数

输出缓冲区的属性有三种：全缓冲_`IOFBF` (fully buffered)、行缓冲_`IOLBF` (line buffered), 非缓冲_`IONBF` (no buffering)。

- 全缓冲的含义是，即使遇到换行符也不会写文件，只有当缓冲区满时才会将缓冲区内容真正写入文件fd中；
- 行缓冲的含义是，遇到换行符或者缓冲区满就将缓冲区内容写文件fd；
- 非缓冲的含义是直接写到文件fd中。普通文件的缓冲区属性为全缓冲。

◎第六章 程序中的I/O操作实现

考点4 C标准I/O库函数

2. 标准文件

在上述stdio.h文件中，定义了三个特殊的标准文件，分别是标准输入(stdin)、标准输出(stdout)和标准错误(stderr)，它们分别定义为打开的文件列表中的前三个文件，对应的文件描述符fd分别是0、1和2，它们在结构数组_iob中前三项的初始化定义如下：

```
FILE _iob[OPENMAX] = { /*stdin, stdout, stderr:*/  
    { 0, (char *)0, (char *)0, READ, 0, },  
    { 0, (char *)0, (char *)0, _WRITE, 1, },  
    { 0, (char *)0, (char *)0, _WRITE|_UNBUF, 2, },  
};
```

◎第六章 程序中的I/O操作实现

考点5 设备无关I/O软件层

1. 文件系统概述

一方面。文件系统要为上层的用户和应用程序提供文件抽象以及文件的创建、打开、读/写和关闭等所有操作接口：

一方面。文件系统需要将抽象的文件标识(文件名和文件描述符)与具体的硬件设备建立关联。并通过相应的设备驱动程序实现系统调用接口规定的操作。要实现这些功能，文件系统必须提供一套用于存储和管理文件数据及其元数据的机制。

FAT、NTFS、Ext4等文件系统FAT、NTFS、Ext4等文件系统

◎第六章 程序中的I/O操作实现

考点5 设备无关I/O软件层

2. 缓存层

I/O设备的工作速度较慢，为了提升I/O请求的处理效率，操作系统充分利用数据访问的局部性特点，在内核空间对应主存区中开辟一块空间作为高速缓存，用于存储最近访问的文件数据。作为高速缓存的主存RAM区可称为高速缓存RAM。

◎第六章 程序中的I/O操作实现

考点5 设备无关I/O软件层

3. 通用块设备I/O层

通用块设备I/O层提供了所有像磁盘、SSD和光盘之类块设备的统一抽象，负责调用具体的设备驱动程序向设备发起I/O请求。同时，通用块设备I/O层为这类设备设置统一的逻辑块大小。例如，无论磁盘扇区和光盘扇区有多大，所有逻辑数据块的大小均相同。高层的文件系统只需与这一抽象设备交互，从而简化了数据定位等处理。

◎第六章 程序中的I/O操作实现

考点6 设备无关I/O软件层

1. 程序直接控制I/O方式

程序直接控制I/O方式的基本思想是直接通过查询程序来控制主机和外设的数据交换。因此，也称为查询或轮询(Polling)方式。该方式在查询程序中通过I/O指令读出外设或其设备控制器的状态后，根据状态来控制外设和主机的数据交换。

◎第六章 程序中的I/O操作实现

考点6 设备无关I/O软件层

程序直接控制I/O方式的特点是简单、易控制，设备控制器中的控制电路也简单。但是，CPU需要从设备控制器中读取状态信息，并在外设未就绪时一直处于忙等待。如果外设的速度比CPU慢很多，CPU等待外设完成任务将浪费大量处理器时间。

◎第六章 程序中的I/O操作实现

考点6 设备无关I/O软件层

2. 中断控制I/O方式

中断控制I/O方式的基本思想是，当需要进行I/O操作时，首先启动外设进行第一个数据的I/O操作，然后阻塞请求I/O的用户进程，并调度其他进程到CPU上执行，期间，设在设备控制器的控制下工作。外设完成I/O操作后。向CPU发送一个中断请求信号，CPU检测到该信号后，则进行上下文切换，调出相应的中断服务程序执行。

中断服务程序好启动后续数据的I/O操作，然后返回到被打断的进程继续执行。

◎第六章 程序中的I/O操作实现

考点6 设备无关I/O软件层

3. DMA控制I/O方式

直接存储器访问(Direct Memory Access, DMA)控制I/O方式使用专门的DMA接口硬件直接控制在外设和主存之间交换数据,此时数据不经过CPU。通常把该接口硬件称为DMA控制器。

DMA控制器与设备控制器一样。其中也有若干寄存器,包括主存地址寄存器、设备地址寄存器、字计数器、控制寄存器等。还有其他控制逻辑,用于控制设备通过总线与主存直接交换数据。

◎第六章 程序中的I/O操作实现

考点6 设备无关I/O软件层

DMA控制I/O过程如下：首先进行DMA初始化，然后发送“启动DMA传送”命令启动外设工作。之后，CPU阻塞请求I/O的用户进程，转去执行其他进程。在CPU执行其他进程的过程中，DMA控制器控制外设和主存交换数据，此时CPU和外设并行工作。DMA控制器每完成一个数据的传送，就将字计数器减1，并更新主存地址，其功能可看作使用专用硬件来执行memcpy函数。

◎第六章 程序中的I/O操作实现

考点6 设备无关I/O软件层

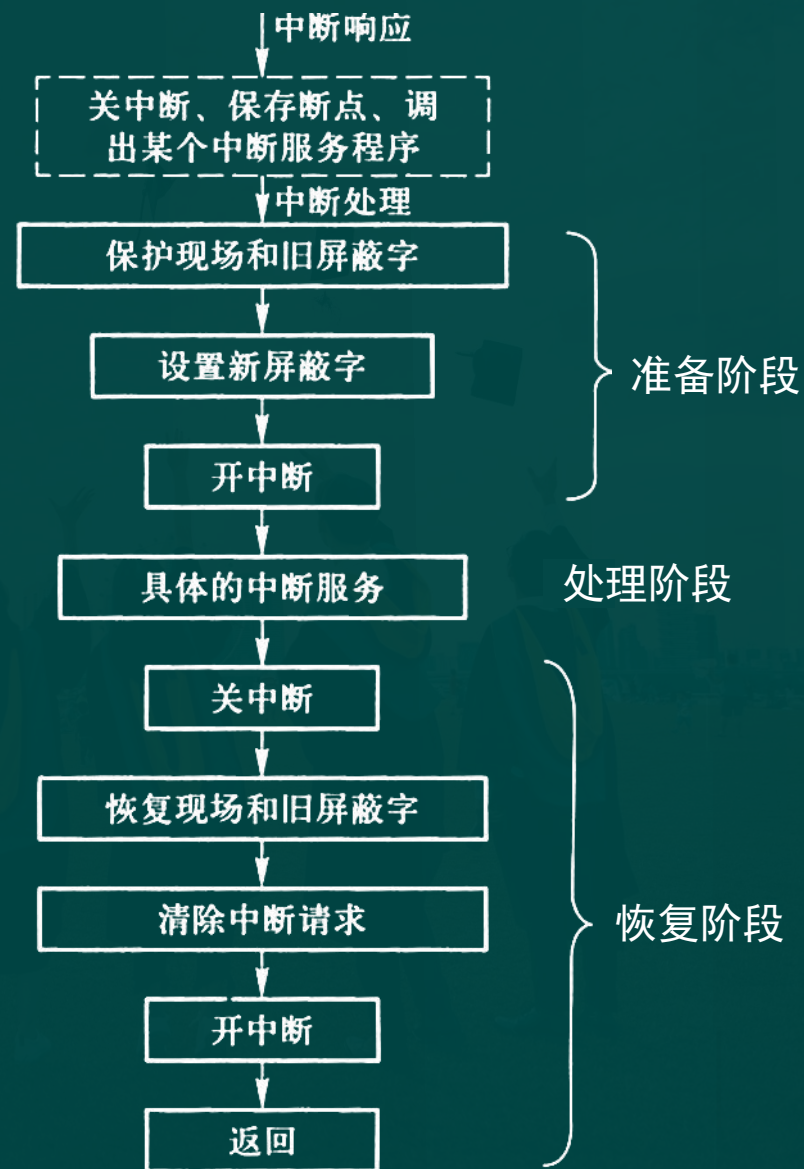
DMA控制I/O方式下，CPU只需在最初的DMA初始化和最后处理“DMA结束”中断时介入，而无须参与整个数据传送过程，因而CPU用于I/O的开销非常小。

◎第六章 程序中的I/O操作实现

考点7 设备无关I/O软件层

图给出了整个中断过程，包括中断响应和中断处理两个阶段。中断响应完全由硬件完成。包括关中断。保存断点，并跳转到预先设定的中断服务程序，进入中断处理阶段。

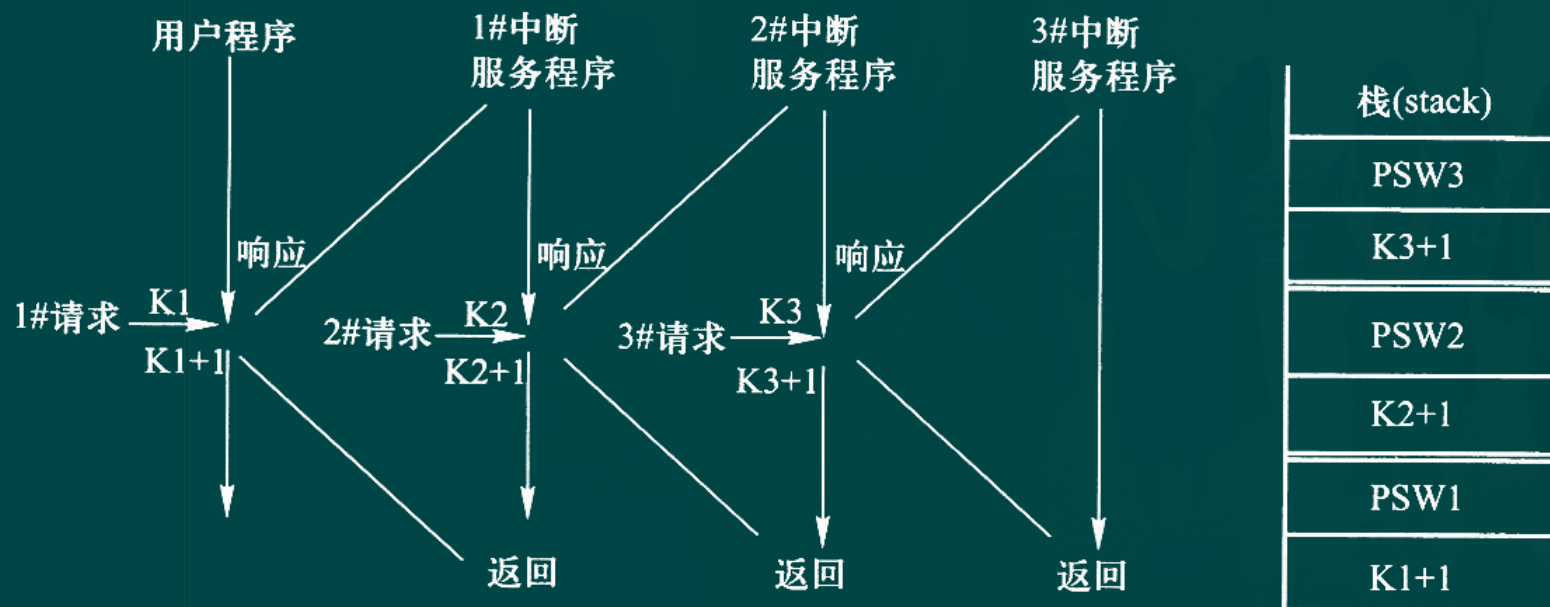
中断服务程序包含三个阶段：准备阶段、处理阶段和恢复阶段。



◎第六章 程序中的I/O操作实现

考点7 设备无关I/O软件层

在中断处理过程中，若又来到了优先级更高的新中断请求，CPU应立即暂停当前执行的中断服务程序，转去处理新中断，这种情况称为多重中断或中断嵌套，如图



◎第六章 程序中的I/O操作实现

考点8 设备无关I/O软件层

I/O设备又称外围设备、外部设备，简称外设，是计算机系统与人类或其他计算机系统之间交换信息的装置。操作系统为了统一管理I/O设备，通常将I/O设备分成两类：字符设备和块设备。

字符设备是以字符为单位向主机发送或从主机接收字符流的设备。字符设备传送的字符流不能形成数据块，无法定位和寻址。

◎第六章 程序中的I/O操作实现

考点8 设备无关I/O软件层

输入设备的功能是把数据、命令、字符、图形、图像、声音或电流、电压等信息，以计算机可以接收或识别的二进制编码形式输入计算机中

输出设备的功能是把计算机处理的结果，变成最终可以被理解的数据、文字、图形、图像和声音等信息。例如，显示器、打印机和绘图仪等都是输出设备。

还有一类主要用于计算机和计算机之间通信的设备，称为机-机通信设备，例如，网络接口、调制解调器、数/模和模/数转换器等。通常，大多数机-机通信设备也可看作字符设备。

◎第六章 程序中的I/O操作实现

考点8 设备无关I/O软件层

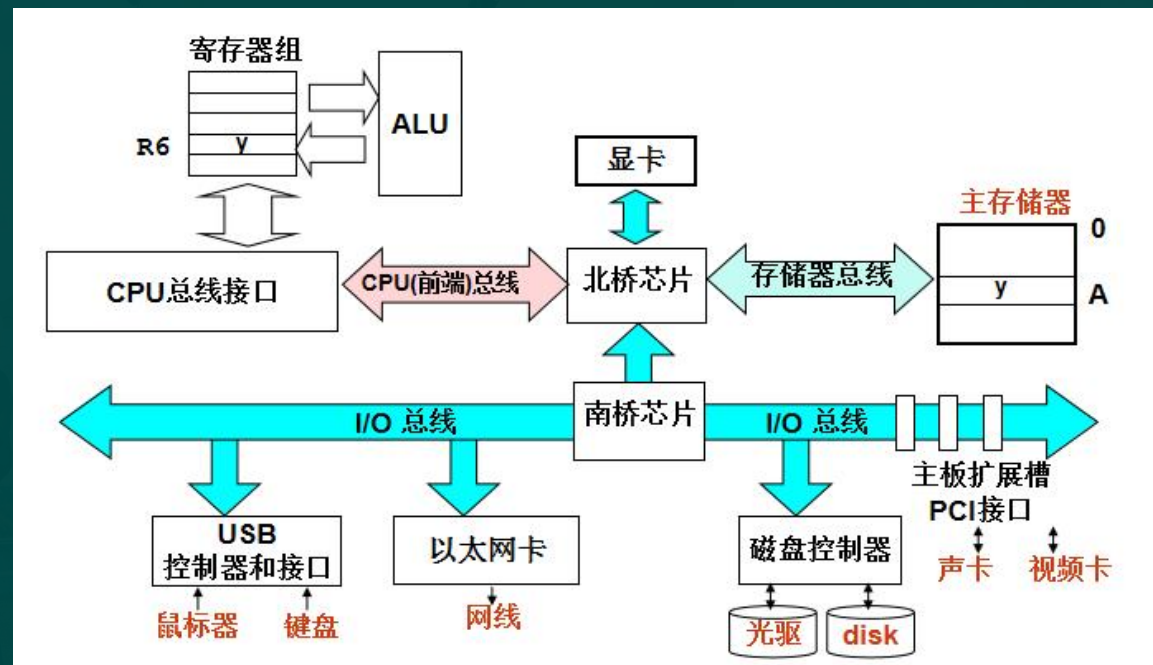
块设备以一个固定大小的数据块为单位与主机交换信息。块设备中的数据块大小通常在512字节以上，通常按照某种组织方式对其进行读/写，每个数据块都有唯一的位置信息，因而是可寻址的。

典型的块设备是外部存储器，例如，磁盘驱动器、固态硬盘等。

◎第六章 程序中的I/O操作实现

考点9 设备无关I/O软件层

如图所示，CPU与主存之间由处理器总线（也称为前端总线）和存储器总线相连，各类I/O设备通过相应的设备控制器（如USB控制器、以太网卡、磁盘控制器）连接到I/O总线上，而I/O总线通过芯片组与主存和CPU连接。



◎第六章 程序中的I/O操作实现

考点9 设备无关I/O软件层

1. 处理器总线

早期Intel微处理器的处理器总线称为前端总线(Front Side Bus, FSB), 它是主板上最快的总线, 主要用于处理器与北桥芯片之间交换信息。

◎第六章 程序中的I/O操作实现

考点9 设备无关I/O软件层

QPI总线是一种基于包传输的高速点对点连接协议，采用差分信号与专门的时钟信号进行传输。QPI总线有20条数据线，发送方(TX)和接收方(RX)有各自的时钟信号，每个时钟周期传输两次。一个QPI数据包包含80位，需要两个时钟周期或4次传输才能完成整个数据包的传送。在每次传输的20位数据中，有效数据占16位，其余4位用于循环冗余校验，以提高系统的可靠性。由于QPI是双向的，在发送的同时也可以接收另一端传输的数据，因此，每个QPI总线的带宽计算公式如下。

$$\text{每秒传输次数} \times \text{每次传输的有效数据} \times 2$$

◎第六章 程序中的I/O操作实现

考点9 设备无关I/O软件层

2. 存储器总线

早期的存储器总线由北桥芯片控制，处理器通过北桥芯片和主存、图形卡(显卡)以及南桥芯片进行互连。但后来的处理器芯片(如Core i7)集成了主存控制器，因而存储器总线直接连接到处理器。

◎第六章 程序中的I/O操作实现

考点9 设备无关I/O软件层

3. I/O总线

I/O总线用于为系统中各种I/O设备提供输入/输出通路，其物理表现通常是主板上的I/O扩展槽。早期的第一代I/O总线有XT总线、ISA总线、EISA总线、VESA总线，这些I/O总线早已被淘汰；第二代I/O总线包括PCI、AGP、PCI-X；第三代I/O总线是PCI-Express（可简称为PCI-e）。

前两代I/O总线采用并行传输的同步总线，而PCI-e总线采用串行传输方式。两个PCI-e设备之间以一个链路(Link)相连，每个链路可包含多条通路(Lane)，可能的通路数为

1、2、4、8、16或32，PCI-eXn表示具有n个通路的PCI-e链路

◎第六章 程序中的I/O操作实现

考点9 设备无关I/O软件层

PCI-e1.0规范支持通路中每个方向的发送或接收速率为2.5Gb/s。因此，PCI-e1.0总线的总带宽计算公式(单位为GB/s)如下。

$$2.5\text{Gb/s} \times 2 \times \text{通路数} / 10$$

根据上述公式可知，在PCI-e1.0规范下，PCI-ex1的总带宽为0.5GB/s；

PCI-ex2的总带宽为1GB/s；

PCI-e×16的总带宽为8GB/s。

◎第六章 程序中的I/O操作实现

考点10 设备无关I/O软件层

I/O接口的主要职能包括以下几个方面。

- 1) 数据缓冲。
- 2) 错误和就绪检测。提供错误和就绪检测逻辑，并将结果保存在状态寄存器，供CPU查用。状态信息包括各类就绪和错误信息，如外设是否完成打印或显示、是否准备好输入数据供CPU读取、打印机是否缺纸、磁盘数据校验是否正确等。
- 3) 控制和定时。接收主机侧送来的控制信息和定时信号，根据相应的定时和控制逻辑，向外设发送控制信号，控制外设工作。主机送来的控制信息存放在控制(命令)寄存器中。
- 4) 数据格式的转换。提供数据格式转换部件(如进行串-并转换的移位寄存器)，将从外部接口接收的数据转换为内部接口所需格式，或进行反向的数据格式转换。

◎第六章 程序中的I/O操作实现

考点11 设备无关I/O软件层

1. 独立编址方式

独立编址方式对所有I/O端口单独编号，使它们成为一个与主存地址空间独立的I/O地址空间。采用该编址方式时，无法从地址码区分CPU访问的是I/O端口还是主存单元，因此指令系统中需要有专门的I/O指令表明访问的是I/O地址空间，I/O指令中地址码部分给出I/O端口号。CPU执行I/O指令时，会产生I/O读/写的总线事务，CPU通过该总线事务访问I/O端口。

◎第六章 程序中的I/O操作实现

考点11 设备无关I/O软件层

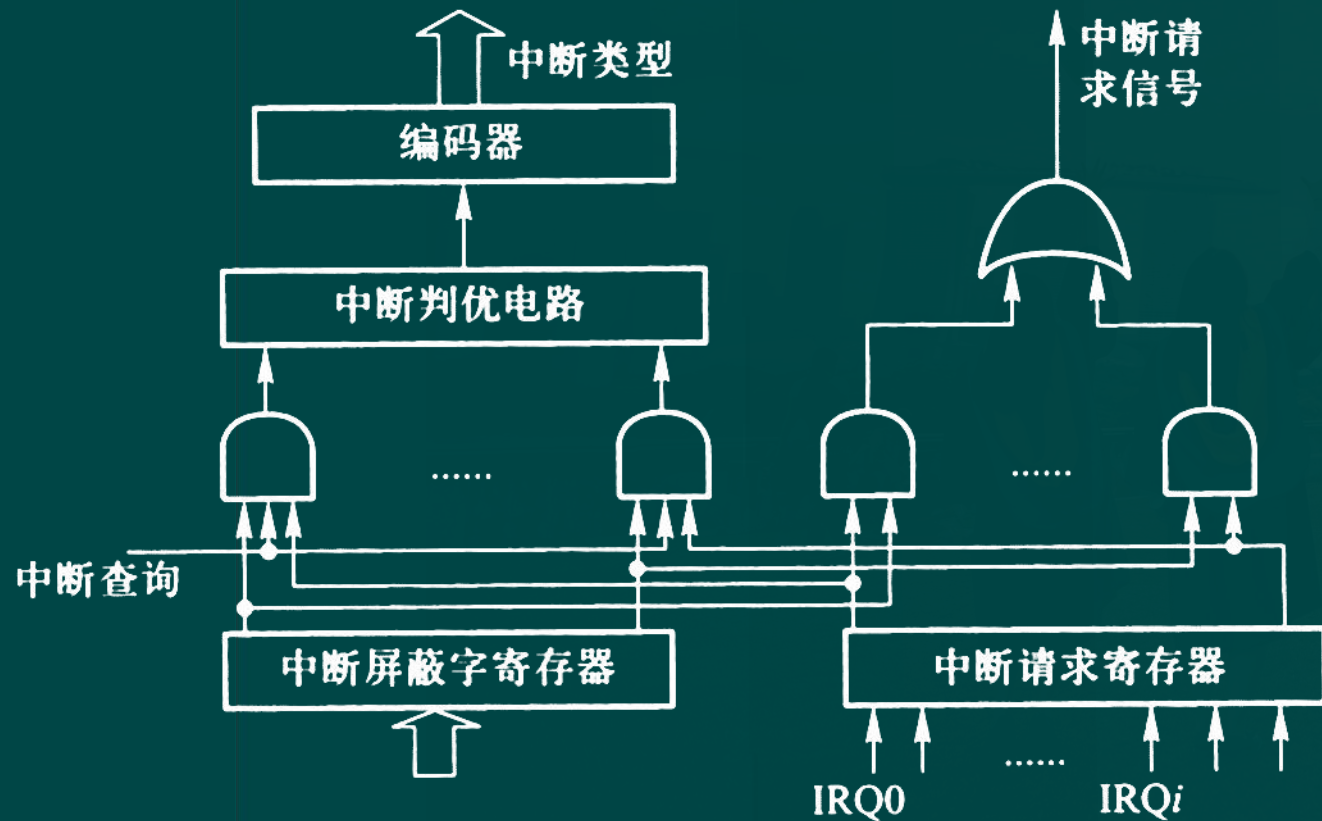
2. 统一编址方式

统一编址方式下，I/O地址空间与主存地址空间统一编址，主存地址空间分出一部分地址给I/O端口编号。因为I/O端口和主存单元在同一个地址空间的不同区域中，故可根据地址范围区分访问的是I/O端口还是主存单元，因此无须添加专门的I/O指令，只要用一般的访存指令即可访问I/O端口。因为这种方式是将I/O端口映射到主存空间的某段地址，所以也称为存储器映射方式。

◎第六章 程序中的I/O操作实现

考点12 中断系统

中断系统的基本结构如图



谢谢